

BitZ: proofs and commitments in arbitrary rings through binary fields

Remco Bloemen¹, Albert Garreta², Ulrich Haböck (?)¹, Marcin Kostrzewa³, Shreyas Londhe⁴, and John Wu²

¹World Foundation

²Nethermind Research

³Reilabs

⁴Ivy Research

September 9, 2026

Abstract

Contents

1	Introduction	2
1.1	Our contributions	4
1.2	Related work	7
1.3	Acknowledgements	8
2	New technical overview	9
3	Technical overview	9
3.1	A simple version of BitZ	9
3.1.1	Intuition of soundness proofs	12
3.2	Virtual $\mathbb{F}_2$ -linear transforms in BitZ and NP-complete dually linear relations	12
3.3	GKR for low-entropy batched grand products	14
3.3.1	Comparison to other “prove-in-the-exponent” usages	15
3.4	BitZ in full generality	16
4	Preliminaries	19
4.1	Algebra	19
4.2	Linear codes and relations	20
4.3	Interactive Oracle Reductions (IOR) of proximity	22

5	BitZ IOP for LinBitsRings	23
5.1	The core LinBitsRings relation	24
5.2	Tensor decomposition and products of constrained codes	25
5.3	The BitZ IOP for the core LinBitsRings relation	26
5.4	Soundness and completeness	31
5.5	The general LinBitsRings relation	35
5.6	The BitZ IOP for the general LinBitsRings relation	37
5.7	Instantiation	40
6	A GKR protocol for low entropy batched grand products	45
6.1	Prover optimization via lookup tables	49
7	Experiments	55
7.1	Raw performance of the BitZ PCS for the core LinBitsRings relation	55
7.2	Integer multiplication	56
7.3	RSA accumulation and Poseidon hashing	60
7.4	SHA-256 hashing followed by ECDSA signature verification	60
7.5	Mod 2^{32} multiplication and SHA-256 hashing	61
	References	61
A	Deferred proofs	63
A.1	Completeness and soundness of Construction 5.14	63
A.2	Soundness of Out Of Domain Sampling (OOD) in Construction 5.4	65
B	Ring switching via Galois orbits	66

1 Introduction

Albert: Interesting stuff to mention: 1) cost of BitZ do not depend on the ring, only on bit-size. 2) BitZ PCS can be used as drop-in replacement of any PCS? 3) Why more than

Albert: Need to discuss somewhere that range checks are free

Albert: discuss somewhere (technical overview) the succinctness/non-succinctness of verifier

Albert: List of reasons why the tensor decomposition is important: 1) low entropy grand product; 2) succinct verifier (otherwise it would need to compute an MLE for g^γ of full witness dimension); 3) achieving 128 bits of soundness while using $\mathbb{F}_{2^{128}}$ and largeish witness sizes: the tensor split allows us to only put inner products with $\sqrt{\text{witness dimension}}$ many terms (there's other things that contribute to achieving 128 bits of soundness, like ROUNDED 2 of Construction 5.4).

Albert: This is an important point. It should be highlighted more (this is a quote from the GKR section). "Crucially, we note that our sumcheck features summands of full field bit size $\log |\mathbb{K}|$, which voids some known powerful optimizations when the summands are small, e.g. the univariate skip [Gru24] and some optimizations from [DT24, BDDT25]."

Albert: Ultimately, for a proof system that requires proving a very large number of SHA (or similar) compressions, together with other type of arithmetic operations, e.g. integer or modulo 2^n , we recommend using a hybrid arithmetization and for constraints over $\mathbb{Z}$ and binary fields (in the

style of Zinc+ ??), with a witness $(\mathbf{f}_{\mathbb{Z}}, \mathbf{f}_2)$ split into a part $\mathbf{f}_{\mathbb{Z}}$ that is used in the integer constraints, and another part $\mathbf{f}_2$ used in the binary field constraints. Then the overall constraint is proved as:

1. Run a PIOP that is composed of a PIOP for integer constraints (as in ??) and a PIOP for binary field constraints (such as Binius64). This results in two MLE claims, one for (a projection of) $\mathbf{f}_{\mathbb{Z}}$ over a finite field, and another for $\mathbf{f}_2$ over a binary field.
2. Use BitZ to reduce the resulting MLE evaluation claim for (a projection of) $\mathbf{f}_{\mathbb{Z}}$ to an MLE evaluation claim over a binary field;
3. Use WHIR or another PCS over binary fields to prove the above claim, together with (in a batched manner) the MLE evaluation claim for $\mathbf{f}_2$.

A Succinct Non-interactive Argument of Knowledge (SNARK) for a relation $\mathcal{R}$ is a non-interactive protocol that enables a prover to convince a verifier that, for a given public input $\mathbf{x}$, they know a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, and where communication and verifier runtime are sublinear (i.e. the scheme is succinct).

In the vast majority of scenarios, $\mathcal{R}$ is defined over one (or more) algebraic object R , usually a finite field, and $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ if and only if $(\mathbf{x}, \mathbf{w})$ consists of elements that satisfy some polynomial equations. The choice of R has a crucial impact on the design and end-to-end performance of the proof system, for two reasons:

1. Usually the computations one wants to prove are not native to R , so one has to perform a more or less complex emulation of the computation into polynomial equations over R . This can affect the size of the input-witness pair $\mathbf{x}, \mathbf{w}$ significantly [?]. Additionally, the emulation step can be technically involved and prone to security errors [?].
2. The prover and verifier algorithms efficiency and design is affected by the choice of R : for example, modern CPUs are more performant for operations in certain small fields (e.g. M31 [?, HLP24], Babybear [?], Goldilocks [?]) than in arbitrary fields, especially if these are larger. Importantly, another class of fields where CPUs operate exceptionally fast are *binary fields*, i.e. fields of order a power of two.

Recently, two independent lines of research have received increased attention:

- Designing proof systems for relations expressed over $R = \mathbb{Z}$ the ring of integers, and over other rings that are not fields [CHA24, GWV⁺25, AGK⁺26, SSPP26, CXNB26, ?]. The motivation behind this is to address Item 1 above: in many practical scenarios, the computation to be proved is most succinctly and easily expressed over such rings, see Remark 3.4 Albert: more refs?.
- Designing proof systems for $R = \mathbb{F}_{2^\nu}$ a binary field, for example [BSBHR18b, ?, DP24, ?, BRW26]. The reason for this is twofold: first, some computations like classic hashes (SHA-256, Blake, etc.) are easier to express over $\mathbb{F}_{2^\nu}$ than over other structures such as prime fields (however, as we discuss in ??, these hash functions remain relatively un-native to $\mathbb{F}_{2^\nu}$ due to the hardness of emulating operations modulo 2^{32} or 2^{64}). Second, binary fields are highly performant on modern CPU's, and further enable using a boon of protocol techniques that further benefit performance, most importantly *packed* polynomial commitment schemes [DP23, DP24].

1.1 Our contributions

Albert: It should be very clear throughout the paper: The PIOP machinery is for the most part Zinc+’s or Zaratan. The dual field PCS primitive is novel; its instantiation is pretty much novel (though ideas can be traced back to [Bin26, Sou26]), and its enabling optimizations are novel. The PCS in binary fields is completely known.

In this work we bring these two lines of research together. We present $\mathbb{BitZ}$, a hash-based proof system that:

1. $\mathbb{BitZ}$ can prove polynomial constraints over essentially arbitrary rings R , including $\mathbb{Z}$, prime fields, lattices, binary fields, arbitrary field extensions, polynomial rings, etc., addressing Item 1 above ($\mathbb{BitZ}$ can handle constraints over different rings at once).
2. In its initial commitment step, $\mathbb{BitZ}$ ’s decomposes the witness w into a vector of bits $\text{bits}(w)$, and commits to $\text{bits}(w)$ over a binary field $\mathbb{F}_{2^\nu}$ with an error correcting code (leveraging the packing technique from [DP23, DP24]).
3. The initial polynomial constraints over the ring(s) R are reduced to an MLE evaluation claim (or more generally, an inner product claim) over a finite field $\mathbb{F}$, of the form $\langle \mathbf{v}, \pi(w) \rangle = \mu \in \mathbb{F}$, where π is a projection from R to $\mathbb{F}$ and $\mathbf{v}$ is a vector of elements from $\mathbb{F}$. The field $\mathbb{F}$ can be a fixed field extension, a random prime field (following [CHA24]), or a fixed prime field, depending on what the ring(s) R is (are).
4. $\mathbb{BitZ}$ ’s Polynomial Commitment Scheme (PCS) enables proving such linear claim efficiently. It operates, for the most part, over $\mathbb{F}_{2^\nu}$. Thus, $\mathbb{BitZ}$ benefits of many of the advantages of running proof systems over binary fields (Item 2 discussed earlier one). See below for a brief description of $\mathbb{BitZ}$ ’s PCS.

$\mathbb{BitZ}$ provides, among others, a compiler that takes any PCS over $\mathbb{F}_{2^\nu}$ and transforms it into a PCS over any of the above rings R , following the “compiler” frameworks of [SSPP26, CXNB26].

We implement and open-source $\mathbb{BitZ}$ [?], and use it to prove a number of computations of practical interest. In ?? we provide a summary of the results. In all computations but one, $\mathbb{BitZ}$ outperforms the current state-of-the-art approaches, sometimes by orders of magnitudes (for example: ***). $\mathbb{BitZ}$ is even able to prove Babybear multiplications more efficiently than Plonky3, a mature proof system that operates natively on the Babybear field, and which is used in industry within a variety of projects [?]. Albert: Mention what is the security parameter

We manage to instantiate our protocol using $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$ in GHASH representation, attain 128 bits of security, and have no limitation on the size of the ring R we start with. See ?? and Section 5.7 for more comments and details on this. Albert: For BabyBear bench, give separate numbers for PIOP part and the rest

Computation	BitZ	Other schemes
SHA-256 (7KB) + ECDSA		Albert: Zinc+, Vega, Longfewllow
2^{17} 32 bit integer mults		Albert: Binius, Limber, Zinc+
2^{20} Babybear mults		Albert: Plonk3
RSA accumulator + Poseidon has (MultiSwap [?], ??) [CXNB26]		Albert: Limber
Falcon		
SHA-256 in 8KB message (2^7 compressions)		Albert: flock/binius
SHA-256 in 2MB message (2^{15} compressions)		Albert: flock/binius

Table 1: Albert: Include memory. Albert: Revisit: should we include the experiemnts on hashing big messages here? Performance of BitZ on a suite of various computations of interest. For each such computation, we also benchmark the performance of prior state of the art protocols. All experiments were run on a MacBook Air M4 16GB, multi-threaded. Albert: Mention what is the security parameter Albert: Maybe report numbers for both the AI and human implementation

BitZ’s PCS in a nuthsell. BitZ proves the aforementioned claim $\langle \mathbf{v}, \pi(\mathbf{w}) \rangle = \mu$ over a finite field $\mathbb{F}$ as follows. We describe first an initial, mostly insufficient, attempt.

1. **Bitification.** First, it transforms the claim into a claim of the form $\langle \mathbf{u}, \pi(\text{bits}(\mathbf{w})) \rangle = \mu$ over $\mathbb{F}$, for some $\mathbf{u}$.
2. **Lift to integers.** Assuming $\mathbb{F}$ is a prime field of size q and $\pi_q : \mathbb{Z} \rightarrow \mathbb{F}$ is reduction modulo $|\mathbb{F}|$, the prover sends $\mu' \in \mathbb{Z}$ such that $\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle = \mu'$ in $\mathbb{Z}$.
3. **Proof in the exponent of $\mathbb{F}_{2^\nu}$.** Now take a generator g of the multiplicative subgroup $\mathbb{F}_{2^\nu}^*$ of $\mathbb{F}_{2^\nu}$, and prove that $g^{\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle} = g^{\mu'}$ in $\mathbb{F}_{2^\nu}$. Concretely, we have

$$g^{\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle} = \prod_i \left((g^{\pi_q^{-1}(\mathbf{u}_i)} - 1) \cdot \text{bits}(\mathbf{w})_i + 1 \right) = g^{\mu'}, \quad (1)$$

which is a polynomial expression over $\mathbb{F}_{2^\nu}$ on the vector $\text{bits}(\mathbf{w})$ that we committed over $\mathbb{F}_{2^\nu}$.

This is a standard situation where there is no “wrong-field” arithmetic, hence in principle one can use any proof system over $\mathbb{F}_{2^\nu}$ to prove (1).

There are several issues with this attempt:

- First, $\mathbb{F}$ may not be a prime field, in which case there is no natural lift to integers in Step 2; rather, there is a lift to integer polynomials $\mathbb{Z}[X]$.
- Second, even if $\mathbb{F}$ is a prime field, $q = |\mathbb{F}|$ may be large and the linear expression $\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle$ may overflow the order of $g \in \mathbb{F}_{2^\nu}^*$, which is $2^\nu - 1$. In this case, proving $\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle = \mu'$ “in the exponent of g ” is unsound.
- Third, even if neither of these problems hold, (1) is, in principle, an expensive constraint to prove: indeed, it is a length $|\text{bits}(\mathbf{w})|$ grand product of, essentially, arbitrary factors from $\mathbb{F}_{2^\nu}$ (typically ≈ 128 bit elements). Grand products can be expensive to prove, and furthermore the fact that the factors are large prevents us from using known sumcheck optimizations such as univariate skips [Gru24] and Albert: what else?.
- Four, when proving (1), typically one ends up with an MLE evaluation claim on the vector $(g^{\pi_q^{-1}(\mathbf{u}_i)})_i$ (and another on $\text{bits}(\mathbf{w})$, which is proved with any PCS over $\mathbb{F}_{2^\nu}$). If the verifier checks the claim itself, then it pays a linear cost. If the prover proves this claim, then it is essentially back to square one.

We solve these issues as follows: 1) if $\mathbb{F}$ is a proper extension, at Step 2 we lift to a claim

$\langle \pi_{\text{canon}}^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle = \mu' \in \mathbb{Z}[X]$ over $\mathbb{Z}[X]$, where $\pi_{\text{canon}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$ is a canonical projection of $\mathbb{Z}[X]$ onto $\mathbb{F}$. Then the verifier sends a random prime q' of $\approx |\mathbb{F}_{2^\nu}^*|/|\text{bits}(\mathbf{w})|$ size (small enough to avoid the issue of overflowing in the exponent), and a random element $\alpha \in \mathbb{F}_{q'}$; then the previous claim is replaced by $\langle \pi_{q',\alpha} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}), \pi_{q',\alpha}(\text{bits}(\mathbf{w})) \rangle = \pi_{q',\alpha}(\mu') \in \mathbb{F}_{q'}$ where $\pi_{q',\alpha}(f(X)) = \pi_{q'}(f)(\alpha) \in \mathbb{F}_{q'}$. Now we re-enter Step 2 with this new claim.

2) If $\mathbb{F}$ is too large of a prime field, we follow a similar approach as above: once we have the integer claim $\langle \pi_q^{-1}(\mathbf{u}), \text{bits}(\mathbf{w}) \rangle = \mu'$, the verifier samples a random “small” prime q' , reduce the claim modulo q' , and then re-enter Step 2 with this new claim.

Albert: remove? We emphasize again that we manage to instantiate BitZ with 128 bits of security, using $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$, while having no limitation on the size of the ring R we start with.

3) The main idea is to split (1) into $\approx \sqrt{|\text{bits}(\mathbf{w})|}$ grand products of the same form as (1), so that, even though in total there are $|\text{bits}(\mathbf{w})|$ factors among all of them, these factors can only take $\sqrt{|\text{bits}(\mathbf{w})|}$ values. More concretely, we assume $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ tensor-decomposes into $\approx \sqrt{|\text{bits}(\mathbf{w})|}$ sized factors (this is typically the case since $\mathbf{v}$ is usually an “eq” vector), and then, assuming the notation of Step 3 above, we aim to prove $\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{w})_{*j} \rangle = \mu'_j$ in $\mathbb{Z}$ for all $j \in [|\mathbf{u}^{(2)}|]$, such that $\sum_j \pi_q(\mu'_j) \cdot \mathbf{u}_j^{(2)} = \mu$, where we index $\text{bits}(\mathbf{w})$ with indices $i \in [|\mathbf{u}^{(1)}|], j \in [|\mathbf{u}^{(2)}|]$. We then set up a grand product for each of these claims, obtaining

$$g^{\langle \pi_q^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{w})_{*j} \rangle} = \prod_i (g^{\pi_q^{-1}(\mathbf{u}_i^{(1)})} - 1) \cdot \text{bits}(\mathbf{w})_{ij} + 1 = g^{\mu'_j} \quad \text{for all } j. \quad (2)$$

There are $|\text{bits}(\mathbf{w})|$ factors in total, but only $\approx \sqrt{|\text{bits}(\mathbf{w})|}$ different values these factors can take.

We say (2) is a *low entropy batched grand product*. To prove (2) we use a batched GKR protocol with the following optimization, among others: because of the low-entropy phenomenon, we build several small lookup tables of size $O(\sqrt{|\text{bits}(\mathbf{w})|})$ (of a few MB at most) which are used during the most expensive sumcheck rounds of the GKR protocol. Concretely, the last sumcheck (the most expensive one), uses 3 lookup tables to compute the first three messages of the prover; the second last sumcheck does the same for the first two prover messages (with two tables); and the third last sumcheck does the same for the first prover message (with one table).

The above solution also takes care of Issue 4: the batched GKR protocol outputs MLE evaluation claims on $(g^{\pi_q^{-1}(\mathbf{u}^{(1)})_i})_i$ (and $\text{bits}(\mathbf{w})$, which are dealt with the PCS over $\mathbb{F}_{2^\nu}$). This is an $O(\sqrt{|\text{bits}(\mathbf{w})|})$ vector, and the verifier can check this claim by itself cheaply.

Virtual $\mathbb{F}_2$ -linear transforms, NP-complete dually linear relations, and arithmetization of SHA-256 Observe that, once we have a standard linear claim on $\text{bits}(\mathbf{w})$ in $\mathbb{F}_{2^\nu}$, we could replace $\text{bits}(\mathbf{w})$ by a linear expression on some other committed vector $\mathbf{f}$ over $\mathbb{F}_{2^\nu}$, so that $\text{bits}(\mathbf{w}) = M \cdot \mathbf{f}$ for some matrix M . The final linear claim can then be replaced by a linear claim for $\mathbf{f}$. Hence, we can, instead of committing to $\text{bits}(\mathbf{w})$, commit to $\mathbf{f}$ and treat $\mathbf{w}$ and $\text{bits}(\mathbf{w})$ as virtual vectors. See ?? for details.

This allows BitZ to prove polynomial constraints over R on vectors $\mathbf{w}$ that are a virtual $\mathbb{F}_2$ -linear combination of a committed vector $\mathbf{f} \in \mathbb{F}_{2^\nu}^{\ell'}$. For example, BitZ can efficiently prove the following R1CS-like relation:

$$\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z})$$

$$:= \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, A, B, C, M) \in \mathbb{F}_2^k \times (\mathbb{Z}^{m \times \ell})^3 \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \mid \begin{array}{l} (A \cdot \mathbf{w}) \circ (B \cdot \mathbf{w}) = C \cdot \mathbf{w} \text{ in } \mathbb{Z} \\ \mathbf{w} = \pi_2^{-1}(M \cdot (\mathbf{z}, \mathbf{f})) \end{array} \right\},$$

where π_2 denotes reduction modulo 2. A particularly interesting subclass of this relation is that obtained by setting $A = B = \mathbf{0}$, in which case the resulting relation, which we call the *CM relation*, degenerates to linear constraints over $\mathbb{Z}$ and over $\mathbb{F}_2$, i.e. a “dually linear” constraint.

In [Remark 3.3](#) we show that the CM relation is NP-complete [Albert: Or NP-hard?](#). In ?? we arithmetize SHA-256 as an instance of the CM relation, with *** smaller witnesses than Binius64 [\[Bin26\]](#) and Flock [\[BRW26\]](#) [Albert: add numbers. Mention that we don't prove hashes faster though \(but we have better proof size and memory so...\)](#)

1.2 Related work

Proof systems over integers and other rings. In [\[CHA24\]](#), Campanelli and Hall-Andersen propose proving polynomial constraints over integers, by projecting them onto a random prime field (sampled by the verifier), a technique which the authors call *fingerprinting*. Subsequent works have adopted this idea [\[GWV⁺25, SSPP26, AGK⁺26, CXNB26\]](#). [\[CHA24\]](#) also proposes a hidden-order group based PCS suitable for their PIOP, similar to the ones in [\[BFS19, BHR⁺21\]](#). The PCS is not implemented and so we do not compare against it.

The Zinc [\[GWV⁺25\]](#) work adopts *fingerprinting* and designs a hash-based PCS over the rationals $\mathbb{Q}$, suitable for the fingerprinting approach, providing the first relatively practical proof system over the integers. [\[GWV⁺25\]](#) addresses subtle technical issues that arise when aiming to commit to *integer* vectors with error correcting codes (namely, malicious provers can potentially use rational numbers that are very close to being integers of the target magnitude).

The Zinc+ [\[AGK⁺26\]](#) work extends [\[GWV⁺25\]](#) in two directions: 1) It proposes a constraint system that includes polynomial constraints simultaneously over many types of rings¹ and fields, and provides a method for constructing PIOP’s for those. This method extends the fingerprinting approach by not only projecting $\mathbb{Z}$ onto a random prime field, but projecting all sorts of rings and fields onto a random prime field. 2) It improves the soundness analysis of the PCS in [\[GWV⁺25\]](#), plus it proposes a new linear code over $\mathbb{Z}$ or $\mathbb{Q}$, called Integer Pseudo Reed-Solomon (IPRS) code, obtained by “lifting” (in a suitable way) an FFT encoding algorithm over a small finite field, to the integers. [Albert: compare BitZ to Zinc+?](#)

In [\[SSPP26\]](#) the authors propose Qaren, a compiler that takes any PCS over a finite field of sufficiently large characteristic and creates a PCS for integer polynomials (or, more precisely, rational polynomials close to the target integer polynomials, see the overview of [\[GWV⁺25\]](#) above). The authors instantiate their compiler to obtain the first transparent and fully succinct PCS over integers/rationals. The prior work [\[GWV⁺25, AGK⁺26\]](#) is not succinct on the bit-size of the integers being committed. No implementation or performance reports is provided in [\[SSPP26\]](#).

The concurrent work Limber [\[CXNB26\]](#) proposes a new compiler that transforms any PCS over fields of sufficiently large characteristic q (in this case, q is $\Omega(\lambda^2 \log(N))$, N being the witness size, while in [\[SSPP26\]](#) it is $\Omega(2^\lambda)$). Additionally Limber compiles into strictly integer PCS, and does not present the issue that a malicious prover could use rationals that are “almost integers”. Limber is instantiated in practice with the Hyrax PCS (in contrast to [\[GWV⁺25, AGK⁺26\]](#) which use a hash

¹In the terminology of [\[?\]](#), the constraints are over *polynomial rings with ideal membership constraints*, for certain types of ideals (generated by a monic and non-constant polynomial, or zero). However this is equivalent to saying the constraints are over different types of rings (within a certain class)

based PCS). **Albert:** Wait for the implementation to come out so we can discuss its performance against prior work better. In [?] we compare BitZ against Limber, obtaining *** **Albert:** fill in.

Proof systems over binary fields. Hash-based SNARKs over binary fields were pioneered by the line of work of Ben-Sasson et al. [BSBHR18a, BSBHR18b, ?, ?], but did not see widespread adoption until the introduction of *packed* polynomial commitment schemes by Diamond and Posen [DP23, DP24]; see also [Bin26]. Such schemes allow one to commit to a bit-valued witness over an extension field without incurring embedding overhead, while still providing evaluation proofs for its original “uncompressed” representation. The *ring-switching* technique of Diamond and Posen [DP24] is generic, enabling the use of codes over larger alphabets without introducing representational redundancy.² A particularly attractive choice is Reed–Solomon codes over $GF(2^k)$ [Bin26, ?], which offer an excellent trade-off between distance and encoding cost, although several works [BCF⁺24, ?, ?] explore this design space using more efficient codes.

At the time of writing, two further works push the performance envelope of binary proof systems. Bünz, Rothblum, and Wang [BRW26] combine [Bin26] with the batched R1CS strategy inspired by [?], achieving a striking proving throughput of several hundred thousand hashes per second on an M4 Max CPU. BinarySpartan [?], in turn, incorporates the optimizations of [Bin26]-Flock into Spartan, resulting in a proof system with again high hash-proving throughput.

Commitments in binary fields with openings in rings R of characteristic different than 2. To our knowledge this primitive is studied only in Soukhanov’s note [Sou26] and in the present work. The note [Sou26] proposes a scheme for committing over the binary field $\mathbb{F}_{2^{127}}$ and then executing a PCS opening over $\mathbb{F}_{2^{127}-1}$ (a prime field), by running the PCS in the exponent of a generator of $\mathbb{F}_{2^{127}}^*$. The approach, substantially different to ours, is limited to $R = \mathbb{F}_{2^{127}-1}$ (or analogous configurations where $2^n - 1$ is prime) and does not seem to be able to leverage our aforementioned “low-entropy” based optimizations. The note does not provide an implementation nor performance reports. We discuss this further in Section 3.3.1.

Qaren and Limber [SSPP26, CXNB26] provide a related primitive when $R = \mathbb{Z}$, but can only support commitments over fields of relatively large characteristic (compared to 2).

Albert: more? Swirl, Frepack?

1.3 Acknowledgements

We thank the engineering teams at World Foundation, Nethermind, and Ivy Research for their contributions towards implementing BitZ, concretely we thank Alexander Abdugafarov, Xander van der Groot, and Sina **Albert:** Last name?. We also thank Lev Soukhanov and Giacomo Fenzi for insightful and helpful discussions.

Albert: World Foundation

Usage and non-usage of AI This document was written with minimal assistance of AI. It was only used to identify typos, make refactorings, and for cost accountings in Section 6 and Sections 5.3 and 5.5. **Albert:** I also used it in the standard appendix proofs

We used AI in our implementation and experiments, as explained in Section 7.

²Alternative approaches based on Galois conjugates have been proposed thereafter [?, ?]. Remarkably, the Galois approach is present in prior work [BSBHR18b, Appendix F], although not explicitly framed in full generality.

2 New technical overview

1. Start with PCS with $\mathbf{f} \in \mathbb{Z}^{\ell_0}$ and R is a prime field.
2. Then low entropy grand product.
3. Generalizations of PCS. $\mathbf{f} \in S^\ell$ and R an extension field (e.g. Babybear proving) or a f.g. ring (for the sake of lattices).
4. General proof system.
5. Virtual $\mathbb{F}_2$ -linear transforms in $\mathbb{Bit}\mathbb{Z}$ and NP-complete dually linear relations.

3 Technical overview

3.1 A simple version of $\mathbb{Bit}\mathbb{Z}$

We explain how $\mathbb{Bit}\mathbb{Z}$ works. Throughout the exposition we make several simplifying assumptions, for the sake of clarity. Then we address the general cases in [Section 3.4](#).

Let R be a ring, for example R may be the integers $\mathbb{Z}$, a prime field, a lattice ring, a finite field extension, etc. For simplicity, for now let R be either $\mathbb{Z}$ or a finite field $\mathbb{K}$ (possibly an extension field). In [Section 3.4](#) we treat the general case. If $R = \mathbb{Z}$, then we assume a public parameter B specifying the maximum bit-size of a witness element.

In $\mathbb{Bit}\mathbb{Z}$, given a public vector $\mathbf{x} = \mathbf{x} \in R^k$, the prover wants to prove it knows $\mathbf{w} = \mathbf{w} \in R^n$ such that

$$\mathbf{p}_i(\mathbf{x}, \mathbf{w}) = 0 \quad \text{in } R \quad \text{for all } i \in [m], \quad (3)$$

where $\mathbf{p} = (\mathbf{p}_i)_{i \in [m]}$ is a collection of polynomials over R . For example, in an *R1CS constraint*,

$$\mathbf{p}_i(\mathbf{x}, \mathbf{w}) = (A \cdot (\mathbf{x}, \mathbf{w}))_i \cdot (B \cdot (\mathbf{x}, \mathbf{w}))_i - (C \cdot (\mathbf{x}, \mathbf{w}))_i$$

where A, B, C are $m \times (k + n)$ matrices over R .

Step 1: Commitment in $\mathbb{F}_{2^\nu}$. The prover creates a vector $\text{bits}(\mathbf{w}) \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ of integer bits, according to some public specification of how to represent elements $r \in R$ as a linear combination of bits, i.e. $r = L(\text{bits}(r))$ for $L : \{0, 1\}^{\ell/n} \rightarrow R$ a linear map. For example, if $R = \mathbb{Z}$, then $L(b_1, \dots, b_B) = \sum_{i \in [B]} 2^{i-1} \cdot b_i$.

Then the prover looks at $\text{bits}(\mathbf{w})$ as a vector from $\mathbb{F}_2^\ell$, encodes it with a Reed-Solomon code (or any other code) over $\mathbb{F}_{2^\nu}$, and sends a Merkle root of the resulting codeword to the verifier.

When encoding $\text{bits}(\mathbf{w})$, every ν bits $b_1, \dots, b_\nu$ are packed as a single field element $\sum_{i \in [\mathbb{F}_{2^\nu}]} X^{i-1} \cdot b_i$ from $\mathbb{F}_{2^\nu}$, following [\[DP23\]](#) (one can use a different $\mathbb{F}_2$ -base than the monomial base $(X^i)_i$).

Albert: We may want to mention that we do an OOD sample if we are working beyond the UDR

Step 2: Projection to a finite field $\mathbb{F}$. **Albert:** If necessary, the verifier samples a random map π from R to a finite field $\mathbb{F}$ of $\approx \lambda$ bit size (not necessarily a prime field). Then the claim (3) is replaced by

$$\pi(\mathbf{p}_i)(\pi(\mathbf{x}), \pi(\mathbf{w})) = 0 \quad \text{in } \mathbb{F} \quad \text{for all } i. \quad (4)$$

The form of the samplable maps π depends on the type of ring R we started with. For example, if $R = \mathbb{Z}$, then π is reduction modulo a random prime q , following [CHA24]. If $R = \mathbb{K}$ is a finite field (not necessarily a prime field), then π is always the identity. In Section 3.4 we discuss other rings and other projections.

This step can be skipped if one already has an efficient PIOP for polynomial constraints over R ; for example, if R is a finite field of reasonable size.

If R is a finite field of very large size, one may prefer to lift the initial polynomial constraints to $\mathbb{Z}$ first (or $\mathbb{Z}[X]$ if R is field extension), and then project onto a random prime field $\mathbb{F}$ of bit-size $\approx \lambda$ during this step. This can compress the bit-size of the witness in the next steps by a very large factor.

Step 3: PIOP over $\mathbb{F}$. As in [CHA24, AGK⁺26, CXNB26], a PIOP is run over $\mathbb{F}$, reducing (4) to a linear claim over $\mathbb{F}$ or a field extension $\mathbb{F}_{\text{ext}(e)}$ of $\mathbb{F}$. The claim is of the form

$$\langle \mathbf{v}, \pi(\mathbf{w}) \rangle = \mu \quad \text{in } \mathbb{F}_{\text{ext}(e)}, \quad (5)$$

for some $\mu \in \mathbb{F}_{\text{ext}(e)}$, $\mathbf{v} \in (\mathbb{F}_{\text{ext}(e)})^\ell$.

In general, if $\mathbb{F}$ is a prime field of $\approx \lambda$ bits (e.g. if we started with $R = \mathbb{Z}$), then $\mathbb{F}_{\text{ext}} = \mathbb{F}$. The field $\mathbb{F}_{\text{ext}}$ may be a proper extension if, for example, we started with $\mathbb{R} = \mathbb{K}$ a small finite field (in which case $\pi = \text{Id}$ and $\mathbb{K} = \mathbb{F}$), and the PIOP had to use challenges in an extension of $\mathbb{F}$ to guarantee soundness.

We assume $\mathbf{v}$ admits a tensor decomposition $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \in \mathbb{F}_q^{n_1 \cdot n_2}$. This is typically the case since $\mathbf{v}$ is usually an “eq” vector. This will be leveraged crucially, for efficiency purposes, at the later Step 5.2.

For simplicity, we assume $\mathbb{F}_{\text{ext}} = \mathbb{F}$ is a prime field. We discuss the general case in Section 3.4.

Step 4: Bitification of the linear claim. The claim (5) is replaced by

$$\langle \mathbf{u}, \pi(\text{bits}(\mathbf{w})) \rangle = \mu \quad \text{in } \mathbb{F}, \quad (6)$$

for a new $\mathbf{u} \in \mathbb{F}^\ell$. This transformation is done using the linear bit representation L mentioned in Step 1. Concretely, representing L as matrix multiplication and indexing vectors appropriately, $\langle \mathbf{v}, \pi(\mathbf{w}) \rangle = \langle \mathbf{v}, \pi(L \cdot \text{bits}(\mathbf{w})) \rangle = \langle \pi(L)^\top \cdot \mathbf{v}, \pi(\text{bits}(\mathbf{w})) \rangle$.

The new vector $\mathbf{u}$ still admits tensor decompositions $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with, say $\mathbf{u}^{(1)} \in \mathbb{F}^{\ell_1}$ and $\mathbf{u}^{(2)} \in \mathbb{F}^{\ell_2}$. Indeed, one can see that $L = \text{Id}_{n \times n} \otimes L'$ for some vector $L' \in R^{\ell/n}$ and Id the identity matrix of dimension n . Then for example $\mathbf{u}^{(1)} = \mathbf{v}^{(1)}$ and $\mathbf{u}^{(2)} = \mathbf{v}^{(2)} \otimes \pi(L')$, see Remark 5.15 for details **Albert: Review. Maybe I could prove it here.** Other choices of decompositions may lead to better efficiency later.

We assume that $|\mathbb{F}| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, a condition needed for soundness later in Step 5.2. Below in Remark 3.1 we explain how to handle larger $\mathbb{F}$. We note that this is a very constraining assumption because, when for example aiming to use $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$ (a highly performant binary field) and 128 bits of security, one can easily have $\mathbb{F}$ with $|\mathbb{F}| \geq (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$.

Step 5: BitZ Polynomial Commitment Scheme (PCS). Our BitZ PCS enables efficiently proving (6) with $\text{bits}(\mathbf{w})$ being committed over $\mathbb{F}_{2^\nu}$ (in packed form). This is done as follows:

Step 5.0: reducing the prime modulus if necessary. This step triggers if the condition $|\mathbb{F}| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ is not met. We explain this step below in Remark 3.1 and assume for now that the condition holds.

When $\mathbb{F}_{\text{ext}}$ is a proper field extension, this step always triggers.

Step 5.1: Transformation of (6) into a batch of low-magnitude integer claim. The prover organizes $\text{bits}(\mathbf{w})$ into a $\ell_1 \times \ell_2$ matrix with columns $\text{bits}(\mathbf{w})_{*j} \in \{0, 1\}^{\ell_1}$ ($j \in [\ell_2]$). Then it publishes $\eta_1, \dots, \eta_{\ell_2} \in \mathbb{Z}$ defined as $\eta_j = \langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{w})_{*j} \rangle$ in $\mathbb{Z}$, for all $j \in [\ell_2]$. Then the verifier checks that [Albert: Add a remark about the possibility of proving the below with a sumcheck](#)

$$\sum_{j \in [\ell_2]} \pi(\eta_j) \cdot \mathbf{u}_j^{(2)} = \mu \quad \text{in } \mathbb{F},$$

and (6) is replaced by the integer claim

$$\langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{w})_{*j} \rangle = \eta_j \quad \text{in } \mathbb{Z} \quad \text{for all } j \in [\ell_2], \quad (7)$$

where π^{-1} is the canonical representation of elements from $\mathbb{F}$ as integers in the range $[0..|\mathbb{F}| - 1]$.

Step 5.2: Exponentiation in $\mathbb{F}_{2^\nu}$ and grand product in $\mathbb{F}_{2^\nu}$. Since we assumed $|\mathbb{F}| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, (7) is true if and only if

$$\begin{aligned} g^{\eta_j} &= g^{\langle \pi^{-1}(\mathbf{u}^{(1)}), \text{bits}(\mathbf{w})_{*j} \rangle} \\ &= g^{\sum_{i \in [\ell_1]} \pi^{-1}(\mathbf{u}_i^{(1)}) \cdot \text{bits}(\mathbf{w})_{ij}} = \prod_{i \in [\ell_1]} g^{\pi^{-1}(\mathbf{u}_i^{(1)}) \cdot \text{bits}(\mathbf{w})_{ij}} \\ &= \prod_{i \in [\ell_1]} \left(\left(g^{\pi^{-1}(\mathbf{u}_i^{(1)})} - 1 \right) \cdot \text{bits}(\mathbf{w})_{ij} + 1 \right) \quad \text{in } \mathbb{F}_{2^\nu} \quad \text{for all } j \in [\ell_2], \end{aligned} \quad (8)$$

[Xander: superscript missing \$\mathbf{u} = \mathbf{u}^{\(1\)}\$](#)

[Albert: Yes, thanks!](#) where g is a generator of the multiplicative subgroup of $\mathbb{F}_{2^\nu}$, and in the middle expression, $\text{bits}(\mathbf{w})_{ij}$ denotes the i, j -th entry (a bit) in $\text{bits}(\mathbf{w})$ seen as an element of $\mathbb{F}_{2^\nu}$.

The prover and verifier run a batched grand product PIOP over the binary field $\mathbb{F}_{2^\nu}$ for the claim (8), obtaining as a result a new claim of the form

$$\langle \mathbf{z}, \text{bits}(\mathbf{w})_{*j} \rangle = c_j \quad \text{in } \mathbb{F}_{2^\nu} \quad \text{for all } j \in [\ell_2], \quad (9)$$

for some $\mathbf{z} \in \mathbb{F}_{2^\nu}^{\ell_1}$ and $c_j \in \mathbb{F}_{2^\nu}$. In general, these claims can be batched to

$$\langle \mathbf{z}', \text{bits}(\mathbf{w}) \rangle = c \quad \text{in } \mathbb{F}_{2^\nu} \quad (10)$$

for $\mathbf{z}' \in \mathbb{F}_{2^\nu}^\ell$, $c \in \mathbb{F}_{2^\nu}$ with, for example, $\mathbf{z}' = (\alpha_1, \dots, \alpha_{\ell_2}) \otimes \mathbf{z}$ for $\alpha_1, \dots, \alpha_{\ell_2}$ verifier's randomness, and $c = \sum_{j \in [\ell_2]} \alpha_j \cdot c_j$. However, in most instantiations this step may not be necessary, especially if using an interleaved code to commit to $\text{bits}(\mathbf{w})$ [Albert: In the GKR grand product section of the main body I may need to mention batching?](#)

This exponentiation trick is inspired by [Bin26, Sou26], where the same idea is used in different contexts, see [Section 3.3.1](#).

This is by far the most expensive step of BitZ. Our grand product PIOP is a simple GKR protocol for grand products, for which we design novel [Albert: to our knowledge](#) prover optimizations, see [Section 3.3](#) below and [Section 6](#). These optimizations exploit the “low entropy” of the factors $(g^{\pi^{-1}} - 1) \cdot \text{bits}(\mathbf{w})_{ij} + 1$ across $(i, j) \in [\ell_1] \times [\ell_2]$, in the sense that across $\ell = \ell_1 \cdot \ell_2$ factors, there are only $\ell_1 + 1 = O(\sqrt{\ell})$ possible values taken by these factors. This is why it is important to have the vectors $\mathbf{v}$ and $\mathbf{u}$ admit a tensor split into $\approx$ square-root sized factors.

Step 5.3: Proving linear claims over $\mathbb{F}_{2^\nu}$. At this point, since $\text{bits}(\mathbf{w})$ was committed over $\mathbb{F}_{2^\nu}$ (in packed form), one can prove (9) (or the batched version of it) via standard methods such as in [DP23, DP24, BRW26]. Concretely, we use the so-called ring switch from [DP23, DP24] to turn (9) into linear claims about the packed binary field elements we encoded, and then use WHIR [ACFY25] over $\mathbb{F}_{2^\nu}$ to prove these claims. See [Section B](#) and ?? for details on ring-switching.

In [Section 5.7](#) we discuss how to efficiently instantiate BitZ with $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$, and attain 128 bits of security.

Remark 3.1 (Description of Step 5.0). Assume that $\mathbb{F}_{\text{ext}} = \mathbb{F}$ is a prime field (the general case is described in [Section 3.4](#)), and that in Step 5.0 the condition $|\mathbb{F}| < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ does not hold. Then the prover sends an integer $\mu' \in \mathbb{Z}$ such that

$$\langle \pi^{-1}(\mathbf{u}^{(1)}) \otimes \pi^{-1}(\mathbf{u}^{(2)}), \text{bits}(\mathbf{w}) \rangle = \mu' \quad \text{in } \mathbb{Z},$$

The verifier checks that $\pi(\mu') = \mu$, and responds with a random prime q such that $q < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. Then prover and verifier proceed to Step 5.1 after updating $\mathbb{F} \leftarrow \mathbb{F}_q$, the prime field with q' elements, $\pi \leftarrow \pi_{q'}$, $\mu \leftarrow \pi_q(\mu')$, and $\mathbf{u} \leftarrow \pi_q \circ \pi^{-1}(\mathbf{u}^{(1)}) \otimes \pi_q \circ \pi^{-1}(\mathbf{u}^{(2)})$.

Remark 3.2 (Other usages of the “prove-in-the-exponent” approach). There are other projects (e.g. Binius64 [Bin26] and Shoukhanov’s note [Sou26]) that use exponentiation to prove integer (or mod $|\mathbb{F}_{2^\nu}| - 1$) claims in binary fields, none of these (to our knowledge) is used or is usable in a context that results in the low entropy grand products of ours, where our optimizations apply. We discuss this further in [Section 3.3.1](#).

3.1.1 Intuition of soundness proofs

We prove that the above protocol (and the more general version in [Section 3.4](#)) is round-by-round knowledge sound. We next provide the main ideas of why this is the case.

[Albert: todo](#)

3.2 Virtual $\mathbb{F}_2$ -linear transforms in BitZ and NP-complete dually linear relations

Observe that, at the end of the day, Step 5.2 outputs a linear claim of the form³

$$\langle \mathbf{z}, \text{bits}(\mathbf{w}) \rangle = c \quad \text{in } \mathbb{F}_{2^\nu}, \tag{11}$$

³We start with the batched version of the claim, but the ideas in this section can be adapted to (9) as well.

for some $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and $c \in \mathbb{F}_{2^\nu}$. This is a standard linear claim over $\mathbb{F}_{2^\nu}$ for the vector $\text{bits}(\mathbf{w})$ which we committed to previously.

Now, say that we know that $\text{bits}(\mathbf{w})$ is a linear transformation of a vector $\mathbf{f} \in \mathbb{F}_2^{\ell'} \subseteq \mathbb{F}_{2^\nu}^{\ell'}$, so that

$$\text{bits}(\mathbf{w}) = M \cdot \mathbf{f} \quad \text{in } \mathbb{F}_{2^\nu},$$

for some matrix $M \in \mathbb{F}_2^{\ell \times \ell'}$. Then

$$\langle \mathbf{z}, \text{bits}(\mathbf{w}) \rangle = \langle M^\top \cdot \mathbf{z}, \mathbf{f} \rangle = c \quad \text{in } \mathbb{F}_{2^\nu}. \quad (12)$$

Hence, in this scenario, one can commit to $\mathbf{f}$ instead of $\text{bits}(\mathbf{w})$ (again, in packed formed if necessary), replace the claim (12), and prove (12) in Step 5.3 of BitZ .

Thus, BitZ can be expanded so as to prove claims where we are given a public vector $\mathbf{x} = \mathbf{x} \in R^k$, and the prover claims it knows $\mathbf{w} = \mathbf{f} \in \mathbb{F}_{2^\nu}^{\ell'}$ such that

$$\begin{cases} \mathbf{p}_i(\mathbf{x}, \mathbf{w}) = 0 & \text{in } R \quad \text{for all } i \in [m], \\ \text{bits}(\mathbf{w}) = \pi_2^{-1}(M \cdot \mathbf{f}) \end{cases} \quad (13)$$

where $\mathbf{p} = (\mathbf{p}_i)_{i \in [m]}$ is a collection of polynomials over R ; and π_2 is reduction modulo 2 [Albert: I may have been loose about when bits\(w\) is in F2 and when in integers during the technical overview, review.](#)

A particularly interesting instantiation of this relation is the following R1CS-like relation over the integers, where $\mathbf{w}$ is a vector of bits (hence $\mathbf{w} = \text{bits}(\mathbf{w})$) and $\mathbf{w} = \pi_2^{-1}(M \cdot \mathbf{f})$:

$\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z})$

$$:= \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, A, B, C, M) \in \mathbb{F}_2^k \times (\mathbb{Z}^{m \times \ell})^3 \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \middle| \begin{array}{l} (A \cdot \mathbf{w}) \circ (B \cdot \mathbf{w}) = C \cdot \mathbf{w} \text{ in } \mathbb{Z} \\ \mathbf{w} = \pi_2^{-1}(M \cdot (\mathbf{z}, \mathbf{f})) \end{array} \right\}$$

An interesting subset of $\mathcal{R}_{\text{R1CS-bits-virtual}}(\mathbb{Z})$ is that where A and B are zero matrices. We call the resulting relation the $\mathcal{R}_{\text{CM}}$ relation because of the matrices C and M :

$$\mathcal{R}_{\text{CM}}(\mathbb{Z}) := \left\{ \begin{array}{l} \mathbf{x} = (\mathbf{x}, C, M) \in \mathbb{F}_2^k \times \mathbb{Z}^{m \times \ell} \times \mathbb{F}_2^{\ell \times (k + \ell')} \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^{\ell'} \subseteq \mathbb{F}_2^{\ell'} \end{array} \middle| \begin{array}{l} C \cdot \mathbf{w} = 0 \text{ in } \mathbb{Z} \\ \mathbf{w} = \pi_2^{-1}(M \cdot (\mathbf{x}, \mathbf{f})) \end{array} \right\} \quad (14)$$

In a sense, the CM relation places only linear constraints over its witness (with some constraints happening in $\mathbb{F}_{2^\nu}$ and other in $\mathbb{Z}$). Yet, the CM relation is NP-complete, as shown below in [Remark 3.3](#).

Indeed, in ?? we **arithmetize SHA-256 as a CM instance**. Our arithmetization leads to witnesses of size (in bits) **** [Albert: report comparison with Flock/Binius in witness size. Mention as well the performance comparison.](#)

Remark 3.3 (The $\mathcal{R}_{\text{CM}}$ relation is NP-complete). Any Boolean circuit can be arithmetized as an instance of $\mathcal{R}_{\text{CM}}$. Indeed, since any boolean circuit can be written as a system of equations using the operations XOR and AND, it suffices to prove that, given $x, y, z \in \{0, 1\}$, we can express both of the following constraints as an instance of $\mathcal{R}_{\text{CM}}$:

$$x \text{ XOR } y = z, \quad x \text{ AND } y = z.$$

Indeed, $x \text{ XOR } y = z$ if and only if $\pi_2^{-1}(\pi_2(x) + \pi_2(y)) = z$, where here $+$ means addition modulo 2. Further, $x \text{ AND } y = z$ if and only if

$$2 \cdot z = x + y - \pi_2^{-1}(\pi_2(x) + \pi_2(y)).$$

One can verify this last equivalence for example by directly inspecting that it holds for the four possible combinations of $x, y \in \{0, 1\}$.

Albert: Todo: Expressiveness result of Remco

3.3 GKR for low-entropy batched grand products

Recall that, in Step 5.2 of BitZ (Section 3.1) the prover must prove that

$$g^{\langle \pi^{-1}(\mathbf{u}), \text{bits}(\mathbf{w})_{*j} \rangle} = \prod_{i \in [\ell_1]} \left(g^{\pi^{-1}(\mathbf{u}_i)} - 1 \right) \cdot \text{bits}(\mathbf{w})_{ij} + 1 = g^{\eta_j} \quad \text{in } \mathbb{F}_{2^\nu} \quad \text{for all } j \in [\ell_2]. \quad (15)$$

For efficiency purposes, we assume both ℓ_1 and ℓ_2 are $O(\sqrt{\ell})$ (see Section 5.7 for exact parameter instantiations). At a protocol level, we simply prove (15) with a standard GKR protocol for grand products [?]. Concretely, let $\ell_1 = 2^{r_1}$ and $\ell_2 = 2^{r_2}$. We set up multilinear polynomials $L_0, L_1, \dots, L_{r_1}$ on $r_2 + i$ variables for $i \in [0..r_1]$, respectively, such that

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = \left(g^{\pi^{-1}(\mathbf{u}_i)} - 1 \right) \cdot \text{bits}(\mathbf{w})_{ij} + 1 \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2},$$

and, for $i = 0, \dots, r_1 - 1$

$$L_i(\mathbf{b}, \mathbf{c}) = L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \quad \text{for all } \mathbf{b} \in \{0, 1\}^i \times \{0, 1\}^{r_2}. \quad (16)$$

We then use a series of sumchecks to certify Eq. (16) for all $i = 0, \dots, r_1 - 1$, and $L_0(\mathbf{c}) = g^{\eta_{\text{int}(\mathbf{c})}}$ for all $\mathbf{c} \in \{0, 1\}^{r_2}$.

We emphasize again that, at a protocol level, this GKR scheme is completely standard. See Section 6 for all the details.

At the end of the GKR grand product, the prover and verifier are left with MLE evaluation claims of the form:

$$\widetilde{g^{\pi^{-1}(\mathbf{u})}}(\mathbf{r}) = a, \quad \widetilde{\text{bits}(\mathbf{w})_{*j}}(\mathbf{r}) = a_j \quad \text{for all } j \in [\ell_2] \quad (17)$$

for some $\mathbf{r} \in \mathbb{F}_{2^\nu}^{r_1}$, where $\widetilde{}$ denotes MLE extension, and $g^{\pi^{-1}(\mathbf{u})}$ is the vector with entries $g^{\pi^{-1}(\mathbf{u}_i)}$.

Since $\mathbf{u}$ is public, the verifiers checks the first claim of (17) on its own in time $O(\sqrt{\ell})$. This is one, among several, reasons why the tensor split of $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ and the splitting the original claim into a batch of ℓ_2 claims (see Step 5.1 of Section 3.1) pays off.

We design novel (to our knowledge **Albert: check**) prover optimizations for the GKR grand product (without altering the protocol itself), see Construction 6.1 for details. Namely, we exploit the following property, which is enabled, again, by the tensor split of $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ and the splitting the original claim into a batch of ℓ_2 claims:

Low entropy grand products: There are at most $\ell_1 = O(\sqrt{\ell})$ values that $L_{r_1}(\mathbf{b}, \mathbf{c})$ can take on the hypercube of size $\ell_1 \cdot \ell_2 = \ell$.

This property trickles down to different stages of the GKR protocol. For example, the above property implies that $L_{r_1-1}(\mathbf{b}', \mathbf{c})$ as well can take only an $O(\sqrt{\ell})$ amount of values (with worse constants now) across all $(\mathbf{b}', \mathbf{c}) \in \{0, 1\}^{r_1-1} \times \{0, 1\}^{r_2}$. Similarly, the coefficients of the first messages sent by the prover at the last stages of the GKR protocol (which involve the most expensive sumchecks), also benefit from a low entropy property.

This allows the prover to compute small lookup tables that are kept on cache and reused across all values of $\mathbf{c} \in [\ell_2]$. We build different lookup tables used at different stages of the GKR protocol. For example, we use 4 lookup tables in the first round of the last sumcheck of GKR (the most expensive sumcheck, being on $r_1 + r_2$ variables). These tables are used to compute the first prover message of such sumcheck. We then construct 3 more tables to compute the second's prover message, and finally we use one last table to compute the third prover message. Similarly, we build lookup tables for the first rounds of the second and third last sumchecks of the GKR protocol.

3.3.1 Comparison to other “prove-in-the-exponent” usages

Albert: Missing refs? Longfellow?

Binius64 For example, [Bin26] multiplies pairs of 64-bit integer witness elements in the exponent of g . This is substantially different from our setting where, when binary fields come into play, we are only interested in proving an integer claim about a linear combination of elements of the form “(public values) · (witness elements)”, i.e. $\langle \mathbf{v}, \mathbf{w} \rangle = \mu$ in $\mathbb{Z}$, with the public values $\mathbf{v}$ admitting a tensor split $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ (also BitZ can prove multiplication of all sorts of ring elements, not just 64-bit integers). This simpler setting is what allows us to use the low entropy optimizations designed in this work. Crucially, **the reason why we only need to address this setting is because we start off with constraints and a PIOP over integers/fields/rings (Steps 2 to 4 inclusive), and only ever move to work on binary fields when we need to prove an MLE evaluation of the witness (a linear claim) over a “small” prime field (Step 5).**

[Bin26] on the other hand multiplies 64-bit integer witness in the exponent directly. More precisely, [Bin26] proves the identity $g^{\mathbf{w}_{1i} \cdot \mathbf{w}_{2i}} = g^{2^{64} \cdot \mathbf{w}_{3i}} \cdot g^{\mathbf{w}_{4i}}$ for $\mathbf{w}_1, \dots, \mathbf{w}_4$ vectors of 64-bit integers (together with a small additional constraint to address possible overflow). This is done through a combination of LogUp* [?], a GKR grand product (for relatively small products), and the Frobenius automorphism trick. It does not seem possible to follow a similar optimization strategy to ours.

We emphasize again that BitZ supports proving operations of all kinds of ring elements (small and large integers, field elements, lattice ring elements, extension field elements), as opposed to ?? where multiplication is limited to 64-bit integers.

In ?? we compare the performance of BitZ and Binius64 when multiplying integers. BitZ outperforms Binius64 by Albert: fill in.

The note [Sou26]. In the note [Sou26], Soukhanov proposes a scheme for committing to a vector of bits $\mathbf{w}$ over the binary field $\mathbb{F}_{2^{127}}$, and proving MLE evaluation claims over the prime field $\mathbb{F}_{2^{127}-1}$, for a vector of elements from $\mathbb{F}_{2^{127}-1}$ obtained by packing bits from $\mathbf{w}$ together. To do so, at a high level, the MLE evaluation claims are proved “in the exponent” in the binary field $\mathbb{F}_{2^{127}}$, leveraging that the multiplicative subgroup of $\mathbb{F}_{2^{127}}$ has prime order $|\mathbb{F}_{2^{127}}^*| = 2^{127} - 1$. The proposed approach is substantially different from ours: [Sou26] essentially takes a polylogarithmic PCS over the prime field $\mathbb{F}_{2^{127}-1}$, such as Basefold [ZCF23], WHIR [ACFY25], etc., and executes it “in the exponent of

$\mathbb{F}_{2^{127}}$ ". To do so, it relies on the lookup argument LogUp* [?], grand product arguments (for small products), the Frobenius trick, and additional commitments, for every of the $\log \ell$ rounds.

In its execution, [Sou26] and Binius64 [Bin26] seem to us to be more similar than any of these two to BitZ. Indeed, they both use, mostly, the same collection of subprotocols, in a similar manner. Because of this we expect both approaches to have comparable performance profiles, though we do not validate this hypothesis due to [Sou26] not reporting performance nor providing an implementation.

Conceptually, [Sou26] shares with BitZ the aim of committing to bits in a binary field, while proving MLE claims of those bits in some other structure R . Note that [Sou26] is limited to $R = \mathbb{F}_{2^{127}-1}$, while BitZ can handle essentially all rings and fields R of practical interest, achieving 128 bits of security and using $\mathbb{F}_{2^{128}}$ as binary field. Additionally it is not clear whether [Sou26] can leverage any of our low-entropy oriented optimizations without substantially modifying the approach, incorporating some of our ideas. **Albert: Ask Lev if he has implemented his thing so we can compare. Also ask him if Binius64 is essentially following his method (Logup*, grand product, Frobenius)**

Albert: The comparison of Lev and BitZ right now has several unbacked claims (which I believe are true, but can't really prove them, especially without an implementation). So probably it's better to not push it, remove the unbacked claims, and leave it as it is

3.4 BitZ in full generality

In general, BitZ can prove polynomial constraints over multiple rings R_0 (in the same instance) of the form $R[X]/(h(X))$, where R is either the ring of integers $\mathbb{Z}$ or a finite field (not necessarily prime), and $h(X) \in R[X]$ is either the zero polynomial, or a monic non-constant polynomial. Examples of such rings R_0 include: the integers $\mathbb{Z}$, arbitrary finite fields, and lattice rings of the form $\mathbb{F}_q[X]/(\Phi(X))$ for $\Phi(X)$ a cyclotomic polynomial and q a prime power. More generally, R_0 can be any two-generated ring of zero or prime characteristic.

Concretely, we define a target relation, which we call *Universal Polynomial Satisfiability (UPESAT)*, as follows. Let ℓ be a witness length and k a public input length. Let R be either $\mathbb{Z}$ or a finite field. Let $\mathbf{R} = (R_i)_{i \in [\ell]}$ be a collection of rings of the form $R_i = R[X]/(h_i(X))$ for $h_i(X) \in R[X]$ either zero or a monic non-constant polynomial. For each $i \in [\ell]$, let $\mathbf{p}^{(i)}$ be a vector of n polynomials

$$\mathbf{p}^{(i)} = \left(\mathbf{p}_j^{(i)} \mid j \in [n] \right) \in ((R[X])[Y_1, \dots, Y_{k+\ell}])^n$$

with coefficients in $R[X]$ and on $k+\ell$ variables $Y_1, \dots, Y_{k+\ell}$. Given $a \in R[X]$ and $i \in [\ell]$, we say that $a = 0$ holds in R_i if the image of a under the canonical projection $R[X] \rightarrow R_i$ is zero. Finally, given a degree bound d and a bit-size bound, we let $R_{\leq B}^{\leq d}[X]$ be the subset of $R[X]$ formed by polynomials of degree $< d$ and coefficients in R of bit-size $\leq B$ (see Step 1 of Section 3.1 for a formalization of how we bit-decompose elements from R).

We define our target relation as

$$\mathcal{R}_{\text{UPESAT}}(R_{\leq B}^{\leq d}[X], \mathbf{p}, \mathbf{R}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \left(R_{\leq B}^{\leq d}[X] \right)^k \\ \mathbf{w} = \mathbf{w} \in \left(R_{\leq B}^{\leq d}[X] \right)^\ell \end{array} \mid \mathbf{p}_j^{(i)}(\mathbf{x}, \mathbf{w}) = 0 \text{ in } R_i \text{ for all } j \in [n], i \in [\ell] \right\}$$

Remark 3.4 (Non prime characteristics or arithmetic modulo arbitrary integers). UPESAT does not allow to work with non-prime and non-zero characteristics m . I.e. it does not support, for example, $\mathbb{Z}/2^n\mathbb{Z}$ (i.e. arithmetic modulo $m = 2^n$). Following [CHA24, GWV⁺25], we arithmetize such characteristics in $\mathbb{Z}$ or $\mathbb{Z}[X]$ via the standard trick of adding a term of the form $+m \cdot w$ to the constraints, for w new witness elements. The correctness and soundness of this step follows from the fact that $a + b = c$ in $\mathbb{Z}/m\mathbb{Z}$ if and only if $a + b = c + m \cdot w$ in $\mathbb{Z}$ for some $w \in \mathbb{Z}$. Similar identities hold for rings of non-prime and non-zero characteristic other than $\mathbb{Z}/m\mathbb{Z}$. See the introduction and technical overviews of [GWV⁺25, CXNB26] for more details.

We note that UPESAT is a subset of the more general UCS relation from [AGK⁺26]. See ?? for a comparison. [Albert: Mention we could allow for different characteristics in the same relation?](#)

Similarly as in Section 3.2, BitZ can also handle a variation of $\mathcal{R}_{\text{UPESAT}}$ where $\text{bits}(\mathbf{w})$ is a $\mathbb{F}_2$ -linear transformation of a vector of bits $\mathbf{f} \in \mathbb{F}_2^{\nu'}$.

Example 3.5. [Albert: I could briefly explain here how to arithmetize the computations in our experiments.](#)

We now explain how to modify the basic version of BitZ in Section 3.1 for the relation $\mathcal{R}_{\text{UPESAT}}$. There are two aspects that need modification:

1. In Step 2 we need to design a method for transforming constraints in R_i to constraints in a finite field $\mathbb{F}$.
2. In Step 3, $\mathbb{F}_{\text{ext}}$ could be a proper extension of a prime field $\mathbb{F}_p$. This occurs if for example $R_i = \mathbb{F}_p$ for all $i \in [\iota]$ (i.e. all our constraints are over the prime field $\mathbb{F}_p$), and p is small (say, 32-64 bits), and so the PIOP in Step 3 needs to work on a field extension of $\mathbb{F}_p$.

Generalized Step 2. Regarding Step 2, we discuss first the case that $\iota = 1$ and $h_1 = (0)$, so that all constraints are over $R[X]$, for R the integers or a finite field, with witnesses belonging to $R_{\leq B}^{<d}[X]$. In this case the verifier samples a map π of one of the following forms: If $R = \mathbb{Z}$ and $d > 0$,

$$\pi : \mathbb{Z}[X] \rightarrow \mathbb{F}_q, \quad g(X) \mapsto \pi_q(g)(\alpha)$$

for a random prime q and a random $\alpha \in \mathbb{F}_q$, where π_q denotes reduction modulo q .

If $R = \mathbb{F}$ is a finite field of characteristic q , then

$$\pi : \mathbb{F}[X] \rightarrow \mathbb{F}', \quad g(X) \mapsto g(\alpha)$$

for $\mathbb{F}'$ possibly a field extension of $\mathbb{F}$ (so that $|\mathbb{F}'| = O(2^\lambda)$) and $\alpha \in \mathbb{F}'$. [Albert: Say a few words of why this is sound \(once I've written the soundness intuition for the basic protocol](#)

The general case where not all ideals $(h_i(X))$ are zero is dealt in the same way as in Zinc+ [AGK⁺26]. We omit the details here for the sake of brevity. We refer to ?? (Items 3 and 4 of Round 1, and Round 2, of ??) or to the technical overview of [AGK⁺26] [Albert: Should I describe this here?.](#)

The case when $\mathbb{F}_{\text{ext}}$ is a proper extension Next we discuss the scenario where $\mathbb{F}_{\text{ext}}$ is a proper field extension at the end of Step 3 of [Section 3.1](#). In this case, Step 4 proceeds in the same way as in the normal case. At the end of Step 4 prover and verifier are left with the claim

$$\langle \mathbf{u}, \pi(\text{bits}(\mathbf{w})) \rangle = \mu \quad \text{in } \mathbb{F}_{\text{ext}},$$

for some $\mathbf{u} \in \mathbb{F}_{\text{ext}}^\ell$ and $\mu \in \mathbb{F}_{\text{ext}}$.

After Step 4, Step 5.0 triggers **Albert: Again, in the main body this step triggers after step 5.1 and is slightly different.** Since $\mathbb{F}_{\text{ext}}$ is a finite field, it is of the form $\mathbb{F}_{\text{ext}} = \mathbb{Z}[X]/(q, f(X))$ for an irreducible polynomial $f(X)$ over $\mathbb{F}_q[X]$. Let $\pi_{\text{canon}} : \mathbb{Z}[X] \rightarrow \mathbb{F}_{\text{ext}}$ be the canonical homomorphism sending $g(X) \in \mathbb{Z}[X]$ to its reduction modulo q and modulo $f(X)$.

The prover sends an integer polynomial $\mu' \in \mathbb{Z}[X]$ such that

$$\langle \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}), \text{bits}(\mathbf{u}) \rangle = \mu' \quad \text{in } \mathbb{Z}[X],$$

The verifier checks that $\pi_{\text{canon}}(\mu') = \mu$, and responds with a random prime q' such that $q' < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ and with a random prime field element $\alpha \in \mathbb{F}_{q'}$. Then prover and verifier proceed to Step 5.1 after updating $\mathbb{F}_{\text{ext}} \leftarrow \mathbb{F}_{q'}$, $\pi \leftarrow \pi_{q', \alpha}$, $\mu \leftarrow \pi_{q', \alpha}(\mu') \in \mathbb{F}_{q'}$, and

$$\mathbf{u} \leftarrow \pi_{q', \alpha} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(1)}) \otimes \pi_{q', \alpha} \circ \pi_{\text{canon}}^{-1}(\mathbf{u}^{(2)}),$$

where $\pi_{q', \alpha} : \mathbb{Z}[X] \rightarrow \mathbb{F}_{q'}$ is defined as

$$\pi_{q', \alpha}(g(X)) = \pi_{q'}(g)(\alpha) \in \mathbb{F}_{q'}.$$

Albert: Again, say a few words of why this is sound (once I've written the soundness intuition for the basic protocol)

One can, instead of sampling morphisms of the form $\pi_{q', \alpha}$, sample other types of $\mathbb{Z}$ -module homomorphisms $\pi' : \mathbb{Z}^{\deg(\mathbb{F}_{\text{ext}})}[X] \rightarrow$ a prime field. See ???. This can modestly improve the soundness errors at the expense of increasing the communication complexity. **Albert: It may be a good idea to present the protocols as parametrized by a family of module homomorphisms where we sample from. Right now the protocols have the fixed family above. Let's see if there's time**

The rest of BitZ proceeds exactly as in [Section 3.1](#).

Albert: To discuss somewhere: this technique allows us to do things such as: 1) start with a constraint in Babybear, do the PIOP in an extension, and then open with BitZ, 2) run PIOP's with primes of size, say ≥ 128 bits, and then change the prime to a 108 bit prime to fit in the exponent of $\mathbb{F}_{2^\nu}^*$ without overflowing

Albert: Thoughts: We can handle arbitrary linear weights in our claims, of arbitrary bit-size and in arbitrary extension degrees, and still use a smallish binary field like GHASH during our protocol (not obvious at all because "proving in the exponent in principle only allows for using $\lesssim 90$ -ish integer bits. Albert: This is a good story for the technical overview

Scratch from another section. Delete? **Albert: These two examples would be illustrative in the intro and TO** To fix intuition, $\deg(\mathbb{K})$ and q are determined by the field or ring a PIOP runs on. For example, in our full proving system in ??, constraints can be (among others) over $\mathbb{Z}$ or $\mathbb{Z}[X]$, then they are projected to a random prime field $\mathbb{F}_q$, a PIOP is ran on them, and an MLE claim over

$\mathbb{F}_p$ is produced. Then [Construction 5.14](#) is used to prove those claims. The size of q is set so that the PIOP achieves λ bits of security. A very conservative choice in this setting is $q \leq 2^{256}$, but also $q \leq 2^{128}$ is feasible if $\lambda = 100$ or one is willing to use grinding during the PIOP.

Another setting is to have constraints over a finite field $\mathbb{K}$ such as a degree 4 or 5 extension of the Babybear field, or a degree 2 or 3 extension of the Goldilocks field

4 Preliminaries

General notation Given an integer $n \geq 1$, we write $[n]$ to denote the set $\{1, \dots, n\}$. Given integers $m \leq n$, we let $[m..n]$ denote the set of integers $\{m, m+1, \dots, n\}$.

We use $\circ$ to denote composition of maps, so that for maps $g : S_1 \rightarrow S_2$ and $f : S_2 \rightarrow S_3$, we have $f \circ g : S_1 \rightarrow S_3$ defined as $f \circ g(x) = f(g(x))$.

Any map $f : S_1 \rightarrow S_2$ applies entry-wise to vectors from S_1^n , for any n , and similarly for matrices. A map f also applies coefficient-wise to polynomials with coefficients in S_1 . A multiplication of a vector or a matrix by a scalar means that we multiply each entry of the vector or matrix by that scalar.

We denote vectors with boldface letters, e.g. $\mathbf{u}, \mathbf{v}$. By $\mathbf{u}_i$ we denote the i -th position of $\mathbf{u}$. Sometimes we index the entries of a vector $\mathbf{u}$ with elements of a boolean hypercube $\{0, 1\}^t$.

We look at vectors both as row vectors and column vectors. When performing matrix-vector multiplications, it will always be clear which one we intend. Given a ring R , a matrix $M \in R^{n \times m}$ and vectors $\mathbf{u} \in R^m$, $\mathbf{v} \in R^n$, one has

$$\langle M \cdot \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{u}, M^\top \cdot \mathbf{v} \rangle, \quad (18)$$

where $M^\top$ denotes transposition and $\langle \cdot, \cdot \rangle$ denotes inner product.

4.1 Algebra

Concrete fields and rings We write $\mathbb{F}_q$ for the finite field with q elements (where q is a prime or a prime power), $\mathbb{Q}$ for the field of rational numbers, and $\mathbb{Z}$ for the ring of integers. For an integer $n \geq 1$, $\mathbb{Z}_n$ denotes the ring of integers modulo n , with the convention $\mathbb{Z}_0 := \mathbb{Z}$. When n is prime, $\mathbb{Z}_n = \mathbb{F}_n$. We let

$$\pi_n : \mathbb{Z} \rightarrow \mathbb{Z}_n$$

denote reduction modulo n . For $n \geq 1$, we fix the inverse $\pi_n^{-1} : \mathbb{Z}_n \rightarrow [0..n-1] \subseteq \mathbb{Z}$ mapping each element a in $\mathbb{Z}_n$ to the integer in the range $[0..n-1]$ representing a .

Rings in general All rings in this work are associative, commutative, have a multiplicative identity. Given a ring R , we write $R[X_1, \dots, X_n]$ for the ring of polynomials with coefficients in R and variables $X_1, \dots, X_n$ (not necessarily multilinear).

The *characteristic* of a ring R , denoted $\text{char}(R)$, is the smallest $c \geq 1$, if it exists, such that $1 + \dots + 1 = 0$ in R . Otherwise, if $1 + \dots + 1 \neq 0$ for all c , the characteristic is zero. By $[0..n]_R$ we denote all elements in R of the form $1 + \dots + 1$ for $i = 0, \dots, n$. For every ring R we have a canonical embedding

$$\mathbb{Z}_{\text{char}(R)} \subseteq R$$

of the ring of integers modulo $\text{char}(R)$ into R . Namely, the set $\{n \cdot 1 \mid n \in \mathbb{Z}\} \subseteq R$ for 1 the identity element of R , forms the ring $\mathbb{Z}_{\text{char}(R)}$. Further, $\pi_{\text{char}(R)}$ induces a natural homomorphism (not necessarily surjective)

$$\pi_{\text{char}(R)} : \mathbb{Z} \rightarrow R \quad (19)$$

sending $n \in \mathbb{Z}$ to $n \cdot 1$ for 1 the identity of R .

Given a set or vector of integers $\mathbf{w} \in \mathbb{Z}^m$, we define

$$\mathbf{w}_R = \pi_{\text{char}(R)}(\mathbf{w}). \quad (20)$$

Finitely generated rings Further, all rings in this work are *finitely generated* as rings. Equivalently, every ring R in this work is isomorphic to a quotient

$$R = \mathbb{Z}[X_1, \dots, X_k]/I$$

of a polynomial ring over $\mathbb{Z}$ by an ideal I . This condition is satisfied by, for example:

1. All finite fields and the rings $\mathbb{Z}_n$;
2. $\mathbb{Z}$, and the polynomial rings $\mathbb{Z}[X]$ and $\mathbb{F}_q[X]$
3. Arbitrary quotients of the previous rings, such as $\mathbb{Z}_n[X]/(h(X))$ for an arbitrary polynomial $h(X)$ and an arbitrary integer $n \geq 0$, which includes lattice rings.

It is not satisfied by, for example, $\mathbb{Q}$ and $\mathbb{Q}[X]$, which are not finitely generated as rings. Some constructions in this work impose further requirements on the rings they use. We state these when needed.

Given a finitely generated ring R with fixed generators $1, \eta_1, \dots, \eta_k$ (we always list 1 among the generators, in particular $k = 0$ for $\mathbb{Z}_n$) and bounds $B, D \geq 1$, we let $R_{\leq B}^{< D}$ denote the set of elements of R of the form

$$h = \sum_{\mathbf{a} \in \mathbb{Z}_{\geq 0}^k, a_1 + \dots + a_k < D} h_{\mathbf{a}} \cdot \eta_1^{a_1} \dots \eta_k^{a_k}, \quad h_{\mathbf{a}} \in [0 \dots 2^B - 1]_R,$$

i.e. $R_{\leq B}^{< D}$ is formed by the elements of R that can be written as a polynomial of total degree less than D in $\eta_1, \dots, \eta_k$, with coefficients obtained by adding 1 together (in R) less than 2^B times.

4.2 Linear codes and relations

Linear codes Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code over $\mathbb{F}$ of dimension ℓ and blocklength m . For $h \geq 1$, the h -interleaving of $\mathcal{C}$ is the code $\mathcal{C}^{\equiv h} \subseteq (\mathbb{F}^h)^m$ of dimension $\ell \cdot h$ and blocklength $m \cdot h$, whose codewords have the form

$$((c_{i1}, \dots, c_{ih}))_{i \in [m]} \quad \text{for } (c_{1j}, \dots, c_{mj}) \in \mathcal{C} \quad \text{for all } j \in [h].$$

Given two words w_1, w_2 of length n , the relative Hamming distance between w_1 and w_2 (the fraction of positions in which they differ) is denoted

$$\Delta(w_1, w_2).$$

The *relative minimum distance* of $\mathcal{C}$, denoted $\Delta(\mathcal{C})$, is the minimum of $\Delta(c_1, c_2)$ over distinct codewords $c_1, c_2 \in \mathcal{C}$. Given $\delta \in (0, 1)$ and an integer $L \geq 1$, we say $\mathcal{C}$ is (δ, L) -list-decodable if for every word $w \in \mathbb{F}^m$ there are at most L codewords $c \in \mathcal{C}$ with $\Delta(w, c) < \delta$. Every code is $(\delta, 1)$ -list-decodable for $\delta \leq \Delta(\mathcal{C})/2$. The value $\Delta(\mathcal{C})/2$ is called the *unique decoding radius* of $\mathcal{C}$.

Relations A *relation* $\mathcal{R}$ consists of triples $(\mathbf{x}, \mathbf{y}; \mathbf{w})$, where $\mathbf{x}$ is the explicit instance, $\mathbf{y}$ the implicit instance or oracle, and $\mathbf{w}$ the witness. The corresponding language is $\text{LANG}(\mathcal{R}) := \{(\mathbf{x}, \mathbf{y}) \mid \exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \mathcal{R}\}$.

Polynomial Equation Satisfiability (PESAT) over finite fields We recall a relation from [BCFW25] [Albert: Linear time accumulation paper, or earlier?](#) over finite fields, which captures popular constraints such as R1CS, CCS, AIR (cf. [BCFW25]), and lookup and permutation constraints (cf. [GWV⁺25]).

Fix a finite field $\mathbb{F}$, a witness length ℓ , an input length k , a number of rows n , a collection of n polynomials $\mathbf{p} = (\mathbf{p}_1, \dots, \mathbf{p}_n)$ on $k + \ell$ variables.

Definition 4.1. The PESAT relation with respect to these parameters is defined as

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}) = \left\{ \begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in \mathbb{F}^\ell \end{array} \middle| \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \right\}$$

Let $\mathcal{C} \subseteq \mathbb{F}^m$ be a linear code of dimension ℓ' and blocklength m over $\mathbb{F}$. Let $W \subseteq \mathbb{F}^\ell$ be a subset and let $T : W \rightarrow \mathbb{F}^{\ell'}$ be an arbitrary map (looking ahead, for us T will be a packing map that packs elements from $\mathbb{F}_2$ into elements of a larger binary field, so that $W = \mathbb{F}_2^\ell \subseteq \mathbb{F}^\ell$). We define the *Encoded PESAT relation* as:

$$\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T) := \left\{ \begin{array}{l} \left(\begin{array}{l} \mathbf{x} = \mathbf{x} \in \mathbb{F}^k, \\ \mathbf{y} = f \in \mathbb{F}^{\ell'}, \\ \mathbf{w} = \mathbf{f} \in W \end{array} \right) \middle| \begin{array}{l} \text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f \\ \wedge \mathbf{p}_i(\mathbf{x}, \mathbf{f}) = 0 \text{ for all } i \in [n] \end{array} \right\}$$

We will drop the reference to T whenever it is understood from the context. When $n = 1$ and $\mathbf{x}$ are such that $\mathbf{x} = (\mathbf{v}, \mu) \in \mathbb{F}^\ell \times \mathbb{F}$; $\mathbf{p}_1(\mathbf{x}, \mathbf{f}) = \langle \mathbf{f}, \mathbf{v} \rangle - \mu$ (so that $k = \ell + 1$), the relation $\mathcal{R}_{\text{PESAT}}(\mathbb{F}, \mathbf{p}, \mathcal{C}, T)$ is called a *constrained code* [ACFY25] [Albert: WHIR?](#). We denote it

$$\mathcal{R}_{\text{Lin}}(\mathbb{F}, \mathcal{C}, T).$$

Definition 4.2 (Relaxation of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$). Given $\delta \in (0, 1)$, the *relaxation* of $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, denoted

$$\mathcal{R}_{\text{PESAT}}^\delta, \quad \mathcal{R}_{\text{Lin}}^\delta$$

are the relations defined exactly as $\mathcal{R}_{\text{PESAT}}$ and $\mathcal{R}_{\text{Lin}}$, respectively, except that the constraint $\text{Enc}_{\mathcal{C}}(T(\mathbf{f})) = f$ is replaced by

$$\Delta(\text{Enc}_{\mathcal{C}}(T(\mathbf{f})), f) < \delta.$$

Multilinear extensions. Given a field $\mathbb{F}$ and $\mathbf{u} \in \mathbb{F}^{2^r}$ indexed by $\{0, 1\}^r$, we denote by $\tilde{\mathbf{u}} \in \mathbb{F}[X_1, \dots, X_r]$ the unique multilinear polynomial with $\tilde{\mathbf{u}}(\mathbf{b}) = \mathbf{u}_{\mathbf{b}}$ for all $\mathbf{b} \in \{0, 1\}^r$. For $\alpha, \beta \in \mathbb{F}^r$ we let $\text{eq}(\alpha; \beta) := \prod_{i \in [r]} (\alpha_i \cdot \beta_i + (1 - \alpha_i) \cdot (1 - \beta_i))$, so that $\tilde{\mathbf{u}}(\alpha) = \sum_{\mathbf{b} \in \{0, 1\}^r} \mathbf{u}_{\mathbf{b}} \cdot \text{eq}(\mathbf{b}; \alpha)$. We write $\text{eq}(\mathbf{b}, \mathbf{c}; \alpha)$ for $\text{eq}((\mathbf{b}, \mathbf{c}); \alpha)$.

4.3 Interactive Oracle Reductions (IOR) of proximity

A public-coin *Interactive Oracle Reduction (IOR) of proximity* $\Pi = (\mathsf{P}, \mathsf{V})$ from a relation $\mathcal{R}$ to a relation $\mathcal{R}'$ is specified as follows. The prover P receives $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$. The verifier V receives $\mathbf{x}$ and query access to $\mathbf{y}$. They interact for k rounds. In round $j \in [k]$, the prover sends an oracle π_j and the verifier sends a uniformly random message $\rho_j \in \{0, 1\}^{r_j}$. After the k rounds, the verifier makes queries to $\mathbf{y}$ and to $\pi_1, \dots, \pi_k$. Then:

- V either rejects or outputs a new pair $(\mathbf{x}', \mathbf{y}')$, where $\mathbf{y}'$ is specified implicitly in terms of the oracles $\mathbf{y}$ and $\pi_1, \dots, \pi_k$.
- P outputs a witness $\mathbf{w}'$ such that $(\mathbf{x}', \mathbf{y}', \mathbf{w}') \in \mathcal{R}'$.

We write $\langle \mathsf{P}, \mathsf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for the interaction of prover and verifier during the IOR.

Definition 4.3 (Completeness). Π has (*perfect*) *completeness* if for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr_{\rho_1, \dots, \rho_k} [(\mathbf{x}', \mathbf{y}'; \mathbf{w}') \in \mathcal{R}' \mid (\mathbf{x}', \mathbf{y}'; \mathbf{w}') \leftarrow \langle \mathsf{P}, \mathsf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})] = 1.$$

Remark 4.4. An IOP for a relation $\mathcal{R}$ is an IOR from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\text{triv}} = \{(\mathbf{x} = \perp, \mathbf{y} = \perp, \mathbf{w} = \perp)\}$.

We proceed to define round-by-round knowledge soundness of an IOR, following [BCFW25] in the formulation with relaxed relations of [CFW26].

Definition 4.5 (Knowledge state function with relaxation). Let $\Pi = (\mathsf{P}, \mathsf{V})$ be an IOR from a relation $\mathcal{R}$ to $\mathcal{R}'$. A *knowledge state function with relaxation* $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}}')$ for Π is a (possibly inefficient) function KState that, on input $(\mathbf{x}, \mathbf{y})$, interaction transcript tr , and knowledge state witness $\mathbf{w}$, outputs a bit and has the following properties.

- **Empty transcript.** If $\text{tr} = \emptyset$ is the empty transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}; \mathbf{w}) \in \widetilde{\mathcal{R}}$.
- **Prover moves.** If tr is a transcript where the prover is about to move and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- **Full transcript.** If $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if V on tr outputs $(\mathbf{x}', \mathbf{y}')$ such that $(\mathbf{x}', \mathbf{y}'; \mathbf{w}) \in \widetilde{\mathcal{R}}'$.

Definition 4.6 (Round-by-round knowledge soundness). A k -round IOR Π has *round-by-round knowledge soundness with relaxation* with errors $(\kappa_1, \dots, \kappa_k)$ if there exists a knowledge state function KState as in Definition 4.5 and a deterministic extractor $\mathcal{E}_{\text{rbr}}$ such that for every $(\mathbf{x}, \mathbf{y})$, every $j \in [k]$, and every transcript tr where the verifier is about to send ρ_j ,

$$\Pr_{\rho_j} \left[\exists \mathbf{w} \text{ such that } \begin{array}{l} \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_j, \mathbf{w}) = 1 \end{array} \right] \leq \kappa_j(\mathbf{x}, \mathbf{y}).$$

If $(\widetilde{\mathcal{R}}, \widetilde{\mathcal{R}}') = (\mathcal{R}, \mathcal{R}')$ then we say the round-by-round knowledge soundness of Π is *without relaxation*.

Lemma 4.7 (Composition of IORs [CFW26, Theorem 4.5]). *Let Π_1 be a perfectly complete k_1 -round IOR from $\mathcal{R}_1$ to $\mathcal{R}_2$ with round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_2)$ and errors $\kappa_{11}, \dots, \kappa_{1k_1}$, and let Π_2 be a perfectly complete k_2 -round IOR from $\mathcal{R}_2$ to $\mathcal{R}_3$ with round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_2, \widetilde{\mathcal{R}}_3)$ and errors $\kappa_{21}, \dots, \kappa_{2k_2}$. Then the IOR from $\mathcal{R}_1$ to $\mathcal{R}_3$ that runs Π_1 and then Π_2 on the output of Π_1 is perfectly complete, has $k_1 + k_2$ rounds, and has round-by-round knowledge soundness with relaxation $(\widetilde{\mathcal{R}}_1, \widetilde{\mathcal{R}}_3)$ and errors $\kappa_{11}, \dots, \kappa_{1k_1}, \kappa_{21}, \dots, \kappa_{2k_2}$.*

Grinding [Albert: Review](#) Grinding [?] is a technique where, given an IOP Π for a relation $\mathcal{R}$ with relaxation $\tilde{\mathcal{R}}$ with round-by-round knowledge errors $\kappa_1, \dots, \kappa_k$, produces a new IOP Π' for $\mathcal{R}$ with round-by-round knowledge with relaxation $\tilde{\mathcal{R}}$ with errors

$$2^{-\alpha_1} \cdot \kappa_1, \dots, 2^{-\alpha_k} \cdot \kappa_k.$$

The new IOP Π' works exactly as Π , except that, at each round $i \in [k]$, the verifier samples a random string of α_i bits, called *grinding string*, which it sends alongside its message, and rejects unless all bits are 0.

Clearly Π' is not necessarily complete, even if Π is. The goal is to use the BCS compilation of IOPs, and not the IOPs themselves (i.e. the compilation where oracles are replaced by Merkle roots and where interactivensness is removed through the Fiat-Shamir transform). To this end, one chooses $\alpha_1, \dots, \alpha_k$ small enough that the prover of the BCS transform of Π' can *grind* transcripts until all of the grinding strings in the transcript are the zero string. This results in the prover having to compute per round $i \in [k]$, an average of 2^{α_i} hashes of its i -th message followed by a small string of bits (in practice, one has the prover send a small string of bits next to its message, so that during grinding, it does not need to produce fresh Merkle roots at each grinding iteration). In [Table 2](#), we provide example grinding times for different values of α_i .

The round-by-round knowledge errors of Π' are obtained by defining a state function which is exactly as the state function of Π , except that the function outputs zero if some of the aforementioned strings of bits are not the zero string. The extractor for Π' is defined as the extractor for Π , ignoring the additional strings of bits in the transcript.

Table 2: Cost of finding a 64-bit nonce such that $\text{BLAKE3}(\text{msg}, \text{nonce})$ has at least α leading zero bits, with msg of 256 bits. Single and multi-threaded on a MacBook Air M4 16GB. [Albert: As implemented in Flock?](#)

α	18	19	20	21	22	23	24	25
Mean, 1 thread (ms)	12.3	19.5	42.8	82.2	203.8	464.4	808.5	1291.4
Median, 1 thread (ms)	9.2	16.3	27.8	55.2	140.6	358.1	608.5	852.0
Mean, 10 threads (ms)	2.3	3.5	8.9	14.3	35.5	84.6	148.6	240.8
Median, 10 threads (ms)	1.8	2.5	6.8	9.6	26.9	68.0	111.3	161.8

5 BitZ IOP for LinBitsRings

In this section we describe the BitZ IOP and the main relations it proves. We begin with a summary of the notation we use throughout the paper. Most of the notation will be introduced subsequently, but nevertheless we record it here for convenience of the reader.

ℓ	Bit-length of witness cf. Section 5.1
$\mathbf{f}$	Witness, either over $\{0, 1\} \subseteq \mathbb{Z}$, or $S_{\leq B}^{< D_S}$ (see below)
$\mathbf{v}, \mathbf{u}$	public vectors in inner product constraints, over R
R	Ring where constraints live, cf. Sections 5.1 and 5.5
ℓ_1, ℓ_2	Tensor decomposition dimensions, with $\ell_1 \cdot \ell_2 = \ell$
π_n	Reduction modulo n
$k + 1$	Number of generators of R
π_{canon}^R	Canonical morphism $\pi_{\text{canon}}^R : \mathbb{Z}[X_1, \dots, X_k] \rightarrow R$ (see Section 5.3)
$\mathbb{F}_{2^\nu}$	A binary field, typically with $\nu = 128$
$\mathcal{C}$	Linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν , and length m
$\mathcal{P}$	Set of primes
$\mathcal{M}_q$	Set of homomorphisms $\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ for $q \in \mathcal{P}$
ℓ_0	Witness length in the general LinBitsRings relation, cf. Section 5.5
S	Ring where witness lives, cf. Section 5.5
$S_{\leq B}^{< D_S}$	Elements of S expressible as polynomials of degree $< D_S$ in the generators of S with coefficients of bit-size $\leq B$, cf. Section 4.1 (In the core LinBitsRing relation, $S = \mathbb{Z}$, $B = 1$)
ψ	Ring homomorphism $\psi : S \rightarrow R$
d	Number of monomials $\gamma_1 = 1, \dots, \gamma_d$ of degree $< D_S$ in the generators of S (Section 5.5)
L	Bitification matrix, $L \in S^{\ell_0 \times \ell}$, cf. Definition 5.10

Table 3: Summary of notation used throughout the paper.

5.1 The core LinBitsRings relation

In this section we define the main linear relation we work with. We start by fixing notation:

- Let $\mathbb{F}_{2^\nu}$ be a field with 2^ν elements. Let $\beta_1, \dots, \beta_\nu \in \mathbb{F}_{2^\nu}$ be an $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^\nu}$ seen as an $\mathbb{F}_2$ -vector space of dimension ν . Given a vector $\mathbf{u} \in \mathbb{F}_2^\ell$, with ν dividing ℓ , we define

$$\text{Pack}_\nu(\mathbf{u}) \in \mathbb{F}_{2^\nu}^{\ell/\nu}$$

as the vector whose j -th entry is

$$\text{Pack}_\nu(\mathbf{u})_j = \mathbf{u}_{(j-1) \cdot \nu + 1} \cdot \beta_1 + \dots + \mathbf{u}_{(j-1) \cdot \nu + \nu - 1} \cdot \beta_{\nu-1} + \mathbf{u}_{j \cdot \nu} \cdot \beta_\nu \in \mathbb{F}_{2^\nu}.$$

- Let $\ell \geq 1$ be a dimension parameter divisible by ν , $\mathcal{C}$ a code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and blocklength m .
- Let R be a finitely generated ring (typically R is $\mathbb{Z}$, a finite field, not necessarily a prime field, or a quotient of a polynomial ring $\mathbb{Z}[X]$ or $\mathbb{F}_q[X]$) and let $\pi : \mathbb{Z} \rightarrow R$ be the homomorphism mapping an integer a to $a \cdot 1 \in R$ (i.e. reduction modulo the characteristic of R).

We begin by defining a particular subclass of our target relation.

Definition 5.1. The *Core LinBitsRings* relation is defined as

$$\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}) := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f}) \\ \langle \pi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R \end{array} \right\}$$

For $\delta \in (0, 1)$, we let $\mathcal{R}_{\text{LinBitsRings}}^\delta$ be the relation $\mathcal{R}_{\text{LinBitsRings}}$ after replacing the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})$ by

$$\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta.$$

5.2 Tensor decomposition and products of constrained codes

As discussed in [Section 3.1 Albert: TO](#), BitZ crucially demands and uses a tensor decomposition of $\mathbf{v}$. We next establish notation and basic definitions around it.

Let $(\mathbf{x}, \mathbf{y}, \mathbf{w}) = ((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. We fix a tensor decomposition of the vector $\mathbf{v} \in R^\ell$ as

$$\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} = \left(\mathbf{v}_i^{(1)} \cdot \mathbf{v}_j^{(2)} \right)_{i \in [\ell_1], j \in [\ell_2]}$$

for some $\mathbf{v}^{(1)} \in R^{\ell_1}, \mathbf{v}^{(2)} \in R^{\ell_2}$, with $\ell_1 \cdot \ell_2 = \ell$. Note that the trivial tensor decomposition is admitted, i.e. we may effectively not decompose $\mathbf{v}$ by choosing $\ell_1 = \ell, \ell_2 = 1, \mathbf{v}^{(1)} = \mathbf{v}$, and $\mathbf{v}^{(2)} = (1)$.

We index the entries of $\mathbf{f}$ as $\mathbf{f} = (\mathbf{f}_{ij})_{i \in [\ell_1], j \in [\ell_2]}$. Note that

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle = \sum_{j \in [\ell_2]} \left(\sum_{i \in [\ell_1]} \pi(\mathbf{f}_{ij}) \cdot \mathbf{v}_i^{(1)} \right) \cdot \mathbf{v}_j^{(2)} = \sum_{j \in [\ell_2]} \langle \pi(\mathbf{f}_{*j}), \mathbf{v}^{(1)} \rangle \cdot \mathbf{v}_j^{(2)},$$

where $\mathbf{f}_{*j} = (\mathbf{f}_{ij})_{i \in [\ell_1]}$. Consider the relation (cf. [Definition 4.1](#)) $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ for ℓ_2 constraint polynomials Q_{BitZ} so that

$$\begin{aligned} & \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu) \\ &= \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{F}_{2^\nu}^{\ell_1} \times \mathbb{F}_{2^\nu}^{\ell_2}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \mathbb{F}_2^{\ell_1 \times \ell_2} \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{f}) \\ \bigwedge \prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_{ij} + 1) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \end{array} \right\}. \end{aligned}$$

Here Q_{BitZ} consists of the ℓ_2 polynomials $\prod_{i=1}^{\ell_1} ((\mathbf{y}_i - 1) \cdot \mathbf{f}_{ij} + 1) - \boldsymbol{\mu}_j, j \in [\ell_2]$.

We assume prover and verifier have access to an IOR Π_{red} that reduces $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$, where, recall, this latter relation has predicates of the form $\langle \mathbf{g}, \mathbf{z} \rangle = c$ with $\mathbf{g} \in \mathbb{F}_{2^\nu}^\ell, \mathbf{z}$ a vector over $\mathbb{F}_{2^\nu}, c \in \mathbb{F}_{2^\nu}$, and $g = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{g})$. In our instantiation Π_{red} is an optimized GKR protocol for grand products (cf. [Section 6](#)), which reduces the initial grand product claims to MLE evaluation claims of $\mathbf{y}$ and $\mathbf{f}$ over $\mathbb{F}_{2^\nu}$.

We assume Π_{red} works so that the verifier never queries $\mathbf{y}$; and that, given an input

$$(\mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu),$$

Π_{red} outputs

$$((\mathbf{z}, c), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$$

for some vector $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and field element $c \in \mathbb{F}_{2^\nu}$. In particular, Π_{red} outputs the same oracle f and witness $\pi_2(\mathbf{f})$ as the oracle and witness it is given as inputs.

This is indeed the case with GKR: GKR reduces the initial circuit evaluation claim with a witness input, into an MLE evaluation claim of the same witness, without querying any oracle in-between. Consequently an IOR between the versions of these relations with oracle $\perp$ (such as [Construction 6.1](#), cf. [Lemma 6.2](#)) can be turned, in a straightforward manner, into an IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta)$ and the same errors, for every δ . The input to the IOR now includes an oracle f , and this same oracle is output at the end of the IOR. The knowledge state function adds the condition $\Delta(f, \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{w})) < \delta$.

5.3 The BitZ IOP for the core LinBitsRings relation

Algebraic set-up Recall ([Section 4.1](#)) that all rings in this work are finitely generated as rings. We fix generators $1, \eta_1, \dots, \eta_k \in R$ [Albert: I added a 1 here \(one can assume without loss that 1 is part of the generating set\)](#). Let AI check that nothing breaks and we let

$$\pi_{\text{canon}}^R : \mathbb{Z}[X_1, \dots, X_k] \rightarrow R$$

be the surjective ring homomorphism mapping each integer a to $a \cdot 1 \in R$, and X_i to η_i for $i \in [k]$. The choice of $\eta_1, \dots, \eta_k$ is far from unique, see [Example 5.2](#) below.

We also fix an inverse map $\pi_{\text{canon}}^{R^{-1}} : R \rightarrow \mathbb{Z}[X_1, \dots, X_k]$ for π_{canon}^R , so that $\pi_{\text{canon}}^R \circ \pi_{\text{canon}}^{R^{-1}} = \text{id}$. We refer to $\pi_{\text{canon}}^{R^{-1}}(a)$ as the *lift* of $a \in R$. Note that, given π_{canon}^R , there may be many possible choices for $\pi_{\text{canon}}^{R^{-1}}$. In [Example 5.2](#) we provide example inverse maps.

Recall (see [Section 4.1](#)) that π_n denotes reduction modulo n and that π_n^{-1} maps each element from $\mathbb{Z}_n$ to its integer representative in $[0..n-1]$.

Example 5.2. We provide some examples of rings R and choices for projections π_{canon}^R and their inverses:

- **Prime fields.** If $R = \mathbb{F}_q$ is a prime field with q elements, then we can take $k = 0$, so that $\pi_{\text{canon}}^{\mathbb{F}_q} = \pi_q : \mathbb{Z} \rightarrow \mathbb{F}_q$ is simply reduction modulo q , with its inverse being π_q^{-1} (see [Section 4.1](#)).
- **Integers.** Similarly, for $R = \mathbb{Z}$ we can take $k = 0$, and then $\pi_{\text{canon}}^{\mathbb{Z}}$ and its lifts are the identity map on $\mathbb{Z}$.
- **Field extensions.** If R is a field extension of a prime field $\mathbb{F}_q$ of degree $d > 1$, then we can take $k = 1$ and $\eta_1 = x$, where x is a root of an irreducible polynomial $h(X_1) \in \mathbb{F}_q[X_1]$ of degree d (so that $R = \mathbb{F}_q(x)$). Then $\pi_{\text{canon}}^R : \mathbb{Z}[X_1] \rightarrow R$ consists in reducing coefficients modulo q , and then sending X_1 to x . Writing $a \in R$ as $a = \sum_{i=0}^{d-1} a_i \cdot x^i$ with $a_i \in \mathbb{F}_q$, one can then define a natural lift of a as $\sum_{i=0}^{d-1} \pi_q^{-1}(a_i) \cdot X_1^i$.
- **Univariate integer polynomials.** If $R = \mathbb{Z}[X]$, then we can take $k = 1$ and $\eta_1 = X$, and then $\pi_{\text{canon}}^{\mathbb{Z}[X]} : \mathbb{Z}[X_1] \rightarrow \mathbb{Z}[X]$ is an isomorphism, with lifts given by its inverse.
- **Lattices, Galois rings, etc.** More generally, if R is finitely generated as a $\mathbb{Z}_{\text{char}(R)}$ -module and it is free as a $\mathbb{Z}_{\text{char}(R)}$ -module, with $\mathbb{Z}_{\text{char}(R)}$ -module-basis $\beta_1 = 1, \beta_2, \dots, \beta_m$ (a condition satisfied by, e.g., all finite fields, $\mathbb{Z}$, and $\mathbb{Z}_n[X]/(h(X))$ for any monic $h(X)$ and any integer $n \geq 0$), then we may take $k = m - 1$ and $\eta_i = \beta_{i+1}$ for $i \in [k]$. Then given $a \in R$ and writing $a = \sum_{i \in [m]} a_i \cdot \beta_i$ with $a_i \in \mathbb{Z}_{\text{char}(R)}$, we can define the lift of a as $\pi_{\text{char}}^{-1}(a_1) + \sum_{i=2}^m \pi_{\text{char}}^{-1}(a_i) \cdot X_{i-1}$.

We assume without further comment that it is efficient to compute values $\pi_{\text{canon}}^R(h)$ for $h \in \mathbb{Z}[X_1, \dots, X_k]$, and to check whether $\pi_{\text{canon}}^R(h) = 0$. This is the case in all examples above.

The BitZ IOP. We provide an IOP for $\mathcal{R}_{\text{LinBitsRings}}$ next. We assume the notation from the previous two [Sections 5.1](#) and [5.2](#), and from the preliminary [Section 4.1](#), together with:

- Let Π_{iopp} be an IOP for $\mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$. In our instantiation, Π_{iopp} is the WHIR/Ligerito protocol [\[ACFY25, NA25\]](#) over binary fields as implemented in Flock [\[BRW26\]](#), with ring-switching [\[DP23\]](#) and a preliminary sumcheck over $\mathbb{F}_{2^\nu}$ that transforms arbitrary inner products to MLE evaluation claims (since the ring switch of [\[BRW26\]](#) does not support arbitrary inner product claims). We set the proximity parameter to the Johnson bound when targeting 100 bits of security, and to the unique decoding radius for 128 bits of security, see [Section 5.7](#) for more details.
- Let $\ell = \ell_1 \cdot \ell_2$ be dimension parameters, and R a ring with generators $1, \eta_1, \dots, \eta_k$, as in the previous sections. We let $\mathcal{P}$ be a set of primes with $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. Given the tensor decomposition $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ from [Section 5.2](#), we define [Albert: Why a subscript \$R\$ in \$N_R\$ and not in the others?](#)

$$\begin{aligned} \mathbf{u} &:= (\mathbf{v}^{(1)}, \mathbf{v}^{(2)}) \in R^{\ell_1 + \ell_2} \\ D &:= \max_{i \in [|\mathbf{u}|]} \deg(\pi_{\text{canon}}^R{}^{-1}(\mathbf{u}_i)), \quad N_R := \max_{i \in [|\mathbf{u}|]} \|\pi_{\text{canon}}^R{}^{-1}(\mathbf{u}_i)\|_\infty, \end{aligned} \quad (21)$$

where $\deg$ denotes total degree, and $\|\cdot\|_\infty$ denotes the maximum absolute value of the coefficients. We also define a quantity T as follows. T is the maximum, over monomials m in variables $X_1, \dots, X_k$, of the number of pairs (m_1, m_2) of monomials of total degree at most D each such that $m_1 \cdot m_2 = m$.

For example, $T \leq D + 1$ if $k = 1$, and $T \leq 2$ if $D \leq 1$. In general we have $T \leq (D + 1)^k$.

- For every prime $q \in \mathcal{P}$, let $\mathcal{M}_q$ be a set of ring homomorphisms

$$\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q,$$

and let $\mathcal{M} := (\mathcal{M}_q)_{q \in \mathcal{P}}$. The verifier will sample primes and morphisms from $\mathcal{P}$ and $\mathcal{M}_q$. We let $\theta(\mathcal{P}, \mathcal{M}, \ell)$ be the maximum probability that $\pi_{\text{rand}}(h) = 0$, over random $q \leftarrow \mathcal{P}$, $\pi_{\text{rand}} \leftarrow \mathcal{M}_q$, and where the maximum is taken over nonzero polynomials $h \in \mathbb{Z}[X_1, \dots, X_k]$ of total degree at most $2 \cdot D$, with $\|h\|_\infty \leq 2 \cdot T \cdot \ell \cdot N_R^2$, and such that $\pi_{\text{canon}}^R(h) \neq 0$. Precisely:

$$\theta(\mathcal{P}, \mathcal{M}, \ell) := \max_h \left\{ \Pr_{q \leftarrow \mathcal{P}, \pi_{\text{rand}} \leftarrow \mathcal{M}_q} [\pi_{\text{rand}}(h) = 0] \right\},$$

where h ranges over the polynomials just described. See [Lemma 5.3](#) for an example. The quantities D , N_R , T and $\theta(\mathcal{P}, \mathcal{M}, \ell)$ depend on R and on the instance vector $\mathbf{v}$. We omit this dependence from the notation.

Lemma 5.3 (Example definition of $\mathcal{M}_q$ and the corresponding $\theta(\mathcal{P}, \mathcal{M}, \ell)$). *Let $\mathcal{P}$ be a set of primes. For each $q \in \mathcal{P}$ let $\mathcal{M}_q$ be the set of all maps $\pi : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ of the form*

$$\pi(h(X_1, \dots, X_k)) = \pi_q(h)(\alpha_1, \dots, \alpha_k), \quad \text{for } h \in \mathbb{Z}[X_1, \dots, X_k],$$

for some $\alpha_1, \dots, \alpha_k \in \mathbb{F}_q$, and where $\pi_q(h)$ is h after reducing its coefficients modulo q . Then

$$\theta(\mathcal{P}, \mathcal{M}, \ell) \leq \frac{2 \cdot D}{q_{\min}} + \frac{\log(4 \cdot T \cdot \ell \cdot N_R^2)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor}.$$

where $q_{\min}$ is the smallest prime in $\mathcal{P}$.

Proof. We defer the proof to later in [Section 5.4](#). $\square$

Construction 5.4 (IOP for the core $\mathcal{R}_{\text{LinBitsRings}}$ relation). **Parameters.** $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$ comprised of: a witness vector length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 satisfying $\ell_1 \cdot \ell_2 = \ell$; the IOP Π_{iopp} described above, the IOR Π_{red} from [Section 5.2](#); and the sets $\mathcal{P}, \mathcal{M}_q$ from above.

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. We assume the tensor split from the previous [Section 5.2](#), and adopt its notation.

Let $g \in \mathbb{F}_{2^\nu}$ be a fixed generator of the multiplicative group $\mathbb{F}_{2^\nu}^*$. Denote π_{canon}^R simply as π_{canon} .

1. **Round 0 (optional): Out Of Domain (OOD) sample.** *Albert: I think this is not implemented?* This step can be skipped if the proximity parameter δ is set within the unique decoding radius of $\mathcal{C}$ (cf. [Section 4.2](#), and [Theorem 5.8](#)). Otherwise the step is executed. In that case, the verifier samples and sends a random $\zeta \in \mathbb{F}_{2^\nu}$, and the prover responds with

$$y = \widetilde{\text{Pack}_\nu \circ \pi_2(\mathbf{f})}(\zeta), \quad \text{where } \zeta := (\zeta^{2^0}, \zeta^{2^1}, \dots, \zeta^{2^{\log(\ell/\nu)-1}}).$$

2. **Round 1 (optional): replacing R by a “small” prime field $\mathbb{F}_q$.** This step can be skipped (i.e. prover and verifier move to Round 2 directly) if R is a prime field $\mathbb{F}_p$ and $p < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$; in that case we set $q := p$ in Rounds 2 and 3. Otherwise the step is executed.

The prover computes and sends $\mu' \in \mathbb{Z}[X_1, \dots, X_k]$ such that

$$\langle \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \mathbf{f} \rangle = \mu' \quad \text{in } \mathbb{Z}[X_1, \dots, X_k].$$

The verifier checks that μ' has total degree at most $2 \cdot D$ and $\|\mu'\|_\infty \leq T \cdot \ell \cdot N_R^2$, then it checks that $\pi_{\text{canon}}(\mu') = \mu$, and replies with a random prime $q \in \mathcal{P}$ and a random morphism $\pi_{\text{rand}} : \mathbb{Z}[X_1, \dots, X_k] \rightarrow \mathbb{F}_q$ from $\mathcal{M}_q$. The prover and verifier update:

$$\begin{aligned} \mathbf{v}^{(1)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \in \mathbb{F}_q^{\ell_1}, \\ \mathbf{v}^{(2)} &\leftarrow \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \in \mathbb{F}_q^{\ell_2}, \\ \mu &\leftarrow \pi_{\text{rand}}(\mu') \in \mathbb{F}_q, \end{aligned}$$

The prover and verifier now set to prove the updated claim

$$\langle \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}, \pi_q(\mathbf{f}) \rangle = \mu \quad \text{over } \mathbb{F}_q. \tag{22}$$

Remark 5.5 (Skipping Round 1 when $R = \mathbb{Z}$ and $\|\mathbf{v}\|_\infty$ is “small”). *Albert: Make sure our completeness and soundness theorem proves this case as well* This step can also be skipped when $R = \mathbb{Z}$ and all entries of $\mathbf{v}^{(1)}$ are nonnegative and less than $(|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. In that case the claim to prove at the start of Round 2 is $\langle \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}, \mathbf{f} \rangle = \mu$ over $\mathbb{Z}$. In this case the map π_q in Round 2 is simply replaced by the identity map, $\mathbb{F}_q$ is replaced by $\mathbb{Z}$, and q in the range checks of Round 2 denotes $\max_{i \in [\ell_1]} \mathbf{v}_i^{(1)} + 1$. In Phase 3, γ is $\mathbf{v}^{(1)}$.

3. **Round 2: transforming (22) to ℓ_2 integer constraints with bounded integers.**

- The prover computes and sends integers $\mu_1, \dots, \mu_{\ell_2} \in [0..(q-1) \cdot \ell_1]$ such that, over $\mathbb{Z}$,

$$\langle \mathbf{f}_{*j}, \pi_q^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j \quad \text{for all } j \in [\ell_2]. \quad (23)$$

- The verifier checks that $\mu_j \in [0..(q-1) \cdot \ell_1]$ for all $j \in [\ell_2]$, and that

$$\sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \mathbf{v}_j^{(2)} = \mu \quad \text{over } \mathbb{F}_q. \quad (24)$$

Next, prover and verifier will focus on certifying that (23) holds for all $j \in [\ell_2]$.

In [Remark 5.6](#) we describe a variation in which, instead of sending the elements $\mu_1, \dots, \mu_{\ell_2}$, the prover commits to their bit-decomposition over $\mathbb{F}_{2^\nu}$, and proves (24) with [Construction 5.14](#). This saves on proof size and verifier time when ℓ is large.

4. **Phase 3: Transforming the claim into a PESAT claim over $\mathbb{F}_{2^\nu}$, and invoking the IOR Π_{red} .**

- Prover and verifier compute and define

$$\gamma = \pi_q^{-1}(\mathbf{v}^{(1)}) \in [0..q-1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1},$$

and set to prove the following claim:

$$\prod_{i \in [\ell_1]} g^{\gamma_i \cdot \mathbf{f}_{ij}} = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2].$$

This claim is equivalent to

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2].$$

Notice that

$$(\mathbf{x} = ((g^{\gamma_i})_{i \in [\ell_1]}, (g^{\mu_j})_{j \in [\ell_2]}), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu). \quad (25)$$

Let $\boldsymbol{\mu} := (\mu_1, \dots, \mu_{\ell_2})$, $g^\gamma := (g^{\gamma_i})_{i \in [\ell_1]}$ and $g^\mu := (g^{\mu_j})_{j \in [\ell_2]}$.

- To prove (25), prover and verifier engage in the IOR Π_{red} ⁴ with input

$$(\mathbf{x} = (g^\gamma, g^\mu), \mathbf{y} = f, \mathbf{w} = \pi_2(\mathbf{f})).$$

At the end of this IOR, they are left with the following claim to prove:

$$((\mathbf{z}, c), f, \pi_2(\mathbf{f})) \in \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{F}_{2^\nu}^\ell \times \mathbb{F}_{2^\nu}, \\ \mathbf{y} = g \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{g} \in \mathbb{F}_{2^\nu}^\ell \end{array} \right) \middle| \begin{array}{l} g = \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{g}) \\ \langle \mathbf{u}, \mathbf{g} \rangle = \mu \end{array} \right\} \quad (26)$$

for some vector $\mathbf{z} \in \mathbb{F}_{2^\nu}^\ell$ and field element $c \in \mathbb{F}_{2^\nu}$ output during the IOR Π_{red} .

5. **Phase 4: Using an IOP over $\mathbb{F}_{2^\nu}$.** Prover and verifier engage in the IOP Π_{iopp} to certify the claim (26). If Round 0 (Out Of Domain sampling) triggered, they also use Π_{iopp} to certify the evaluation claim from Round 0.

In [Table 4](#) we summarize the main costs of [Construction 5.4](#).

⁴Recall that, in our instantiation, Π_{red} is the GKR protocol applied to multiple grand products, cf. [Section 6](#).

Costs ($k \leq 1$). In Table 4 we summarize the costs of Construction 5.4 beyond those of Π_{red} and Π_{iopp} (cf. Sections 5.7 and 6), for the specific case $k \leq 1$, corresponding to R being of the form $\mathbb{Z}_n$ or $\mathbb{Z}_n[X]/(h(X))$ for some $n \geq 0$ and some $h(X)$. This is the setting used in our instantiation Sections 5.7 and 7 **Albert: Do we do any Babybear extension? If not, we only use $k = 0$.** The number of rounds is between $t_{\text{red}} + t_{\text{iopp}}$ and $2 + t_{\text{red}} + t_{\text{iopp}}$, depending on whether Rounds 0 and 1 execute or not, where $t_{\text{red}}, t_{\text{iopp}}$ denote the number of rounds of $\Pi_{\text{iopp}}, \Pi_{\text{red}}$.

Rounds 0, 1 and 2 send only non-oracle messages, so the oracles and queries of Construction 5.4 are exclusively those of Π_{red} and Π_{iopp} . If Round 1 is skipped, q denotes $|R|$ when $R = \mathbb{F}_q$, and $\max_i \mathbf{v}_i^{(1)} + 1$ when $R = \mathbb{Z}$ (cf. Remark 5.5). The exponentiations of Phase 3, when computing $(g^{\gamma_i})_{i \in [\ell_1]}$ and $(g^{\mu_j})_{j \in [\ell_2]}$, take $\lceil \log(q \cdot \ell_1)/w \rceil \leq \nu/w$ multiplications each using a fixed-base table with w -bit windows ($w = 8$ in our implementation).

In our instantiation, we set $\ell_1 \approx \sqrt{\ell}$, $\ell_2 \approx \sqrt{\ell}$, making the verifier succinct. We assume implicitly that the instance is given to the verifier as $(\mu, \mathbf{v}^{(1)}, \mathbf{v}^{(2)})$, so that it does not read $\mathbf{v}$ in full.

When $k = 0$ (so that $R = \mathbb{Z}_n$ for some n), we have $D = 0$ and $T = 1$. When $k = 1$ and $R = \mathbb{Z}_n[X]/(h(X))$ for some monic $h(X)$, then $D \leq \deg(h(X)) - 1$ and $T \leq D + 1$, cf. (21) for a definition of these quantities. If $n > 0$, then $N_R \leq |\mathbb{Z}_n| = n$. Otherwise if $n = 0$, $N_R \leq \|(\mathbf{v}^{(1)}, \mathbf{v}^{(2)})\|_\infty$.

Round 0	Prover Verifier Proof (bits)	$O(\ell/\nu) \cdot \mathbb{F}_{2^\nu}$ – ν
Round 1	Prover Verifier Proof (bits)	$O(\ell \cdot (D + 1))$ additions of $\log(\ell_1 \cdot N_R)$ -bit integers $O(\ell_2 \cdot (D + 1)^2)$ additions of $\log(\ell \cdot N_R^2)$ -bit integers $O(\ell_2 \cdot (D + 1)^2)$ multiplications of $\log(\ell \cdot N_R^2)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1))$ reductions mod q of $\log(N_R)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1)) \cdot \mathbb{F}_q$ $O(D + 1)$ reductions mod $\text{char}(R)$ of $\log((D + 1) \cdot \ell \cdot N_R^2)$ -bit integers $O(D + 1) \cdot R$ $O((\ell_1 + \ell_2) \cdot (D + 1))$ reductions mod q of $\log(N_R)$ -bit integers $O((\ell_1 + \ell_2) \cdot (D + 1)) \cdot \mathbb{F}_q$ $(2D + 1) \cdot \log((D + 1) \cdot \ell \cdot N_R^2)$
Round 2	Prover Verifier Proof (bits)	$O(\ell)$ additions of $\log(q \cdot \ell_1)$ -bit integers $O(\ell_2)$ reductions mod q of $\log(q \cdot \ell_1)$ -bit integers $O(\ell_2) \cdot \mathbb{F}_q$ $\ell_2 \cdot \log(q \cdot \ell_1)$
Phase 3	Prover Verifier Proof (bits)	$O((\ell_1 + \ell_2) \cdot \nu/w) \cdot \mathbb{F}_{2^\nu}$ same as prover –

Table 4: Costs of Construction 5.4 when $k \leq 1$, in addition to those of Π_{red} and Π_{iopp} . By $N \cdot \mathbb{F}_{2^\nu}$, $N \cdot \mathbb{F}_q$, $N \cdot R$ we indicate N operations in the respective ring. Rounds 0 and 1 are optional.

Albert: Was AI generated and reviewed and edited by me after We justify the costs listed in Round 1 of Table 4, which are arguably the most involved to derive. The first three Prover costs in Round 1 come from the computation of μ' . The prover can compute μ' by first building, for

each $j \in [\ell_2]$, the polynomial $h_j := \sum_{i \in [\ell_1]} \mathbf{f}_{ij} \cdot \pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)})$, which requires no multiplications since $\mathbf{f}$ has bit entries. The coefficients of these polynomials are bounded by $\ell_1 \cdot N_R$. Hence the h_j can be computed with $O(\ell \cdot (D+1))$ additions of $\log(\ell_1 \cdot N_R)$ -bit integers. Now, to obtain μ' the prover computes $\sum_{j \in [\ell_2]} \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)}) \cdot h_j$. Each product in this sums costs at most $(D+1)^2$ multiplications of $\log(\ell_1 \cdot N_R)$ -bit integers, and there are ℓ_2 of them, this yields the third cost in the prover's Round 1. The second cost correspond to adding together each of the terms in $\sum_{j \in [\ell_2]} \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)}) \cdot h_j$.

The last two cost of the prover and verifier in Round 1 are the costs of computing $\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(i)})$ for $i = 1, 2$. For both i , this is the cost of reducing $D+1$ integers in the range $[0..N_R-1]$ modulo a prime, and then computing a linear combination of the resulting elements.

The first two verifier costs in Round 1 correspond to computing $\pi_{\text{canon}}(\mu')$. For that, it first reduces the at most $2D+1$ coefficients of μ' (which are integers of at most $\log(T \cdot \ell \cdot N_R^2)$ bits) modulo $\text{char}(R)$ and evaluates the resulting polynomial at η_1 in R using Horner's rule, and then compares the result with μ .

Remark 5.6 (Reducing proof size by committing to the elements $\mu_1, \dots, \mu_{\ell_2}$). *Albert: How does the prover get the g^{μ_j} afterward though? Prover must prove an MLE evaluation of $(g^{\mu_j})_j$* When $\ell = \ell_1 \cdot \ell_2$ is such that ℓ_2 is large, the elements $\mu_1, \dots, \mu_{\ell_2} \in \mathbb{Z}$ sent at Round 2 of [Construction 5.4](#) may start to dominate the proof size. For example, for $\ell_2 = 2^{13}$ and μ_i of 128 bits, the elements $\mu_1, \dots, \mu_{\ell_2}$ amount to 131 KB. To mitigate this cost, one can have the prover commit to the vector $(\mu_1, \dots, \mu_{\ell_2})$ and then prove [\(24\)](#), i.e.

$$\langle \mathbf{f}_{*j}, \pi_q^{-1}(\mathbf{v}^{(1)}) \rangle = \mu_j \quad \text{for all } j \in [\ell_2].$$

using an IOP for constrained codes. Concretely, the prover commits to the vector $\text{bits}(\mu)$ of bits forming the elements $(\mu_1, \dots, \mu_{\ell_2})$ using $\text{Bit}\mathbb{Z}$, and then proves [\(24\)](#) using [Construction 5.14](#) from [Section 5.6](#).

Remark 5.7 (Succinct verifier and proofs). By [Table 4](#), choosing $\ell_1 = O(\sqrt{\ell})$ and $\ell_2 = O(\sqrt{\ell})$, the verifier and the proof size of [Construction 5.4](#) have asymptotic cost $O(\sqrt{\ell})$ (where ℓ is the bit-length of the witness).

5.4 Soundness and completeness

For readability, we temporarily denote

$$\mathcal{R}_{\text{PESAT}} = \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{Bit}\mathbb{Z}}, \mathcal{C}, \text{Pack}_\nu), \quad \mathcal{R}_{\text{Lin}} = \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu).$$

Theorem 5.8 (Completeness and round-by-round knowledge soundness of [Construction 5.4](#)). *Let $\delta \in (0, 1)$ be a proximity parameter within the unique decoding radius of $\mathcal{C}$ (cf. [Section 4.2](#)), and assume that Round 0 of [Construction 5.4](#) is skipped. We treat the general case in [Section A.2](#) Assume*

$$\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1.$$

Assume Π_{red} is a perfectly complete t_{red} -round IOR from $\mathcal{R}_{\text{PESAT}}$ to $\mathcal{R}_{\text{Lin}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta)$, with errors $(\kappa_{\text{red}1}, \dots, \kappa_{\text{red}t_{\text{red}}})$. Assume Π_{iopp} is a perfectly complete t_{iopp} -round IOP for $\mathcal{R}_{\text{Lin}}$ (i.e. an IOR from $\mathcal{R}_{\text{Lin}}$ to the trivial relation $\perp$) with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{Lin}}^\delta, \perp)$, and errors

$$\kappa_{\text{iopp}1}, \dots, \kappa_{\text{iopp}t_{\text{iopp}}}.$$

Then [Construction 5.4](#) is a perfectly complete $(1 + t_{\text{red}} + t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{LinBitsRings}}$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \perp)$, with errors

$$\kappa_0, (\kappa_{\text{red}_i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}_i})_{i \in [t_{\text{iopp}}]},$$

where

$$\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$$

(see [Lemma 5.3](#)). If Round 1 is not executed, the error term κ_0 and its round are absent, and [Construction 5.4](#) has $t_{\text{red}} + t_{\text{iopp}}$ rounds.

To prove [Lemma 5.3](#) we will use the following result, which generalizes [Albert: point to lemma](#) in [\[AGK⁺26\]](#).

Lemma 5.9. *Let $h \in \mathbb{Z}[X_1, \dots, X_m]$ be a nonzero polynomial of total degree $\leq d$ and with coefficients in the range $[-2^B + 1, 2^B - 1]$. Let $\mathcal{P}$ be a set of primes, and let $q_{\min}$ be the smallest prime in $\mathcal{P}$. Given a prime q and $\alpha \in \mathbb{F}_q^m$, write $\pi_{q,\alpha}(h) := \pi_q(h)(\alpha) \in \mathbb{F}_q$. Then*

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^m}} [\pi_{q,\alpha}(h) = 0] \leq \frac{B}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{d}{q_{\min}}.$$

Proof. Let $\mathcal{E}_{\text{prime}}$ be the event that all coefficients of h are multiples of q . Then, by the Schwartz–Zippel lemma,

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^m}} [\pi_{q,\alpha}(h) = 0 \mid \neg \mathcal{E}_{\text{prime}}] \leq \frac{d}{q_{\min}}.$$

Now, every nonzero coefficient of h is divisible by at most $B/\lfloor \log(q_{\min}) \rfloor$ primes from $\mathcal{P}$. Hence, the probability that $\mathcal{E}_{\text{prime}}$ occurs is at most $(B/\lfloor \log(q_{\min}) \rfloor)/|\mathcal{P}|$, and the lemma follows. $\square$

Proof of [Lemma 5.3](#). Let $h \in \mathbb{Z}[X_1, \dots, X_k]$ be a polynomial of total degree at most $2 \cdot D$, with coefficients in $[-2 \cdot T \cdot \ell \cdot N_R^2, 2 \cdot T \cdot \ell \cdot N_R^2]$, and with $\pi_{\text{canon}}^R(h) \neq 0$ (cf. the definition of $\theta(\mathcal{P}, \mathcal{M}, \ell)$). In particular h is nonzero, and its coefficients lie in $[-2^B + 1, 2^B - 1]$ for $B := \lfloor \log(T \cdot \ell \cdot N_R^2) \rfloor + 2 \leq \log(4 \cdot T \cdot \ell \cdot N_R^2)$. Since h is nonzero and has total degree at most $2 \cdot D$, [Lemma 5.9](#) (applied with $m = k$ variables and total degree $2 \cdot D$) yields

$$\Pr_{\substack{q \leftarrow \mathcal{P}, \\ \alpha \leftarrow \mathbb{F}_q^k}} [\pi_q(h)(\alpha) = 0] \leq \frac{B}{\lfloor \log(q_{\min}) \rfloor \cdot |\mathcal{P}|} + \frac{2 \cdot D}{q_{\min}},$$

as needed. $\square$

Proof of [Theorem 5.8](#). The completeness of [Construction 5.4](#) is clear from its description [Albert: Review](#). Recall that, by hypothesis, Round 0 (OOD sampling) is skipped; the case where Round 0 is executed is treated in [Section A.2](#).

We decompose [Construction 5.4](#) into three IORs Π_1, Π_2, Π_3 . The first one, Π_1 , is an IOR from $\mathcal{R}_{\text{LinBitsRings}}$ to $\mathcal{R}_{\text{PESAT}}$, and it consists of applying Rounds 1 and 2 of [Construction 5.4](#), and outputting

$$(\mathbf{x}' = (g^\gamma, g^\mu), \mathbf{y}' = f, \mathbf{w}' = \pi_2(\mathbf{f})),$$

as described in Phase 3 of [Construction 5.4](#). Formally, Π_1 is a 2-round IOR whose second verifier message is empty: the prover first sends μ' , the verifier replies with (q, π_{rand}) , and the prover then sends $\mu = (\mu_1, \dots, \mu_{\ell_2})$. The second IOR, Π_2 , is simply Π_{red} , reducing $\mathcal{R}_{\text{PESAT}}$ to $\mathcal{R}_{\text{Lin}}$. Finally, the third IOR, Π_3 , is simply the IOP Π_{iopp} for $\mathcal{R}_{\text{Lin}}$.

Below we show that Π_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \mathcal{R}_{\text{PESAT}}^\delta)$, with errors $(\kappa_0, 0)$, assuming Round 1 is not skipped. The second verifier message of Π_1 is empty, hence the last prover message of Π_1 can be sent together with the first prover message of Π_2 , and thus [Construction 5.4](#) has $1 + t_{\text{red}} + t_{\text{iopp}}$ rounds. The theorem then follows from the composition of IORs result (cf. [Lemma 4.7](#)).

At the end of the proof we discuss the case when Round 1 is skipped.

Completeness and soundness of Π_1 We describe a knowledge state function KState . Let $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^\ell$ and $\mathbf{y} = f \in \mathbb{F}_2^m$. Let tr be a partial interaction transcript for Π_1 . We start by setting KState so that the first condition in [Definition 4.5](#) is met. Precisely, if $\text{tr} = \emptyset$, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}^\delta$. Further, if $\text{tr} = (\mu')$ consists of a single prover message, we set $\text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathbf{w}) = \text{KState}(\mathbf{x}, \mathbf{y}, \emptyset, \mathbf{w})$, so that the second condition in [Definition 4.5](#) holds for the first prover message round.

After the verifier's message, tr is as follows:

$$\text{tr} = (\mu', (q, \pi_{\text{rand}})) \in \mathbb{Z}[X_1, \dots, X_k] \times (\mathcal{P} \times \mathcal{M}_q).$$

We define $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier's degree and range checks on μ' in Round 1 pass, $\mathbf{w} = \mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$, $\mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in (\{0, 1\}^{\ell_1})^{\ell_2}$, and

$$\begin{aligned} \pi_{\text{canon}}(\mu') &= \mu, \\ \langle \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \pi_q(\mathbf{f}) \rangle &= \pi_{\text{rand}}(\mu') \quad \text{over } \mathbb{F}_q, \\ \Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) &< \delta. \end{aligned} \tag{27}$$

We continue with our definition of KState . Let $\text{tr} = (\mu', (q, \pi_{\text{rand}}), \mu)$ for a prover message $\mu = (\mu_1, \dots, \mu_{\ell_2}) \in [0..(q-1) \cdot \ell_1]^{\ell_2}$, or let tr be the full transcript obtained by appending the empty second verifier message to $(\mu', (q, \pi_{\text{rand}}), \mu)$. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if the verifier accepts on tr (i.e. its checks in Rounds 1 and 2 pass) and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$, where $(\mathbf{x}', \mathbf{y}')$ is the output of the verifier, namely

$$\mathbf{x}' = (g^\gamma, g^\mu), \quad \mathbf{y}' = f, \quad \text{with } \gamma = \pi_q^{-1} \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \right) \in [0..q-1]^{\ell_1} \subseteq \mathbb{Z}^{\ell_1}.$$

Thus KState satisfies the third condition in [Definition 4.5](#).

We define an extractor as $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = \mathbf{w}$. It remains to prove two statements: first, that

$$\Pr_{q, \pi_{\text{rand}}} \left[\exists \mathbf{w} \left| \begin{array}{l} \text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w})) = 0 \\ \wedge \text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1 \end{array} \right. \right] \leq \kappa_0;$$

and second, that KState satisfies the second condition in [Definition 4.5](#) for transcripts where the prover is about to send μ (its second message).

We start with the first statement. Assume

$$\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1 \quad \text{and} \quad \text{KState}(\mathbf{x}, \mathbf{y}, \mu', \mathbf{w}) = \text{KState}(\mathbf{x}, \mathbf{y}, \mathbf{w}) = 0.$$

The latter occurs if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$. The former implies that $\mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^\ell$ satisfies (27); in particular $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$, and hence, since $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$, necessarily

$$\langle \pi(\mathbf{f}), \mathbf{v} \rangle \neq \mu \quad \text{over } R. \quad (28)$$

Define the polynomial

$$h := \langle \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \mathbf{f} \rangle - \mu' \in \mathbb{Z}[X_1, \dots, X_k].$$

Since π_{canon} is a ring homomorphism satisfying $\pi_{\text{canon}} \circ \pi_{\text{canon}}^{-1} = \text{id}$ and restricts to π on $\mathbb{Z}$, the first condition in (27) gives

$$\pi_{\text{canon}}(h) = \langle \pi(\mathbf{f}), \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)} \rangle - \pi_{\text{canon}}(\mu') = \langle \pi(\mathbf{f}), \mathbf{v} \rangle - \mu,$$

which is nonzero by (28). In particular $h \neq 0$. On the other hand, since π_{rand} is a ring homomorphism restricting to π_q on $\mathbb{Z}$, the second condition in (27) states precisely that $\pi_{\text{rand}}(h) = 0$. We claim that h has total degree at most $2 \cdot D$ and satisfies

$$\|h\|_\infty \leq 2 \cdot T \cdot \ell \cdot N_R^2.$$

Indeed, the lifts $\pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)})$ and $\pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)})$ have total degree at most D and coefficients of absolute value at most N_R , so each product $\pi_{\text{canon}}^{-1}(\mathbf{v}_i^{(1)}) \cdot \pi_{\text{canon}}^{-1}(\mathbf{v}_j^{(2)})$ has total degree at most $2 \cdot D$ and coefficients of absolute value at most $T \cdot N_R^2$ by the definition of T . Further, by the verifier's checks in Round 1,

$$\|\mu'\|_\infty \leq T \cdot \ell \cdot N_R^2$$

and the total degree of μ' is at most $2 \cdot D$. The claim follows now from the fact that the entries of $\mathbf{f}$ are bits and there are ℓ of them.

Since additionally $\pi_{\text{canon}}(h) \neq 0$, the polynomial h belongs to the class of polynomials over which $\theta(\mathcal{P}, \mathcal{M}, \ell)$ is defined. Finally, since δ is within the unique decoding radius of $\mathcal{C}$, the code $\mathcal{C}$ is $(\delta, 1)$ -list-decodable, and hence there is at most one $\mathbf{w}$ such that $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{w})) < \delta$. Therefore [Albert: review](#)

$$\begin{aligned} & \Pr_{q, \pi_{\text{rand}}} [\exists \mathbf{w} \text{ such that } (\mathbf{x}, \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta \text{ and } \text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1] \\ & \leq \Pr_{q \leftarrow \mathcal{P}, \pi_{\text{rand}} \leftarrow \mathcal{M}_q} [\pi_{\text{rand}}(h) = 0] \leq \theta(\mathcal{P}, \mathcal{M}, \ell) = \kappa_0, \end{aligned}$$

where the last inequality holds by the definition of $\theta(\mathcal{P}, \mathcal{M}, \ell)$.

It remains to prove the second statement, namely: if $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 0$, then $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}}), \boldsymbol{\mu}), \mathbf{w}) = 0$ for every prover message $\boldsymbol{\mu} = (\mu_1, \dots, \mu_{\ell_2}) \in [0..(q-1) \cdot \ell_1]^{\ell_2}$. Suppose towards a contradiction that $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}}), \boldsymbol{\mu}), \mathbf{w}) = 1$, i.e. that the verifier accepts and $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$. Then $\mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in \{0, 1\}^\ell$, $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$, and, for all $j \in [\ell_2]$,

$$g^{\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle} = \prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j},$$

and hence

$$\langle \mathbf{f}_{*j}, \boldsymbol{\gamma} \rangle = \mu_j \pmod{\text{ord}(g)},$$

where $\text{ord}(g)$ denotes the multiplicative order of g . Now, since $\gamma_i \in [0..q-1]$, $\mu_j \in [0..(q-1) \cdot \ell_1]$, and $\mathbf{f}_{*j}$ contains bits, we have $|\langle \mathbf{f}_{*j}, \gamma \rangle - \mu_j| \leq \ell_1 \cdot (q-1)$. Since g is a generator of $\mathbb{F}_{2^\nu}^*$ and $q \leq \max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, we have

$$\text{ord}(g) = |\mathbb{F}_{2^\nu}| - 1 > \ell_1 \cdot \max \mathcal{P} \geq \ell_1 \cdot q > \ell_1 \cdot (q-1),$$

and hence

$$\langle \mathbf{f}_{*j}, \gamma \rangle = \mu_j \quad \text{over } \mathbb{Z}, \quad \text{for all } j \in [\ell_2]. \quad (29)$$

Applying π_q to (29), and using that $\pi_q(\gamma) = \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)})$, we obtain

$$\langle \pi_q(\mathbf{f}_{*j}), \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle = \pi_q(\mu_j) \quad \text{over } \mathbb{F}_q, \quad \text{for all } j \in [\ell_2].$$

Since the verifier accepts, its check (24) in Round 2 passes, i.e. $\sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot (\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}))_j = \pi_{\text{rand}}(\mu')$. Now

$$\begin{aligned} & \langle \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \otimes \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}), \pi_q(\mathbf{f}) \rangle \\ &= \sum_{j \in [\ell_2]} \langle \pi_q(\mathbf{f}_{*j}), \pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(1)}) \rangle \cdot \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \right)_j \\ &= \sum_{j \in [\ell_2]} \pi_q(\mu_j) \cdot \left(\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}(\mathbf{v}^{(2)}) \right)_j = \pi_{\text{rand}}(\mu'). \end{aligned}$$

Hence the second condition in (27) holds. The first condition in (27), i.e. $\pi_{\text{canon}}(\mu') = \mu$, holds because the verifier accepts, and the third one, i.e. $\Delta(f, \text{Enc}_C \circ \text{Pack}_\nu \circ \pi_2(\mathbf{f})) < \delta$, holds because $(\mathbf{x}', \mathbf{y}', \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}^\delta$. Hence $\text{KState}(\mathbf{x}, \mathbf{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1$, a contradiction.

The case where Round 1 is skipped. [Albert: Review that I'm not missing any important argument](#) In this case, the messages μ' and (q, π_{rand}) and the error κ_0 disappear, and essentially the arguments above apply after replacing $\pi_{\text{rand}} \circ \pi_{\text{canon}}^{-1}$ with the identity map on $\mathbb{F}_q$ or on $\mathbb{Z}$. In the latter case, π_q is also replaced by the identity map, cf. [Remark 5.5](#). □

5.5 The general LinBitsRings relation

We now proceed to describe the full relation we seek to work with in this section. Compared to $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$, the witness $\mathbf{f}$ will now have entries in an arbitrary finitely generated ring S , instead of in $\{0, 1\} \subseteq \mathbb{Z}$.

The witness entries will be restricted to a bounded subset of S , so that each witness can be expressed as a pre-specified linear expression (over S) of a string of bits.

We set the following additional notation:

- Let S be a ring, possibly different than R , let $\psi : S \rightarrow R$ be a ring homomorphism.
- Let $D_S \geq 1$ be a degree bound, and let $\gamma_1 = 1, \gamma_2, \dots, \gamma_d \in S$ be the monomials of total degree less than D_S in a fixed set of generators of S ([Section 4.1](#)), so that d is the number of such monomials. Concretely $d \leq \binom{D_S - 1 + k_S}{k_S}$, where $1, \eta_1, \dots, \eta_{k_S}$ are the generators of S .

- Recall from [Section 4.1](#) that $S_{\leq B}^{<D_S}$ denotes the set of elements of S of the form $h = \sum_{i \in [d]} h_i \cdot \gamma_i$ for

$$h_i \in [0..2^B - 1]_S \subseteq S,$$

and that for a vector of integers $\mathbf{w}$, we write $\mathbf{w}_S$ for its projection to S via reduction modulo $\text{char}(S)$ (see [\(20\)](#)).

- Let ℓ_0 be a vector witness length, so that our witnesses are $\mathbf{f} \in S^{\ell_0}$.

We will use the natural homomorphism $\pi_{\text{char}(S)} : \mathbb{Z} \rightarrow S$ consisting in reducing an element modulo the characteristic of S , see [\(19\)](#) in [Section 4.1](#).

Definition 5.10 (Bitification of the witness). Let $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$. Let ℓ be a vector length, called *bitification length*. Let $L \in S^{\ell_0 \times \ell}$ be a matrix, called *bitification matrix*. Let $\mathbf{g} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$ be a vector filled with integer bits. We say $\mathbf{g}$ is a *bitification* of $\mathbf{f}$ (with respect to L) if

$$\mathbf{f} = L \cdot \mathbf{g}_S.$$

where for a vector $\mathbf{w}$ of integer bits, $\mathbf{w}_S$ denotes its natural projection to a vector with entries the elements $\{0, 1\}_S \subseteq S$. Additionally, we require that⁵

$$L \cdot \mathbf{g}_S \in (S_{\leq B}^{<D_S})^{\ell_0} \quad \text{for all } \mathbf{g} \in \{0, 1\}^\ell.$$

This condition holds for the standard bitification matrix below.

The choice of L and a method for building a bitification of $\mathbf{f}$ can be customized. Below, we describe what we call the *standard bitification* procedure (see below). This can be used for arbitrary finitely generated rings S and bit bounds B . In some scenarios (see [Remark 5.12](#)), one may choose different more convenient approaches.

For a fixed L and $\mathbf{f}$, bitifications of $\mathbf{f}$ are not unique in general. Nevertheless, abusing the notation, we often denote them as

$$\text{bits}(\mathbf{f}).$$

A *standard bitification* of $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$, denoted $\text{bits}(\mathbf{f})$, is obtained as follows: write $\mathbf{f} = \sum_{j \in [d]} \mathbf{f}_{*j} \cdot \gamma_j$ for some $\mathbf{f}_{*j} \in ([0..2^B - 1]_S)^{\ell_0}$, $j \in [d]$ (this expression is not unique). Then write

$$\mathbf{f}_{*j} = \sum_{k=1}^B \mathbf{f}_{*jk} \cdot \pi_{\text{char}(S)}(2^{k-1})$$

for some vectors

$$\mathbf{f}_{*jk} \in \{0, 1\}_S^{\ell_0} \subseteq S^{\ell_0},$$

for all j, k . We define

$$\begin{aligned} \ell &:= \ell_0 \cdot d \cdot B, \\ \text{bits}(\mathbf{f}) &\in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{aligned}$$

⁵This condition is required to guarantee that extractors obtain witnesses in $S_{\leq B}^{<D_S}$. In some applications, proving that the prover knows a witness in S may suffice, and in that case the condition can be dropped.

as the vector with integer bit entries obtained by concatenating the vectors $\mathbf{f}_{*jk}$ and lifting and reordering their entries to $\{0, 1\} \subseteq \mathbb{Z}$ in a natural way, i.e.

$$\text{bits}(\mathbf{f}) := (\text{lift}(\mathbf{f}_{ijk}))_{i \in [\ell_0], k \in [B], j \in [d]} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell, \quad (30)$$

where lift is the identity function if $\text{char}(S) = 0$, and $\text{lift} = \pi_{\text{char}(S)}^{-1}$ if $\text{char}(S) > 0$. Defining

$$L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, 2, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \in S^{\ell_0 \times \ell},$$

we have

$$\mathbf{f} = L \cdot \text{bits}(\mathbf{f})_S.$$

Assume $\mathcal{C}$ is a linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν and blocklength m .

Definition 5.11. The *general LinBitsRings* relation is defined as [Albert](#): Here we could allow for $\psi = \phi \circ \pi_{\text{canon}}^S$ for $\phi : \mathbb{Z}[X_1, \dots, X_k] \rightarrow R$ a morphism. This is what is actually happening and it allows us to build later full IOPs for constraints over arbitrary f.g. rings

$$\begin{aligned} \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L) \\ := \left\{ \left(\begin{array}{l} \mathbf{x} = (\mu, \mathbf{v}) \in R \times R^{\ell_0}, \\ \mathbf{y} = f \in \mathbb{F}_{2^\nu}^m, \\ \mathbf{w} = \mathbf{f} \in (S_{\leq B}^{< D_S})^{\ell_0} \end{array} \right) \middle| \begin{array}{l} f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f})), \\ \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R, \\ \mathbf{f} = L \cdot (\text{bits}(\mathbf{f}))_S \text{ for some } \text{bits}(\mathbf{f}) \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell \end{array} \right\}. \end{aligned}$$

When there is no risk of confusion, we denote $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ simply by $\mathcal{R}_{\text{LinBitsRings}}$. The Core LinBitsRings relation is precisely $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ after taking $S = \mathbb{Z}$, $B = 1$, $\psi = \pi_{\text{char}(R)}$, and $L = \text{Id}_{\ell \times \ell}$ (the standard bitification matrix in this case, so that $\ell_0 = \ell$).

For $\delta \in (0, 1)$, we let $\mathcal{R}_{\text{LinBitsRings}}^\delta$ be the relation $\mathcal{R}_{\text{LinBitsRings}}$ after replacing the constraint $f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))$ by $\Delta(f, \text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\text{bits}(\mathbf{f}))) < \delta$, as in [Section 5.1](#).

Remark 5.12 (Non-uniform bit-decomposition sizes). In practical settings, it is possible that not all entries of $\mathbf{f}$ have the same bit size. For example, some chunks of $\mathbf{f}$ may be comprised of elements from $\{0, 1\}_S$, while others may be comprised of generic ring elements. However, the standard bitification procedure from [Definition 5.10](#) allocates $d \cdot B$ bits for every entry of $\mathbf{f}$, no matter its size. To avoid this, one can find alternative approaches to defining L and $\text{bits}(\mathbf{f})$.

For efficiency purposes, one needs to make sure that L has enough structure to later support tensor decomposing certain vectors, as discussed in [Remark 5.15](#). In any case, one can always use the matrix L above, which in general does possess enough structure as argued in [Remark 5.15](#).

5.6 The Bit $\mathbb{Z}$ IOP for the general LinBitsRings relation

Next we describe our IOP for the general relation $\mathcal{R}_{\text{LinBitsRings}}$. We assume the notation of [Section 5.5](#). Let L be a *bitification* matrix as in [Definition 5.10](#), with $L \in S^{\ell_0 \times \ell}$ so that ℓ is the vector length of the vector $\text{bits}(\mathbf{f})$. For the standard bitification matrix $L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d)$, we have $\ell = \ell_0 \cdot d \cdot B$. Let further ℓ_1, ℓ_2 be tensor decomposition parameters (to be used in [Construction 5.4](#)) such that $\ell = \ell_1 \cdot \ell_2$. We assume that $\mathcal{C}$ is a linear code over $\mathbb{F}_{2^\nu}$ of dimension ℓ/ν .

We will need the following result.

Lemma 5.13. Let $\mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$, let $\mathbf{v} \in R^{\ell_0}$, and let $\text{bits}(\mathbf{f})$ be a bitification of $\mathbf{f}$ with respect to L . Then

$$\begin{aligned}\langle \psi(\mathbf{f}), \mathbf{v} \rangle &= \langle \text{bits}(\mathbf{f})_R, \mathbf{u} \rangle \in R \\ \mathbf{u} &:= \psi(L)^\top \cdot \mathbf{v} \in R^\ell,\end{aligned}$$

where $\top$ denotes transposition. In particular, if $L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \in S^{\ell_0 \times (\ell_0 \cdot B \cdot d)} = S^{\ell_0 \times \ell}$ (cf. [Definition 5.10](#)), then

$$\mathbf{u} = (\mathbf{v} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d))) \in R^\ell \quad (31)$$

Proof. Rewriting $\langle \psi(\mathbf{f}), \mathbf{v} \rangle$, we obtain by [Definition 5.10](#) and [Eq. \(18\)](#)

$$\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \langle \psi(L \cdot \text{bits}(\mathbf{f})_S), \mathbf{v} \rangle = \langle \psi(\text{bits}(\mathbf{f})_S), \psi(L)^\top \cdot \mathbf{v} \rangle$$

Now $\psi(\text{bits}(\mathbf{f})_S) = \text{bits}(\mathbf{f})_R$ because every entry of $\text{bits}(\mathbf{f})$ consists of bits and ψ is a ring homomorphism, so it sends the 0 and 1 elements of S to the 0 and 1 elements of R . Further,

$$\psi(L) = \text{Id}_{\ell_0 \times \ell_0} \otimes \psi((1, \dots, 2^{B-1})_S) \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)),$$

and $\psi \circ \pi_{\text{char}(S)}(2^i) = \pi_{\text{char}(R)}(2^i)$ because $\psi \circ \pi_{\text{char}(S)}$ is a ring homomorphism, $\psi \circ \pi_{\text{char}(S)}(2^i) = 2^i \cdot (\psi \circ \pi_{\text{char}(S)}(1))$, hence $\psi \circ \pi_{\text{char}(S)}(2^i)$ is 2^i times the identity element of R , and that is precisely the definition of $\pi_{\text{char}(R)}(2^i)$.

The specific form of $\mathbf{u}$ in (31) is now obtained by direct calculation of $\psi(L)^\top \cdot \mathbf{v}$. $\square$

We assume that the input vector $\mathbf{v}$ for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{<D_S}, \psi, R, \mathcal{C}, L)$, the matrix L , and the tensor decomposition dimensions are such that $\mathbf{u} = \psi(L)^\top \cdot \mathbf{v}$ admits a tensor decomposition $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with $\mathbf{u}^{(1)} \in R^{\ell_1}$, $\mathbf{u}^{(2)} \in R^{\ell_2}$, see [Remark 5.15](#) for an explanation that this can be achieved easily in applications, and what to do otherwise.

Construction 5.14 (IOP for $\mathcal{R}_{\text{LinBitsRings}}$). **Parameters.** $(\ell_0, \ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$ comprised of: a witness vector length ℓ_0 ; a *bitified witness* vector length ℓ ; tensor decomposition dimensions ℓ_1, ℓ_2 for [Construction 5.4](#) satisfying $\ell_1 \cdot \ell_2 = \ell$; IORs and IOPs Π_{iopp} and Π_{red} as in [Construction 5.4](#); and sets of primes and homomorphisms $\mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}}$ again as in [Construction 5.4](#).

Inputs. The prover and verifier receive $\mathbf{x} = (\mu, \mathbf{v}) \in R \times R^{\ell_0}$ and $\mathbf{y} = f \in \mathbb{F}_{2^\nu}^m$. The prover receives $\mathbf{w} = \mathbf{f} \in (S_{\leq B}^{<D_S})^{\ell_0}$ such that $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{<D_S}, \psi, R, \mathcal{C}, L)$, i.e.,

$$f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \quad \langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu \quad \text{in } R.$$

for some $\text{bits}(\mathbf{f})$ such that $L \cdot \text{bits}(\mathbf{f})_S = \mathbf{f}$.

1. By [Lemma 5.13](#), the claim $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{<D_S}, \psi, R, \mathcal{C}, L)$ is equivalent to the claim

$$(\mathbf{x}' = (\mu, \mathbf{u}), \mathbf{y}' = f = \text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu \circ \pi_2(\text{bits}(\mathbf{f})), \mathbf{w}' = \text{bits}(\mathbf{f})) \in \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}). \quad (32)$$

where $\mathbf{u} = \psi(L)^\top \cdot \mathbf{v}$.

By assumption, $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ with $\mathbf{u}^{(1)} \in R^{\ell_1}$, $\mathbf{u}^{(2)} \in R^{\ell_2}$. Then the prover and the verifier use [Construction 5.4](#) to prove (32). They use parameters $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$. See [Remark 5.15](#) for comments on how to obtain tensor decompositions of $\mathbf{u}$ in [Construction 5.4](#) assuming $\mathbf{v}$ is appropriately tensor-decomposable.

Remark 5.15 (Tensor decompositions of $\mathbf{u}$). Recall that [Construction 5.4](#) considers a (possibly trivial) tensor decomposition of its public input vector $\mathbf{u}$ (denoted $\mathbf{v}$ in [Construction 5.4](#)). Moreover, as argued in [Sections 3.3](#) and [6](#), this tensor decomposition is crucial for performance of BitZ (concretely, we seek $\ell^{0.5} \leq \ell_1 \leq \ell^{0.6}$, cf. [Section 6](#)).

In many applications, the input vector $\mathbf{v}$ for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ is an “eq” vector, i.e. a vector of the form $(\text{eq}(\mathbf{b}; \mathbf{r}))_{\mathbf{b} \in \{0,1\}^{\log \ell_0}}$ for some $\mathbf{r} \in R^{\log \ell_0}$. In this case $\mathbf{v} = \mathbf{v}^{(1)} \otimes \mathbf{v}^{(2)}$ with the dimensions of $\mathbf{v}^{(1)}$, $\mathbf{v}^{(2)}$ being arbitrary powers of two as long as $|\mathbf{v}^{(1)}| \cdot |\mathbf{v}^{(2)}| = \ell_0$. In that case, if

$$L = \text{Id}_{\ell_0 \times \ell_0} \otimes (1, \dots, 2^{B-1})_S \otimes (\gamma_1, \dots, \gamma_d) \quad (33)$$

(cf. [Definition 5.10](#)), then $\mathbf{u}$ has the form [\(31\)](#), i.e.

$$\mathbf{u} = (\mathbf{v} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)))$$

and we may use a tensor decomposition for $\mathbf{u}$ in [Construction 5.4](#) of the form, say, $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$, where

$$\mathbf{u}^{(1)} := \mathbf{v}^{(1)}, \quad \mathbf{u}^{(2)} := \mathbf{v}^{(2)} \otimes (1, 2, \dots, 2^{B-1})_R \otimes (\psi(\gamma_1), \dots, \psi(\gamma_d)).$$

Other decompositions apply. For example one of the tensor factors in $\mathbf{u}^{(2)}$ above could be placed in $\mathbf{u}^{(1)}$, and the vector $(1, 2, \dots, 2^{B-1})_R$ can admit further tensor decomposition, and some of the factors can be placed in $\mathbf{u}^{(1)}$.

For more general matrices L , one needs to make sure that L has enough structure to allow for a favorable tensor decomposition of $\mathbf{u}$. If that is not possible, one can either:

- Fall back to using the matrix [\(33\)](#), which is always available for arbitrary finitely generated rings S with bit-bound B , or
- Alternatively, if the ring R supports it (e.g. if R is a finite field or a ring with a large exceptional set), one can run a sumcheck to reduce the claim $\langle \text{bits}(\mathbf{f})_R, \mathbf{u} \rangle = \mu$ to an MLE evaluation claim over R , thereby “replacing” the vector $\mathbf{u}$ by an “eq vector” that, as such, admits tensor decompositions of arbitrary dimensions.

Costs. [Albert: Review. Computed with AI. Justify here and in the previous table why we use \$k = 1\$?](#) We discuss the costs of [Construction 5.14](#). The rounds, oracles and queries of [Construction 5.14](#) are exactly those of [Construction 5.4](#) (see [Table 4](#)), which is run on the bitified witness $\text{bits}(\mathbf{f})$ of length $\ell = \ell_0 \cdot d \cdot B$ (if using the standard bitification matrix of [Definition 5.10](#)), and with the weight vector $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ of [Remark 5.15](#). For $R = \mathbb{Z}_n$ ($k = 0$) or $R = \mathbb{Z}_n[X]/(h(X))$ ($k = 1$) for some $n \geq 0$ and polynomial $h(X)$, the costs of executing [Construction 5.4](#) are the same as those listed in [Table 4](#). The only additional costs of [Construction 5.14](#) are those of computing the bitification of $\mathbf{f}$ and of computing $\mathbf{u}^{(1)}$ and $\mathbf{u}^{(2)}$. Note that, when using the standard bitification matrix, the prover and verifier do not need to compute $\mathbf{u}$ entirely: it suffices to compute $\mathbf{u}^{(1)}, \mathbf{u}^{(2)}$. Hence the verifier’s work outside Π_{red} and Π_{iopp} is $O(\ell_1 + \ell_2)$ operations (cf. [Table 4](#)). In [Table 5](#) we summarize the costs of [Construction 5.14](#).

	Prover	Verifier	Proof (bits)
Bitification of $\mathbf{f}$ (Definition 5.10)	$O(\ell)$ bit op.	–	–
Instance translation $(\mu, \mathbf{v}) \mapsto (\mu, \mathbf{u}^{(1)}, \mathbf{u}^{(2)})$	$O(\ell_2) \cdot R$	same as prover	–
Construction 5.4 with length $\ell = \ell_0 \cdot d \cdot B$	Table 4	Table 4	Table 4

Table 5: Costs of Construction 5.14 for the standard bitification matrix and the tensor decomposition of Remark 5.15. By $N \cdot R$ we indicate N operations in R .

Theorem 5.16. *Under the same hypotheses and notation as in Theorem 5.8 for the parameters $(\ell, \ell_1, \ell_2, \Pi_{\text{iopp}}, \Pi_{\text{red}}, \mathcal{P}, (\mathcal{M}_q)_{q \in \mathcal{P}})$, the ring R , the code $\mathcal{C}$, and the binary field $\mathbb{F}_{2^\nu}$ with packing map Pack_ν , Construction 5.14 is a perfectly complete $(1+t_{\text{red}}+t_{\text{iopp}})$ -round IOP for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ with round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \perp)$ and errors*

$$\kappa_0, (\kappa_{\text{red}i})_{i \in [t_{\text{red}}]}, (\kappa_{\text{iopp}i})_{i \in [t_{\text{iopp}}]},$$

where $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$. (see Theorem 5.8; here $\theta(\mathcal{P}, \mathcal{M}, \ell)$ is computed with respect to the tensor decomposition $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ of Remark 5.15 used in Construction 5.4, i.e. the quantities D , T , N_R are determined by the lifts of the entries of $\mathbf{u}^{(1)}$ and $\mathbf{u}^{(2)}$).

The error term κ_0 and its round are absent if Round 1 of Construction 5.4 is not executed. If Round 0 of Construction 5.4 is executed, the same holds under the hypotheses of Theorem A.1, with the additional round and the additional error term κ_{OOD} of that theorem.

Proof. Deferred to Section A.1. In short, the result is a consequence of Lemma 5.13, which establishes an equivalence between triples for $\mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L)$ and $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C})$. Lemma 5.13 further provides a method for efficiently computing one triple from the other. This allows us to build a round-by-round knowledge soundness state and an extractor for Construction 5.14 from those of Construction 5.4. $\square$

5.7 Instantiation

We instantiate Constructions 5.4 and 5.14 by letting Π_{red} be an optimized GKR protocol for multiple grand product claims of certain form (cf. Section 6). The IOP Π_{iopp} is taken to be WHIR/Ligerito [ACFY25, NA25] with the ring switching step from [DP23]. Our implementation of WHIR is taken from Flock’s [BRW26] implementation. The inner product claims when Π_{iopp} is called are not MLE evaluation claims. Since the ring-switch implementation in [BRW26] does not support arbitrary inner product claims, we run a sumcheck to reduce to those to MLE evaluation claims. Alternatively, one could avoid the sumcheck and use the approach from Section B. We instantiate them solely with the following parameters:

Parameter	Choices
R	A finite field (possibly non-prime)
S	Integers $\mathbb{Z}$
π	Reduction mod $\text{char}(R)$
ψ	Reduction mod $\text{char}(R)$
ν	128
$\mathcal{C}$	Reed-Solomon code over $\mathbb{F}_{2^\nu}$ of rate 1/8
Proximity parameter δ	Close to the Johnson bound
L	The standard bitification matrix (Eq. (33))
$\mathcal{P}$	Set of primes. Discussed below
$\mathcal{M}_q = \{\pi_q\}$	Reduction modulo q
ℓ_1, ℓ_2	Discussed below

Table 6: Parameter choices in our deployment of [Constructions 5.4](#) and [5.14](#)

We next provide example instantiations of [Constructions 5.4](#) and [5.14](#) so as to achieve $\lambda = 128$ bits of security, while keeping the protocols as efficient as possible, with the main challenge being to use $\mathbb{F}_{2^\nu}$ of size 2^{128} for efficiency purpose. We remark that in [Section 7](#) we provide experimental results both for 128 and 100 bits of security. Achieving $\lambda = 100$ bits of security is relatively straightforward and we do not discuss it. We use a careful selection of parameters in combination with grinding. In [Remark 5.17](#) we discuss other instantiations and techniques that do not require grinding.

The contents of this section should be taken as examples of how to instantiate [Constructions 5.4](#) and [5.14](#), not as strict rules. Other configurations may be perfectly plausible and practical.

In terms of security, we need to ensure the following:

1. The round-by-round knowledge soundness errors of [Construction 5.4](#) κ_{OOD} (if Round 0 is executed), κ_0 (if Round 1 is executed), $(\kappa_{\text{red}i})_i$ and $(\kappa_{\text{iopp}_i})_i$ are all at most $2^{-\lambda}$.

In terms of efficiency, the most important step of [Construction 5.4](#) to be aware of is Phase 3, where one uses an IOR Π_{red} to reduce the claim

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2], \quad (34)$$

into a linear claim on $\pi(\mathbf{f})$. As mentioned, in our instantiation, Π_{red} is the GKR protocol for multiple grand product claims, equipped with the optimizations discussed in [Section 6](#). This is the most expensive prover part of [Constructions 5.4](#) and [5.14](#), constituting between 45% ($\ell = 2^{20}$) and 87% ($\ell = 2^{30}$) of prover costs, cf. [Table 11](#). Because of this, it is important for efficiency that operating in $\mathbb{F}_{2^\nu}$ is as efficient as possible.

With this in mind, we target using $\mathbb{F}_{2^\nu}$ with $\nu \approx 128$. A natural candidate is the 2^{128} sized binary field in GHASH representation

$$\mathbb{F}_{2^\nu} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1).$$

Before anything else, we discuss the constraint $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$. First of all, we set ℓ_1, ℓ_2 with $\ell_1 \cdot \ell_2 = \ell$ and

$$\ell_1 \in [\ell^{0.5}, \ell^{0.6}]. \quad (35)$$

In general, choosing $\ell_1 \approx \ell^{0.6}$ can lead to better prover time and proof size for the GKR grand product step, as observed in our experiments. However choosing $\ell_1 \approx \ell^{0.5}$ allows to use slightly smaller soundness errors, which reduces grinding times if targeting 128 bits of security.

We now discuss how to make the errors $\kappa_{\text{OOD}}, \kappa_0, (\kappa_{\text{red}_i})_i, (\kappa_{\text{iopp}_i})_i$ be all at most $2^{-\lambda}$.

The errors $\kappa_{i,\text{iopp}}$. The ring switching step adds an error term of the form $7/|\mathbb{F}_{2^\nu}|$. If targeting 128 bits of security, we use grinding (Section 4.3) of 3 bits in this round. The sumcheck before the ring switch adds similarly small error terms, which are handled with a few bits of grinding. WHIR is configured at any desired $\lambda \leq 128$ bits of security using $|\mathbb{F}_{2^\nu}| = 2^{128}$ following [?, ?]. We do not discuss this further since WHIR is a well-known IOP, and it can achieve 128 bits of security (in the Unique Decoding Radius) in 128 bit fields when codewords have dimension approximately 2^{30} assuming an interleaved RS code (to reduce the correlated agreement error) and the use of grinding when needed.

The error κ_{OOD} . As mentioned, we instantiate Π_{iopp} as WHIR with the ring-switching techniques of [DP23] and Section B. For $\lambda = 100$ we use a proximity parameter δ up to the Johnson bound. For $\lambda = 128$ we set δ up to the unique decoding radius, so that the Out Of Domain sampling step (Round 0) is skipped.

The errors $\kappa_{i,\text{red}}$. As mentioned, our Π_{red} IOR is the GKR protocol for multiple grand product claims from Eq. (34), see Section 6. The first round of Section 6 has soundness error $\log \ell_2/|\mathbb{F}_{2^\nu}|$. The rest of rounds are either a round of sumcheck for a polynomial of degree 3, or a unification of two evaluation claims of the same multilinear polynomial. Accordingly we have $\kappa_{\text{red}_i} \leq 3/|\mathbb{F}_{2^\nu}|$ for all i , except $\kappa_{\text{red}_1} \leq \log \ell_2/|\mathbb{F}_{2^\nu}|$. This immediately guarantees $\lambda = 100$ bits of security. To achieve $\lambda = 128$ bits of security, one can do a small number of bits of grinding (Section 4.3) per round.

The error κ_0 . The error κ_0 is perhaps the most delicate, as it demands at the same time that $|\mathcal{P}|$ is large, while still satisfying $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$.

Recall that in many scenarios κ_0 does not exist in the IOP (i.e. sometimes our IOP skips Round 1). Concretely, if our initial inner product claim is over $R = \mathbb{F}_q$ a prime field of size $< (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, then κ_0 does not appear as an error term, and we do not need to do anything to adjust for it. The term κ_0 does appear in all other scenarios ($R = \mathbb{Z}$ with large or negative entries in $\mathbf{v}^{(1)}$ [Albert: negative?](#) (cf. [Remark 5.5](#)), or R a prime field of larger size, or R a proper field extension). We divide our approaches by whether this first step is skipped or not.

For the purposes of this discussion, we assume $\mathbb{BitZ}$ is used within a bigger proof system, as described in Section 3.1, so that $\mathbb{BitZ}$ is used to prove an inner product claim for a committed witness, as the last phase of a bigger proving system where [Albert: review what the technical overview exactly says](#) 1) first the initial constraint is projected to a finite field $\mathbb{F}$, or, alternatively, this step is skipped if the constraint is already over a finite field; and 2) a PIOP is run over $\mathbb{F}$ for the projected constraint, outputting an inner product claim on the projected witness. Now, this inner product claim is the input of [Construction 5.4](#), and we aim to configure [Construction 5.4](#) at 128 bits of security for this setting.

Strategy 1: the PIOP achieves 128 bits of security through grinding and we avoid the error term κ_0 . If allowing, say, 18 – 19 bits of grinding during each round of the PIOP (which adds 1.8 – 3.5ms per round, cf. [Table 2](#)), then, say, a Spartan-like PIOP will achieve 128 bits of security if run on a field $\mathbb{F}$ of size $\approx 2^{112}$ to 2^{113} (since each one of its rounds is either a sumcheck round for a degree ≤ 3 polynomial, or a step that batches some claims with error $1/|\mathbb{F}|$). Hence, if the initial constraint is over $\mathbb{Z}$ or $\mathbb{Z}[X]$ [Albert: do we mention \$\mathbb{Z}\[X\]\$? Why not mention other rings here? Again, wait for the TO](#), then we may project it to a 112-113 bit random prime field $\mathbb{F}$ and run a PIOP over $\mathbb{F}$. We study this case next. Our analysis also applies if the original constraint was already expressed over a prime field of such size.

Note that the projection of the constraint to $\mathbb{F}$ also requires to be set up to 128 bits of security. We discuss this in [Remark 5.18](#).

Let $q = |\mathbb{F}|$. Then if

$$q < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1,$$

then κ_0 is not present as an error term, and there is nothing to adjust. Since $|\mathbb{F}_{2^\nu}| = 2^{128}$ and $q \approx 2^{113}, 2^{112}$, this occurs as long as $\ell_1 \lesssim 2^{15}, 2^{16}$, and using (35), as long as

$$\ell \lesssim 2^{25}, 2^{26} \quad \text{if } \ell_1 \approx \ell^{0.6}$$

and

$$\ell \lesssim 2^{30}, 2^{32} \quad \text{if } \ell_1 \approx \ell^{0.5}.$$

The specific upper bounds depend on how much grinding one is doing at the PIOP level.

Strategy 2: No or essentially no grinding during the PIOP, and κ_0 is present and grinding is used during the first projection step (Round 1) of [Construction 5.4](#). We now assume $\mathbb{F}$ is either a prime field with $|\mathbb{F}| \geq (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$, or a proper extension field. In this case [Construction 5.4](#) goes through an initial projection step with soundness error κ_0 , i.e. Round 1 of [Construction 5.4](#).

Recall ([Theorem 5.8](#)) that $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$. We use $\pi_{\text{canon}} : \mathbb{Z}[X] \rightarrow \mathbb{F}$ as in [Example 5.2](#), so that $\pi_{\text{canon}}(h(X)) = \pi_{\text{char}(\mathbb{F})}(h)(x)$ where $\pi_{\text{char}(\mathbb{F})}(h)$ is the polynomial h after reducing its coefficients modulo $\text{char}(F)$, and x is a root of an irreducible polynomial defining $\mathbb{F}_{q^d}$ (if $d = 1$ and so $\mathbb{F}_{q^d}$ is a prime field, then we can use a morphism $\pi_{\text{canon}} : \mathbb{Z} \rightarrow \mathbb{F}_q$). We have $D = d - 1$, $T \leq d$, and $N_{\mathbb{F}} \leq q - 1$ (see (21) for a definition of these quantities). We instantiate the morphism sets $\mathcal{M}_q$ as in [Lemma 5.3](#), i.e. π_{rand} reduces coefficients modulo a sampled prime and evaluates X_1 at a random $\alpha \in \mathbb{F}_q$. In this case, by [Lemma 5.3](#),

$$\kappa_0 \leq \frac{2 \cdot (d - 1)}{q_{\min}} + \frac{\log(4 \cdot d \cdot \ell \cdot (q - 1)^2)}{|\mathcal{P}| \cdot \lfloor \log(q_{\min}) \rfloor},$$

where q is the characteristic of $\mathbb{F}$.

We select $\mathcal{P}$ to be the set of all primes in a range $[2^a, 2^b]$. There are approximately

$$|\mathcal{P}| \approx (2^b/b - 2^a/a) / \log_e(2)$$

such primes. Using $b = 128 - \tau \cdot \log(\ell)$, $\ell_1 = 2^{\tau \cdot \log(\ell)}$, for a parameter $\tau \in [0.5 .. 0.6]$ and plugging these values into κ_0 we obtain

$$\begin{aligned} \kappa_0 &\lesssim \frac{2 \cdot (d - 1)}{2^a} + \frac{\log(4 \cdot d \cdot \ell \cdot (q - 1)^2) \cdot \log_e(2)}{\frac{a}{b} \cdot 2^b - 2^a}, \\ b &:= 128 - r_1, \quad r_1 := \tau \cdot \log(\ell). \end{aligned} \tag{36}$$

We calculate several minima for the right hand side of (36) when $\mathbb{F}$ is a field of size $\approx 2^{128}$ (not necessarily a prime field), for varying values of ℓ and $\tau \in \{0.5, 0.6\}$. We include the corresponding grinding times in order to achieve 128 bits of security for this initial step of [Construction 5.4](#). [Albert: Review grinding times.](#)

$\log \mathbb{F} $	d	$\log q$	$\log \ell$	τ	a^*	$\kappa_0 \leq$	Bit-deficit to 128	Grinding time (ms)
128	1	128	24	0.5	110	$2^{-108.3}$	20	8.9
128	1	128	24	0.6	107	$2^{-105.9}$	23	84.6
128	1	128	28	0.5	108	$2^{-106.3}$	22	35.5
128	1	128	28	0.6	105	$2^{-103.5}$	25	240.8
128	1	128	30	0.5	107	$2^{-105.3}$	23	84.6
128	1	128	32	0.5	106	$2^{-104.2}$	24	148.6
128	4	32	24	0.5	114	$2^{-109.2}$	19	3.5
128	4	32	24	0.6	111	$2^{-106.8}$	22	35.5
128	4	32	28	0.5	112	$2^{-107.1}$	21	14.3
128	4	32	28	0.6	109	$2^{-104.4}$	24	148.6
128	4	32	30	0.5	111	$2^{-106.1}$	22	35.5
128	4	32	30	0.6	108	$2^{-103.1}$	25	240.8
128	4	32	32	0.5	110	$2^{-105.1}$	23	84.6

Table 7: Minima for the upper bound of κ_0 in (36) for different configurations of $\mathbb{F}$ (of characteristic q and degree d over its prime field), witness sizes ℓ , and τ such that $\ell_1 = \ell^\tau$. By choosing primes in the interval $[2^{a^*}, 2^{128-r_1}]$, we have that the first error term κ_0 is bounded by the quantity in the $\kappa_0 \leq$ column. We include the bits left required to achieve $\kappa_0 < 1/2^{128}$ and the cost of grinding such amount of bits in a MacBook Air M4 16GB, multi-threaded, using Blake3 as hash function.

Remark 5.17 (128 bits of security for larger witness sizes ℓ , without or with minimal grinding). Grinding costs scale exponentially in the number of grinding bits. Because of this, we cannot resort to grinding for very large witness sizes ℓ . There are at least two ways to achieve 128 bits of security (or more), in this case.

Method 1: Use a larger binary field $\mathbb{F}_{2^\nu}$, i.e. simply take $\mathbb{F}_{2^\nu}$ with $|\mathbb{F}_{2^\nu}|$ comfortably larger, rather than $|\mathbb{F}_{2^\nu}| = 2^{128}$.

Method 2: We generalize an idea from [Bin26]. In this setting we make the assumption that the entries in γ (cf. Phase 3 of [Construction 5.4](#)) are smaller than $|\mathbb{F}_{2^\nu}| - 1$. This means we require $\max \mathcal{P} < |\mathbb{F}_{2^\nu}| - 1$ instead of $\max \mathcal{P} < (|\mathbb{F}_{2^\nu}| - 1)/\ell_1$ (and similarly for the conditions that would allow to skip Round 1 in [Construction 5.4](#)). Hence now the bound on the primes in $\mathcal{P}$ does not depend on the witness size.

Note that when we attempt to prove the integer equalities $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ ($j \in [\ell_2]$) “in the exponent” during Phase 3 of [Construction 5.4](#), what we are really doing is proving the equalities modulo $|\mathbb{F}_{2^\nu}| - 1$. So far, we have used $\mathbb{F}_{2^\nu}$ large enough that this distinction does not matter, in that the total possible magnitude of $\langle \gamma, \mathbf{f}_{*j} \rangle$ does not exceed $|\mathbb{F}_{2^\nu}| - 2$.

If that is not the case, one can, in addition to proving $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ modulo $|\mathbb{F}_{2^\nu}| - 1$, prove the same equality modulo ℓ_1 . This works because, since ℓ_1 and $|\mathbb{F}_{2^\nu}| - 1$ are coprime (because ℓ_1 is a power of two), by the CRT, if both equalities are true, then $\langle \gamma, \mathbf{f}_{*j} \rangle - \mu_j$ is an integer multiple of

$\ell_1 \cdot (|\mathbb{F}_{2^\nu}| - 1)$. However, because the entries of $\mathbf{f}$ are bits and the entries of γ are at most $|\mathbb{F}_{2^\nu}| - 2$, this means that $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ as integers.

To prove that $\langle \gamma, \mathbf{f}_{*j} \rangle = \mu_j$ modulo ℓ_1 , one can prove the integer equality $\langle \gamma', \mathbf{f}_{*j} \rangle = \mu'_j + a_j \cdot \ell_1$ for some $a_j \in \mathbb{Z}$ (provided by the prover), where γ' contains the entries of γ reduced modulo ℓ_1 , and μ'_j is μ_j reduced modulo ℓ_1 . The verifier checks that a_j is not too large, namely it checks that $|a_j| \cdot \ell_1 \leq (\ell_1 - 1) \cdot \ell_1$, which is the maximum value $|a_j| \cdot \ell_1$ can achieve when $\mathbf{f}$ is filled with bits (which is guaranteed by our commitment via WHIR). Hence the value $|\langle \gamma', \mathbf{f}_{*j} \rangle - \mu'_j - a_j \cdot \ell_1|$ can be at most $2 \cdot (\ell_1 + 1) \cdot \ell_1$. As long as $2 \cdot (\ell_1 + 1) \cdot \ell_1 < |\mathbb{F}_{2^\nu}| - 1$, this equality can be proved “in the exponent” without causing overflow.

Note that if γ admits a tensor decomposition $\gamma = \gamma^{(1)} \otimes \gamma^{(2)}$, then we can take, instead of γ' , the tensor-product $\gamma^{(1)'} \otimes \gamma^{(2)'}$, where $\gamma^{(i)'}$ denotes $\gamma^{(i)}$ reduced modulo ℓ_1 . This allows to use all of our “low-entropy” grand product optimizations from [Section 6](#), though in this case the required bound becomes $2 \cdot (\ell_1^2 + 1) \cdot \ell_1 < |\mathbb{F}_{2^\nu}| - 1$.

Overall, this method allows to use $\max \mathcal{P} < |\mathbb{F}_{2^\nu}| - 1$, meaning we can choose primes that are $\log(\ell_1)$ bits larger than we otherwise would. This makes soundness errors be smaller by this same amount of bits, which leads to small grinding requirements. The cost is an extra batch of low entropy grand products.

Remark 5.18 (Projecting polynomial constraints to a 112-113 bit prime field with 128 bits of security). The error term of the projection step (also called fingerprinting) in [\[AGK⁺26, GWV⁺25, CHA24\]](#) is (for integer constraints) bounded by (cf. [Lemma 5.9](#) with $d = 0$)

$$\varepsilon_{\text{PIOP-proj}} \leq \frac{\log(N)}{|\mathcal{P}'| \cdot \lfloor \log(q_{\min}) \rfloor}$$

where $\mathcal{P}'$ is a set of primes; $q_{\min}$ is the smallest prime in $\mathcal{P}'$; N is the largest absolute value that an unsatisfying constraint can attain. More precisely, assume integer witness entries $\mathbf{w}$ are guaranteed to have at most certain bit-size B' , and assume the constraints have the form $q_i(\mathbf{x}, \mathbf{w}) = 0$ for all i , for $\mathbf{x}$ a public input vector and q_i constraint polynomials. Then N is the maximum possible value of the form $|q_i(\mathbf{x}, \mathbf{w})|$ among all i , all possible inputs $\mathbf{x}$ and all $\mathbf{w}$ with entries having bit-size at most B' .

Using a similar analysis as in Strategy 2 above, where we let $\mathcal{P}'$ be the set of primes between the interval $[2^{112}, 2^{113}]$, we obtain that, as long as $\log(N) < 1.4 \cdot 2^6 \approx 90$ we have $\varepsilon_{\text{PIOP-proj}} < 2^{-106}$. In that case we could attain 128 bits of security during this projection step by performing 22 bits of grinding (35ms in a MacBook Air M4).

6 A GKR protocol for low entropy batched grand products

The bulk of the work in $\mathbb{Bit}\mathbb{Z}$ ([Construction 5.4](#)) is to reduce the claim over the binary field $\mathbb{F}_{2^\nu}$

$$\prod_{i \in [\ell_1]} ((g^{\gamma_i} - 1) \cdot \pi_2(\mathbf{f}_{ij}) + 1) = g^{\mu_j} \quad \in \mathbb{F}_{2^\nu}, \quad \text{for all } j \in [\ell_2] \quad (37)$$

to an inner product claim of the form $\langle \mathbf{v}, \pi_2(\mathbf{f}) \rangle = \mu$ over $\mathbb{F}_{2^\nu}$, for some $\mathbf{v} \in \mathbb{F}_{2^\nu}^\ell$, where here $\mathbf{f} \in \{0, 1\}^\ell \subseteq \mathbb{Z}^\ell$. Recall that

$$\ell = \ell_1 \cdot \ell_2.$$

To fix intuition, in our instantiation (cf. [Section 5.7](#)) we set $\ell_1 \approx \ell^{0.6}, \ell_2 = \ell^{0.4}$.

Cryptographically speaking, our protocol is simply a standard batched version of the GKR protocol applied to grand products from [?]. However, our implementation uses lookup tables (i.e. preprocessed data) to optimize the prover time of such protocol. In particular, our techniques leverage the fact that the factors in (37) have *low entropy*, in the sense that for every $i \in [\ell_1]$ and $j \in [\ell_2]$, the i -th factor in the j -th grand product of (37) can take only two values, depending on whether $\mathbf{f}_{ij} = 1$ or $\mathbf{f}_{ij} = 0$.

The optimizations we introduce apply to any (batched) grand product with low entropy factors, and not only to the grand product in (37). They also work on any field. Because of this, we describe a general protocol IOR that, given a finite field $\mathbb{K}$; a bivariate polynomial $p \in \mathbb{K}[Y, X]$; a small subset $S \subseteq \mathbb{K}$; and dimensions $\ell = \ell_1 \cdot \ell_2$, reduces triples from

$$\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} = (\mathbf{f}_{*j})_{j \in [\ell_2]} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \prod_{i \in [\ell_1]} p(\mathbf{y}_i, \mathbf{f}_{ij}) = \boldsymbol{\mu}_j \text{ for all } j \in [\ell_2] \right. \right\},$$

to triples in

$$\mathcal{R}_{\text{Lin}}(\mathbb{K}) = \left\{ \left(\begin{array}{l} \mathbf{x} = (\mathbf{u}, \mu) \in \mathbb{K}^\ell \times \mathbb{K}, \\ \mathbf{y} = \perp, \\ \mathbf{w} = \mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2} \end{array} \right) \left| \langle \mathbf{u}, \mathbf{f} \rangle = \mu \right. \right\}.$$

One can additionally add an encoding $f = \text{Enc}_C(\mathbf{f})$ as an oracle in the above relations (matching the context in which we use the IOR), but our IOR does not make any use of it whatsoever, hence we omit it.

In our instantiation, $\mathbf{y}_i = g^{\gamma_i}$ and $\boldsymbol{\mu}_j = g^{\mu_j}$, where γ_i and μ_j are the integers in (37).

We describe Π_{red} now. We will describe our implementation-level optimizations afterwards. As we mentioned, cryptographically/protocol-wise, our protocol is a standard batched GKR for grand products [Tha22]. We assume ℓ_1, ℓ_2 are powers of two:

$$\ell_1 = 2^{r_1}, \quad \ell_2 = 2^{r_2}.$$

We index $[\ell_1], [\ell_2]$ via elements of the hypercubes $\{0, 1\}^{r_1}, \{0, 1\}^{r_2}$, respectively.

Construction 6.1 (An IOR Π_{red} from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$). **Input.** The prover and verifier receive $(\mathbf{y}, \boldsymbol{\mu}) \in \mathbb{K}^{\ell_1} \times \mathbb{K}^{\ell_2}$. The prover receives also $\mathbf{f} \in S^{\ell_1 \times \ell_2} \subseteq \mathbb{K}^{\ell_1 \times \ell_2}$.

Let L_{r_1} be a *multilinear* polynomial over $\mathbb{K}$ on $r_1 + r_2$ variables such that

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}.$$

(note that p may not be multilinear). For $i = 0, \dots, r_1 - 1$, let L_i be a multilinear polynomial on $r_2 + i$ variables such that

$$L_i(\mathbf{b}, \mathbf{c}) = L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^i \times \{0, 1\}^{r_2}.$$

1. The verifier sends $\boldsymbol{\alpha}^{(0)} \in \mathbb{K}^{r_2}$. Both prover and verifier compute $C^{(0)} := L_0(\boldsymbol{\alpha}^{(0)})$ (note that L_0 is the multilinear extension of $\boldsymbol{\mu}$ over $\mathbb{K}$).

2. For $i = 0, \dots, r_1 - 1$:

(a) **i -th sumcheck** The prover and verifier run the sumcheck protocol for the following claim:

$$C^{(i)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^i \times \{0,1\}^{r_2}} L_{i+1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{i+1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)}).$$

At the end of the sumcheck protocol, the prover and verifier are left with a claim of the form

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 0, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_0^{(i)}, \quad L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, 1, \mathbf{r}_{\mathbf{c}}^{(i)}) = a_1^{(i)} \quad (38)$$

for some $\mathbf{r}_{\mathbf{b}}^{(i)} \in \mathbb{K}^i$, $\mathbf{r}_{\mathbf{c}}^{(i)} \in \mathbb{K}^{r_2}$ and $a_0^{(i)}, a_1^{(i)} \in \mathbb{K}$.

(b) **i -th unification step** The prover and verifier unify the two claims from (38) into a single one via the following standard procedure:

- i. The verifier sends a random element $\beta^{(i)} \in \mathbb{K}$.
- ii. The claim (38) is replaced by

$$L_{i+1}(\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}) = C^{(i+1)} := a_0^{(i)} \cdot (1 - \beta^{(i)}) + a_1^{(i)} \cdot \beta^{(i)}.$$

Let

$$\boldsymbol{\alpha}^{(i+1)} := (\mathbf{r}_{\mathbf{b}}^{(i)}, \beta^{(i)}, \mathbf{r}_{\mathbf{c}}^{(i)}).$$

3. At this stage, the prover and verifier are left with the claim

$$L_{r_1}(\boldsymbol{\alpha}^{(r_1)}) = C^{(r_1)}.$$

This claim is equivalent to

$$\sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1} \times \{0,1\}^{r_2}} p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) \cdot \text{eq}((\mathbf{b}, \mathbf{c}), \boldsymbol{\alpha}^{(r_1)}) = C^{(r_1)}, \quad (39)$$

If $p(Y, X)$ is linear on X , then (39) is a linear claim on $\mathbf{f}$. Namely, it is equivalent to

$$\langle \mathbf{u}, \mathbf{f} \rangle = C^{(r_1)} + c$$

for some $\mathbf{u} \in \mathbb{K}^\ell, c \in \mathbb{K}$. In this case, prover and verifier output

$$(\mathbf{x}' = (\mathbf{u}, C^{(r_1)} + c), \perp, \mathbf{w}' = \mathbf{f}) \in \mathcal{R}_{\text{Lin}}(\mathbb{K}).$$

Otherwise, i.e. if p is not linear in X , prover and verifier run a sumcheck for the claim (39), which results in MLE claims for $\mathbf{y}$ and $\mathbf{f}$. The verifier checks the claim for $\mathbf{y}$ itself, and prover and verifier output the MLE claim for $\mathbf{f}$.

Below we discuss what p , $\mathbf{u}$ and $C^{(r_1)} + c$ are specifically in our usage of Π_{red} as a subprotocol of [Constructions 5.4](#) and [5.14](#), and how to keep the verifier succinct.

Using Π_{red} in $\mathbb{B}\text{it}\mathbb{Z}$ and succinctness of the verifier. If $p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b},\mathbf{c}}) = (\mathbf{y}_{\mathbf{b}} - 1) \cdot \mathbf{f}_{\mathbf{b},\mathbf{c}} + 1$ as in [Constructions 5.4](#) and [5.14](#), then

$$\mathbf{u} = ((\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1,1)}))_{\mathbf{b} \in \{0,1\}^{r_1}} \otimes (\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(r_1,2)}))_{\mathbf{c} \in \{0,1\}^{r_2}}, \quad C^{(r_1)} + c = C^{(r_1)} - 1,$$

where $\boldsymbol{\alpha}^{(r_1)} = (\boldsymbol{\alpha}^{(r_1,1)}, \boldsymbol{\alpha}^{(r_1,2)})$. In this case, to avoid having the verifier pay a linear cost, the verifier constructs $\mathbf{u}^{(1)} := ((\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1,1)}))_{\mathbf{b} \in \{0,1\}^{r_1}}$ and $\mathbf{u}^{(2)} := (\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(r_1,2)}))_{\mathbf{c} \in \{0,1\}^{r_2}}$ separately, and outputs these two vectors instead of $\mathbf{u}$.

When used as a subprotocol of [Constructions 5.4](#) and [5.14](#) (concretely, in Phase 3 of [Construction 5.4](#)), after executing the GKR protocol above ([Construction 6.1](#)), we use an IOP Π_{iopp} to prove the linear claim $\langle \mathbf{u}, \pi_2(\mathbf{f}) \rangle = C^{(r_1)} - 1$ (if using [Construction 5.14](#), $\mathbf{f}$ is replaced with $\text{bits}(\mathbf{f})$), with the verifier having oracle access to $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_{\nu} \circ \pi_2(\mathbf{f})$. As we mentioned ([Section 5.7](#)), Π_{iopp} first runs a sumcheck [Albert: Remark somewhere that this sumcheck is avoidable](#), reducing the claim $\langle \mathbf{u}, \pi_2(\mathbf{f}) \rangle = C^{(r_1)} - 1$ to an MLE evaluation claim of the vector $\pi_2(\mathbf{f})$ and of $\mathbf{u}$. For the latter, we have, letting $\boldsymbol{\theta} = (\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}) \in \mathbb{K}^{r_1+r_2}$ be the evaluation point, [Albert: check](#)

$$\begin{aligned} \widetilde{\mathbf{u}}(\boldsymbol{\theta}) &= \widetilde{\mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(1)}, \boldsymbol{\theta}^{(2)}) = \widetilde{\mathbf{u}^{(1)}}(\boldsymbol{\theta}^{(1)}) \cdot \widetilde{\mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(2)}), \\ \widetilde{\mathbf{u}^{(1)}}(\boldsymbol{\theta}^{(1)}) &= \sum_{\mathbf{b} \in \{0,1\}^{r_1}} (\mathbf{y}_{\mathbf{b}} - 1) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\alpha}^{(r_1,1)}) \cdot \text{eq}(\mathbf{b}; \boldsymbol{\theta}^{(1)}), \\ \widetilde{\mathbf{u}^{(2)}}(\boldsymbol{\theta}^{(2)}) &= \text{eq}(\boldsymbol{\theta}^{(2)}; \boldsymbol{\alpha}^{(r_1,2)}), \end{aligned}$$

and so the verifier can compute $\widetilde{\mathbf{u}}(\boldsymbol{\theta})$ by itself in time $O(\ell_1 + \ell_2)$. For the MLE claim on $\pi_2(\mathbf{f})$, we first use ring-switching [\[DP23\]](#) ([Section B](#)) to reduce the claim to a batch of linear claims for $\text{Pack}_{\nu} \circ \pi_2(\mathbf{f})$, and then prove these with WHIR/Ligerito.

Costs. In [Table 8](#) we summarize the number of rounds, the proof size and the verifier work of [Construction 6.1](#) when used within $\mathbb{B}\text{it}\mathbb{Z}$ as described above. These are standard costs corresponding to a standard GKR construction. We discuss an optimized prover, and its costs, in the next subsection. Recall that in our instantiations $\ell_1 \approx \sqrt{\ell}$ and $\ell_2 \approx \sqrt{\ell}$, hence the verifier is succinct.

	Rounds	Proof (elements of $\mathbb{K}$)	Verifier
Step 1 (batching)	1	–	$O(\ell_2) \cdot \mathbb{K}$
Step 2, i -th sumcheck	$i + r_2$	$3 \cdot (i + r_2)$	$O(i + r_2) \cdot \mathbb{K}$
Step 2, i -th unification	1	2	$O(1) \cdot \mathbb{K}$
Computing $\mathbf{u}$	–	–	$O(\ell_1 + \ell_2) \cdot \mathbb{K}$ (only $\mathbf{u}^{(1)}, \mathbf{u}^{(2)}$)
Total	$t_{\text{red}} = 1 + r_1 \cdot r_2 + \binom{r_1+1}{2}$	$3 \cdot t_{\text{red}} - r_1 - 3$	$O(\ell_1 + \ell_2 + \log^2 \ell) \cdot \mathbb{K}$

Table 8: Rounds, proof size and verifier work of [Construction 6.1](#) for the polynomial $p(Y, X) = (Y - 1) \cdot X + 1$ in our deployment of $\mathbb{B}\text{it}\mathbb{Z}$. By $N \cdot \mathbb{K}$ we indicate N operations in $\mathbb{K}$. The proof size counts 3 elements per sumcheck round. Using [\[Gru24\]](#) we send two 2 elements per sumcheck round in our implementation.

The next lemma records the soundness errors of [Construction 6.1](#). We omit its proof since [Construction 6.1](#) is a well-known variation of the GKR protocol, see [\[Tha22\]](#).

Lemma 6.2 (Soundness and completeness of [Construction 6.1](#)). *Assume $p(Y, X)$ is linear in X^6 . Then [Construction 6.1](#) is a perfectly complete $(1 + r_1 r_2 + r_1(r_1 + 1)/2)$ -round IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$*

⁶We focus on this case since it is the one in our instantiation. A generalization to arbitrary polynomials $p(Y, X)$ would add rounds and error terms corresponding to the sumcheck protocol mentioned at the end of [Construction 6.1](#).

to $\mathcal{R}_{\text{Lin}}(\mathbb{K})$, whose verifier makes no oracle queries and whose prover outputs the witness $\mathbf{f}$ it received. Accordingly it also extends to an IOR from $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p, \mathcal{C}, \text{Pack}_\nu)$ to $\mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}_\nu)$ in a natural way (for $S = \{0, 1\}$; these are the same relations as $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$ and $\mathcal{R}_{\text{Lin}}(\mathbb{K})$, but with an additional oracle $\text{Enc}_{\mathcal{C}} \circ \text{Pack}_\nu(\mathbf{f})$). For every δ within the unique decoding radius of $\mathcal{C}$, this latter IOR has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p, \mathcal{C}, \text{Pack}_\nu)^\delta, \mathcal{R}_{\text{Lin}}(\mathbb{K}, \mathcal{C}, \text{Pack}_\nu)^\delta)$ with errors $r_2/|\mathbb{K}|$ in the first round, $3/|\mathbb{K}|$ in every sumcheck round and $1/|\mathbb{K}|$ in every unification step.

6.1 Prover optimization via lookup tables

Albert: Mention somewhere that the optimization has something to do with the method of four russians? We optimize [Construction 6.1](#) as follows. For simplicity we use the subset

$$S = \{0, 1\} \subseteq \mathbb{K},$$

but any small subset of $\mathbb{K}$ is amenable to the techniques discussed here.

Setup We start with the following observation: for every $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}$, we have $\mathbf{f}_{\mathbf{b}, \mathbf{c}} \in S = \{0, 1\}$, and so

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = p(\mathbf{y}_{\mathbf{b}}, \mathbf{f}_{\mathbf{b}, \mathbf{c}}) = \begin{cases} \mathbf{y}_{\mathbf{b}}^{(1)} := p(\mathbf{y}_{\mathbf{b}}, 1) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(0)} := p(\mathbf{y}_{\mathbf{b}}, 0) & \text{if } \mathbf{f}_{\mathbf{b}, \mathbf{c}} = 0. \end{cases}$$

The values $\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}$ do not depend on $\mathbf{c}$. Hence the prover can set up a lookup table $T_{r_1} \in (\mathbb{K} \times \mathbb{K})^{\{0, 1\}^{r_1}}$ defined as

$$T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}) \quad \text{for all } \mathbf{b} \in \{0, 1\}^{r_1}$$

at the beginning of the protocol, and, whenever required, use it to obtain the values $L_{r_1}(\mathbf{b}, \mathbf{c})$. Concretely

$$L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b}, \mathbf{c}}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1} \times \{0, 1\}^{r_2}$$

In our instantiation, $\mathbf{y}_{\mathbf{b}}^{(1)} = g^{\gamma_{\mathbf{b}}}$ and $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ (cf. [\(37\)](#)). Hence additionally the prover precomputes the elements $g^{\gamma_{\mathbf{b}}}$ at the beginning of the protocol, before computing T_{r_1} .

As we will explain later, the prover will also precompute similar tables for the values $L_{r_1-j}(\mathbf{b}, \mathbf{c})$ of the levels close to the leaves (i.e. for small j),

$$T_{r_1-j} \in \left(\mathbb{K}^{2^{2^j}} \right)^{\{0, 1\}^{r_1-j}}, \quad (40)$$

as well as other tables relevant for the different sumcheck rounds of [Construction 6.1](#), constructed from the tables T_{r_1-j} themselves. The superindex $\{0, 1\}^{r_1-j}$ in [\(40\)](#) indicates that the table T_{r_1-j} has one entry per element in the hypercube $\{0, 1\}^{r_1-j}$; and this entry is a vector of 2^{2^j} field elements. Concretely, for $\mathbf{b} \in \{0, 1\}^{r_1-j}$, $T_{r_1-j}[\mathbf{b}]$ consists of all possible products of 2^j consecutive factors of the grand product, namely the factors indexed by $(\mathbf{b}, \mathbf{b}')$ with $\mathbf{b}$ fixed and $\mathbf{b}'$ ranging over $\{0, 1\}^j$. Each factor can take two values, depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{b}', \mathbf{c}}$ is 1 or 0. Hence $T_{r_1-j}[\mathbf{b}]$ consists of 2^{2^j} elements. Hence the size of T_{r_1-j} in [\(40\)](#).

Because $|T_{r_1-j}|$ grows doubly-exponentially in j , these tables are viable only for the bottom few levels of the tree ($j = 0, 1, 2$ in our implementation). In higher levels we use standard sumchecks.

Importantly, all these tables have size independent of ℓ_2 . This (among others, cf. ??), is one of the reasons why we use the tensor decomposition structure, i.e. the split given by $\ell = \ell_1 \cdot \ell_2$, in [Construction 5.4](#).

Remark 6.3. All precomputed data in this section depends on the input vector $\mathbf{y}$ of $\mathcal{R}_{\text{PESAT}}(\mathbb{K}, p)$, and possibly on further verifier challenges. Since, in a wider context where [Construction 6.1](#) is a subprocess of a larger proof system, $\mathbf{y}$ typically depends on verifier's challenges, these precomputations must be done at *runtime*.

First prover message in the last sumcheck in Step 2 of [Construction 6.1](#) We start by analyzing Step 2 with $i = r_1 - 1$. In this step prover and verifier run a sumcheck for the claim

$$C^{(r_1-1)} = \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1} \times \{0,1\}^{r_2}} L_{r_1}(\mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)})$$

This is the most expensive sumcheck of [Construction 6.1](#) for the prover (if done naively), since it is the sumcheck over the largest number of variables throughout [Construction 6.1](#).

For $1 \leq j \leq r_1 - 1$, the j -th prover message in this sumcheck is, if $\boldsymbol{\rho}^{(j)} = (\rho_1, \dots, \rho_{j-1})$ is the vector of verifier challenges so far in this sumcheck,

$$\begin{aligned} h_j(X) &:= \sum_{(\mathbf{b}, \mathbf{c}) \in \{0,1\}^{r_1-1-j} \times \{0,1\}^{r_2}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, \mathbf{c}; \boldsymbol{\alpha}^{(i)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} \left(\underbrace{\sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) \cdot \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)})}_{:= h_{j,\mathbf{c}}(X)} \right) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}) \\ &= \sum_{\mathbf{c} \in \{0,1\}^{r_2}} h_{j,\mathbf{c}}(X) \cdot \text{eq}(X, \mathbf{c}; \boldsymbol{\alpha}^{(i,2)}), \end{aligned}$$

where $\boldsymbol{\alpha}^{(i)} = (\boldsymbol{\alpha}^{(i,1)}, \boldsymbol{\alpha}^{(i,2)}) \in \mathbb{K}^{r_1-2} \times \mathbb{K}^{r_2+1}$. We claim that

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 0, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 0, \mathbf{c}) \quad (41)$$

Indeed, $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ is a linear polynomial, and the right-hand side in the above expression is the line passing through the two evaluations of $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 0, \mathbf{c})$ at $X = 0$ and $X = 1$. An analogous expression holds for $L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c})$:

$$L_{r_1}(\boldsymbol{\rho}^{(j)}, X, \mathbf{b}, 1, \mathbf{c}) = \left(L_{r_1}(\boldsymbol{\rho}^{(j)}, 1, \mathbf{b}, 1, \mathbf{c}) - L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}) \right) \cdot X + L_{r_1}(\boldsymbol{\rho}^{(j)}, 0, \mathbf{b}, 1, \mathbf{c}). \quad (42)$$

Renaming

$$E_{\varepsilon_1 \varepsilon_2}^{(j)}[\mathbf{b}, \mathbf{c}] := L_{r_1}(\boldsymbol{\rho}^{(j)}, \varepsilon_1, \mathbf{b}, \varepsilon_2, \mathbf{c}) \quad \varepsilon_1, \varepsilon_2 \in \{0, 1\}, \quad \omega[\mathbf{b}] := \text{eq}(\boldsymbol{\rho}^{(j)}, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)}),$$

we obtain

$$h_{j,\mathbf{c}}(X) = \sum_{\mathbf{b} \in \{0,1\}^{r_1-1-j}} X^2 \cdot A_2^{(j)}[\mathbf{b}, \mathbf{c}] + X \cdot A_1^{(j)}[\mathbf{b}, \mathbf{c}] + A_0^{(j)}[\mathbf{b}, \mathbf{c}]$$

where,

$$\begin{aligned} A_0^{(j)}[\mathbf{b}, \mathbf{c}] &:= E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}], \\ A_1^{(j)}[\mathbf{b}, \mathbf{c}] &:= \left((E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{01}^{(j)}[\mathbf{b}, \mathbf{c}] + (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot E_{00}^{(j)}[\mathbf{b}, \mathbf{c}] \right) \cdot \omega[\mathbf{b}] \\ A_2^{(j)}[\mathbf{b}, \mathbf{c}] &:= (E_{10}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{00}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot (E_{11}^{(j)}[\mathbf{b}, \mathbf{c}] - E_{01}^{(j)}[\mathbf{b}, \mathbf{c}]) \cdot \omega[\mathbf{b}]. \end{aligned}$$

Now, for $j = 1$, we have $\boldsymbol{\rho}^{(j)} = \perp$, and hence each value $E_{\varepsilon_1 \varepsilon_2}^{(1)}[\mathbf{b}, \mathbf{c}]$ can be obtained, whenever needed, by querying $T_{r_1}[\mathbf{b}]$ and selecting $T_{r_1}[\mathbf{b}][1]$ or $T_{r_1}[\mathbf{b}][0]$ depending on whether $\mathbf{f}_{\mathbf{b}, \mathbf{c}} = 1$ or 0. A consequence is that the coefficients themselves $A_k^{(1)}[\mathbf{b}, \mathbf{c}]$ ($k \in [3]$) can be obtained through lookup tables, as follows: we have

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = T_{r_1}[0, \mathbf{b}, 0][\mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}}] \cdot T_{r_1}[0, \mathbf{b}, 1][\mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}}] \cdot \omega[\mathbf{b}]$$

Hence

$$A_0^{(1)}[\mathbf{b}, \mathbf{c}] = \begin{cases} \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 1, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 1 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 1, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 0 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 0, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 1 \\ \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}] & \text{if } \mathbf{f}_{(0, \mathbf{b}, 0), \mathbf{c}} = 0, \mathbf{f}_{(0, \mathbf{b}, 1), \mathbf{c}} = 0 \end{cases}$$

Thus the prover can precompute the table

$$T_{A_0} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}} = \mathbb{K}^{\ell_1}$$

given by

$$T_{A_0}[\mathbf{b}] = (\mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(1)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(1)} \cdot \omega[\mathbf{b}], \mathbf{y}_{0, \mathbf{b}, 0}^{(0)} \cdot \mathbf{y}_{0, \mathbf{b}, 1}^{(0)} \cdot \omega[\mathbf{b}])$$

and use it to obtain any value $A_0^{(1)}[\mathbf{b}, \mathbf{c}]$ for $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}$ via a single read into T_{A_0} . The size of T_{A_0} , again, does not depend on $\ell_2 = 2^{r_2}$. Building T_{A_0} requires at most $8 \cdot 2^{r_1-2} = 2^{r_1+1}$ multiplications and no additions.

We now explain how to obtain the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ and $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$, again via the construction of a lookup table. Note that these depend on the *four* bits $\mathbf{f}_{(\varepsilon_1, \mathbf{b}, \varepsilon_2), \mathbf{c}}$, $\varepsilon_1, \varepsilon_2 \in \{0, 1\}$, so our tables will need $2^4 = 16$ entries (as opposed to 4 as in T_{A_0}) per index $\mathbf{b}$. The prover proceeds as follows:

- It computes a table $T_{S_1} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-2}}$ analogous to T_{A_0} , which allows to obtain the values

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] := E_{10}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{11}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}],$$

(observe that T_{A_0} allows to obtain the values $E_{00}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot E_{01}^{(1)}[\mathbf{b}, \mathbf{c}] \cdot \omega[\mathbf{b}]$).

- It computes the table $T_{A_2} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-2}}$ given by

$$T_{A_2}[\mathbf{b}] = \left((\mathbf{y}_{1, \mathbf{b}, 0}^{(x_1)} - \mathbf{y}_{0, \mathbf{b}, 0}^{(x_0)}) \cdot (\mathbf{y}_{1, \mathbf{b}, 1}^{(x'_1)} - \mathbf{y}_{0, \mathbf{b}, 1}^{(x'_0)}) \cdot \omega[\mathbf{b}] \right)_{(x_0, x_1, x'_0, x'_1) \in \{0,1\}^4}, \quad (43)$$

so that $A_2^{(1)}[\mathbf{b}, \mathbf{c}]$ is the element

$$A_2^{(1)}[\mathbf{b}, \mathbf{c}] = T_{A_2}[\mathbf{b}][\mathbf{f}_{(0,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(0,\mathbf{b},1),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b},1),\mathbf{c}}].$$

Building T_{A_2} takes at most 20 multiplications and 8 additions per index $\mathbf{b}$ ⁷. I.e. at most $5 \cdot 2^{r_1}$ multiplications and 2^{r_1+1} additions in total. [Albert: Discuss cost of other tables?](#)

- It obtains the values $A_1^{(1)}[\mathbf{b}, \mathbf{c}]$ via the identity

$$S_1^{(1)}[\mathbf{b}, \mathbf{c}] = A_0^{(1)}[\mathbf{b}, \mathbf{c}] + A_1^{(1)}[\mathbf{b}, \mathbf{c}] + A_2^{(1)}[\mathbf{b}, \mathbf{c}] \quad \text{for all } (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-2} \times \{0, 1\}^{r_2}.$$

Once the tables have been computed, obtaining $h_{1,\mathbf{c}}(X)$ for one $\mathbf{c} \in \{0, 1\}^{r_2}$ costs $3 \cdot 2^{r_1-2}$ table reads (one read per table $T_{A_0}, T_{S_1}, T_{A_2}$ per index $\mathbf{b} \in \{0, 1\}^{r_1-2}$) and $3 \cdot 2^{r_1-2} + 2$ additions: this is the cost of adding together the values read at each table, providing $\sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}]$, $\sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$, and then doing two additions to obtain $\sum_{\mathbf{b}} A_1[\mathbf{b}, \mathbf{c}] = \sum_{\mathbf{b}} S_1[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_0[\mathbf{b}, \mathbf{c}] - \sum_{\mathbf{b}} A_2[\mathbf{b}, \mathbf{c}]$. In total, this amounts to at most $3\ell/4$ table reads and $\approx 3\ell/4 + 2 \cdot \ell_2$ additions to compute $h_{1,\mathbf{c}}$ for all $\mathbf{c}$, and no multiplications (in our instantiation, additions in $\mathbb{K}$ are 128-bit XORs).

Finally, the prover computes $h_1(X)$ from the polynomials $h_{1,\mathbf{c}}(X)$ with at most $5 \cdot 2^{r_2}$ additions and $3 \cdot 2^{r_2}$ multiplications (plus 2^{r_2} multiplications to precompute the values $\text{eq}(\mathbf{c}; \boldsymbol{\alpha}^{(i,2)})$, shared by all rounds of this sumcheck, and 2^{r_1-2} multiplications for the weights $\omega[\mathbf{b}]$).

In [Table 9](#) we record the total upper bound costs for computing $h_1(X)$. In our application we obtain lower numbers since $\mathbf{y}_{\mathbf{b}}^{(0)} = 1$ and we work in characteristic two, and so in [Eq. \(43\)](#) some elements collapse to 0.

On the other hand, a standard generic approach to compute $h_1(X)$ would require $O(\ell) = O(\ell_1 \cdot \ell_2)$ multiplications and additions, even if using the few sumcheck optimizations that apply in our settings (to our knowledge) [Albert: which ones, exactly?](#). Crucially, we note that our sumcheck features summands of full field bit size $\log |\mathbb{K}|$, which voids some known powerful optimizations when the summands are small, e.g. the univariate skip [\[Gru24\]](#) and some optimizations from [\[DT24, BDDT25\]](#). The prover message degree reduction through factoring the “eq factor” from [\[Gru24\]](#) does apply to our setting, and our implementation uses it: we send 2 elements instead of the 3 in each sumcheck round.

	Our approach	Standard/generic
Multiplications	$\leq 10 \cdot \ell_1 + 4 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Additions	$\leq 3 \cdot \ell_1 \cdot \ell_2 / 4 + 2 \cdot \ell_1 + 5 \cdot \ell_2$	$\Theta(\ell_1 \cdot \ell_2)$
Table reads	$3 \cdot \ell_1 \cdot \ell_2 / 4$, into tables of $6 \cdot \ell_1$ field elements	–

Table 9: The cost in field operations of the first sumcheck round of the last sumcheck in [Construction 6.1](#) (Step 2 with $i = r_1 - 1$), where $\ell = \ell_1 \cdot \ell_2 = 2^{r_1} \cdot 2^{r_2}$ is the witness size. In our instantiation $\ell_1 \approx \ell^{0.6} = 2^{r_1}$ and $\ell_2 \approx \ell^{0.4} = 2^{r_2}$ ([Section 5.7](#)).

⁷For the multiplications, there are 4 multiplications to compute the 4 elements $t_{x'_0, x'_1, \mathbf{b}} := (\mathbf{y}_{1, \mathbf{b}, 1}^{(x'_1)} - \mathbf{y}_{0, \mathbf{b}, 1}^{(x'_0)}) \cdot \omega[\mathbf{b}]$, and then 16 more multiplications to compute $(\mathbf{y}_{1, \mathbf{b}, 0}^{(x_1)} - \mathbf{y}_{0, \mathbf{b}, 0}^{(x_0)}) \cdot t_{x'_0, x'_1, \mathbf{b}}$

Subsequent prover messages in the last sumcheck in Step 2 of Construction 6.1 Next we focus on computing $h_2(X)$. Substituting X by the verifier's first sumcheck challenge ρ_1 in Eqs. (41) and (42), we have, for every $\mathbf{b} \in \{0, 1\}^{r_1-1}$, $\mathbf{c} \in \{0, 1\}^{r_2}$,

$$\begin{aligned} L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) &= (L_{r_1}(1, \mathbf{b}, \mathbf{c}) - L_{r_1}(0, \mathbf{b}, \mathbf{c})) \cdot \rho_1 + L_{r_1}(0, \mathbf{b}, \mathbf{c}) \\ &= \begin{cases} \mathbf{y}_{\mathbf{b}}^{(11)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(1)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(1)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(10)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 1, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \\ \mathbf{y}_{\mathbf{b}}^{(01)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(1)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 1 \\ \mathbf{y}_{\mathbf{b}}^{(00)} := \left(\mathbf{y}_{1,\mathbf{b}}^{(0)} - \mathbf{y}_{0,\mathbf{b}}^{(0)} \right) \cdot \rho_1 + \mathbf{y}_{0,\mathbf{b}}^{(0)} & \text{if } \mathbf{f}_{(0,\mathbf{b}),\mathbf{c}} = 0, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}} = 0 \end{cases} \end{aligned}$$

The idea now is that the prover can compute $h_2(X)$ in essentially the same way as $h_1(X)$, but this time, instead of starting with the table

$$T_{r_1}^{(1)} := T_{r_1} \in (\mathbb{K}^2)^{\{0,1\}^{r_1-1}} \quad \text{such that } T_{r_1}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(0)}, \mathbf{y}_{\mathbf{b}}^{(1)}) \text{ and } L_{r_1}(\mathbf{b}, \mathbf{c}) = T_{r_1}[\mathbf{b}][\mathbf{f}_{\mathbf{b},\mathbf{c}}], \quad (\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-1} \times \{0, 1\}^{r_2},$$

it starts with the table

$$T_{r_1}^{(2)} \in (\mathbb{K}^4)^{\{0,1\}^{r_1-1}} \quad \text{such that } T_{r_1}^{(2)}[\mathbf{b}] = (\mathbf{y}_{\mathbf{b}}^{(11)}, \mathbf{y}_{\mathbf{b}}^{(10)}, \mathbf{y}_{\mathbf{b}}^{(01)}, \mathbf{y}_{\mathbf{b}}^{(00)}) \quad \text{for } \mathbf{b} \in \{0, 1\}^{r_1-1}$$

for which we have, indexing the entries of $T_{r_1}^{(2)}[\mathbf{b}]$ with the 4 values $(1, 1), (1, 0), (0, 1), (0, 0)$,

$$L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c}) = T_{r_1}^{(2)}[\mathbf{b}][(\mathbf{f}_{(0,\mathbf{b}),\mathbf{c}}, \mathbf{f}_{(1,\mathbf{b}),\mathbf{c}})]$$

This allows the prover, when needed, to obtain $L_{r_1}(\rho_1, \mathbf{b}, \mathbf{c})$ for any $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-1} \times \{0, 1\}^{r_2}$ with a simple read in $T_{r_1}^{(2)}$.

We now have, for $(\mathbf{b}, \mathbf{c}) \in \{0, 1\}^{r_1-3} \times \{0, 1\}^{r_2}$,

$$\begin{aligned} A_0^{(2)}[\mathbf{b}, \mathbf{c}] &= L_{r_1}(\rho_1, 0, \mathbf{b}, 0, \mathbf{c}) \cdot L_{r_1}(\rho_1, 0, \mathbf{b}, 1, \mathbf{c}) \cdot \omega[\mathbf{b}] \\ &= T_{r_1}^{(2)}[0, \mathbf{b}, 0][(\mathbf{f}_{(0,0,\mathbf{b},0),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},0),\mathbf{c}})] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][(\mathbf{f}_{(0,0,\mathbf{b},1),\mathbf{c}}, \mathbf{f}_{(1,0,\mathbf{b},1),\mathbf{c}})] \cdot \omega[\mathbf{b}] \end{aligned}$$

where $\omega[\mathbf{b}] = \text{eq}(\rho_1, \mathbf{b}; \boldsymbol{\alpha}^{(i,1)})$ is the weight defined above for $j = 2$. Hence the elements $A_0^{(2)}[\mathbf{b}, \mathbf{c}]$ can be obtained by querying a table

$$T_{A_0}^{(2)} \in (\mathbb{K}^{16})^{\{0,1\}^{r_1-3}}$$

with

$$T_{A_0}^{(2)}[\mathbf{b}] = (T_{r_1}^{(2)}[0, \mathbf{b}, 0][\mathbf{x}] \cdot T_{r_1}^{(2)}[0, \mathbf{b}, 1][\mathbf{x}'] \cdot \omega[\mathbf{b}])_{\mathbf{x}, \mathbf{x}' \in \{0,1\}^2}$$

A table $T_{S_1}^{(2)}$ is computed analogously to T_{S_1} . The value $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$, however, now depends on the value taken by *eight* witness bits, this being because for $\varepsilon \in \{0, 1\}$, the value $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$ depends on four bits. Hence computing a table for the values $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$ analogous to T_{A_2} would have $2^8 = 256$ -sized vectors as entries. Instead of computing $T_{A_2}^{(2)}$, we compute two tables for the values $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$ (one table for each $\varepsilon \in \{0, 1\}$, with one of the tables also including multiplication by $\omega[\mathbf{b}]$). Then the value $A_2^{(2)}[\mathbf{b}, \mathbf{c}]$ can be obtained by performing a lookup into each table, followed by a single multiplication.

This can be iterated in further sumcheck rounds, for computing $h_3(X), h_4(X)$, etc. The limitation is that the entries in the tables are vectors whose dimension is squared at each round. More precisely, each further round squares the number of cases per index of every table, but it only halves the number of indices. Hence, in practice we stop using lookup tables after a few rounds of each sumcheck and GKR, and after that we execute the sumcheck protocol normally. Concretely, in our implementation we never compute a table with more than 16 elements per index. This is done so that the tables can be kept in cache, since we observe that, when this is not the case, table lookups become more and more expensive compared to multiplications in $\mathbb{F}_{2^{128}}$. [Table 10](#) displays the different tables built or not built in our simplest implementation, the arithmetic cost of doing so, and their size at $\ell_1 = 2^{r_1} = 2^{16}$ (corresponding to a total witness size of $\approx 2^{26}, 2^{27}$ bits in our implementation). In our final implementation we do not explicitly compute all of the tables, similarly as done with the table $T_{A_2}^{(2)}$ above. See [Remark 6.4](#) for details.

Table	Dimension	Adds/mults. in $\mathbb{K}$	Size (MB), $\ell_1 = 2^{16}$	Built?
$T_{r_1}^{(1)}$	$(\mathbb{K}^2)^{2^{r_1}}$	0/0	2.1	yes
$T_{A_0}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{S_1}^{(1)}$	$(\mathbb{K}^{2^2})^{2^{r_1-2}}$	0 / 2^{r_1+1}	1.0	yes
$T_{A_2}^{(1)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+1} / $5 \cdot 2^{r_1}$	4.2	yes
$T_{r_1}^{(2)}$	$(\mathbb{K}^{2^2})^{2^{r_1-1}}$	2^{r_1+2} / 2^{r_1+1}	2.1	yes
$T_{A_0}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{S_1}^{(2)}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$	0 / 2^{r_1+2}	2.1	yes
$T_{A_2}^{(2)}$	$(\mathbb{K}^{2^8})^{2^{r_1-3}}$	—	33.6	no
tables for $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}, \varepsilon \in \{0,1\}$	$(\mathbb{K}^{2^4})^{2^{r_1-3}}$ each	2^{r_1+2} / 2^{r_1+1}	2.1 each	yes
$T_{r_1}^{(3)}$	$(\mathbb{K}^{2^4})^{2^{r_1-2}}$	2^{r_1+3} / 2^{r_1+2}	4.2	yes
$T_{A_0}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{S_1}^{(3)}$	$(\mathbb{K}^{2^8})^{2^{r_1-4}}$	—	16.8	no
$T_{A_2}^{(3)}$	$(\mathbb{K}^{2^{16}})^{2^{r_1-4}}$	—	4295	no

Table 10: Dimensions, build costs, and sizes of the tables for computing $h_1(X), h_2(X), h_3(X)$ in the last sumcheck of [Construction 6.1](#), with $\log |\mathbb{K}| \approx 128$. We use the example size $\ell_1 = 2^{r_1} = 2^{16}$, which, following our instantiation ($\ell_1 \approx \ell^{0.6}$, [Section 5.7](#)), corresponds to a total witness size $\ell \approx 2^{26}, 2^{27}$. See [Remark 6.4](#) for the tables that our implementation does not build.

The rest of sumchecks in Step 2 of [Construction 6.1](#) So far we have discussed the last sumcheck from Step 2 of [Construction 6.1](#), corresponding to the lower layer of the binary tree of products of our GKR circuit for the grand products.

The sumchecks run for $i = r_1 - 2, r_1 - 3, \dots$ can be optimized in the exact same way as we did for the last sumcheck ($i = r_1 - 1$). In this case, instead of starting with the table $T_{r_1}^{(1)}$, which at each index contains the two values $(\mathbf{y}_b^{(0)}, \mathbf{y}_b^{(1)})$, one starts with the table

$$T_{r_1-j} \in (\mathbb{K}^{2^{2^j}})^{\{0,1\}^{r_1-j}}$$

described in [Eq. \(40\)](#). Since T_{r_1-j} has, per index, the square of the number of entries of T_{r_1-j+1} , all subsequent tables also have, per index, the square of the number of entries of the analogous tables from the $(i+1)$ -th sumcheck, and they require correspondingly more operations to compute.

In our implementation we want to keep all tables in cache (otherwise we observe that multiplication in $\mathbb{F}_{2^{128}}$ is cheaper than a table lookup). Because of this, the smaller i is (i.e. the further we get from the last sumcheck in the GKR protocol), the fewer number of sumcheck rounds we use tables in. Concretely, for $i = r_1 - 2$ ($j = 1$) we build and use the table $T_{r_1-1}^{(1)}$, the analogues of $T_{A_0}^{(1)}$ and $T_{S_1}^{(1)}$ (now with 16 entries per index), and $T_{r_1-1}^{(2)}$. Similarly as done for $A_2^{(2)}$, the analogue of $T_{A_2}^{(1)}$ (which would have 2^8 entries per index) is replaced by two tables of 16 entries per index and one multiplication per read. For $i = r_1 - 3$ ($j = 2$) we only use the table $T_{r_1-2}^{(1)}$.

Remark 6.4 (Factored tables). The tables and operation counts in [Tables 9](#) and [10](#) describe the algorithm as presented above. In our implementation, the four 16-entry tables of the second round of the last sumcheck (namely $T_{A_0}^{(2)}$, $T_{S_1}^{(2)}$ and the two tables for the values $E_{1\varepsilon}^{(2)} - E_{0\varepsilon}^{(2)}$), their analogues in the first round of the sumcheck for $i = r_1 - 2$, and the table $T_{r_1-2}^{(1)}$ are not built explicitly. Instead, the prover builds smaller 4-entry tables and recomputes each required value from two of their entries with one multiplication.

Remark 6.5 (Lookup tables in Round 2 of [Construction 5.4](#)). In Round 2 of [Construction 5.4](#), the prover computes the elements $\mu_1, \dots, \mu_{\ell_2}$ of [\(23\)](#) again by means of a lookup table T , which is indexed over $i \in [\ell_1/4]$ and contains the 16-length vectors

$$T[i] = \left(\sum_{k \in [4]} \varepsilon_k \cdot \pi_q^{-1}(\mathbf{v}^{(1)})_{4(i-1)+k} \right)_{\varepsilon_1, \dots, \varepsilon_4 \in \{0,1\}} \in \mathbb{Z}^{16}.$$

Albert: More optimizations?

7 Experiments

Albert: Peak memory missing

Albert: We should optimize the verifier

We implement the BitZ PCS ([Constructions 5.4](#) and [5.14](#)) following the guideline in [Section 5.7](#). We then implement an end-to-end roof system, called BitZ-SNARK, for integer RICS constraints as described in ?? [Albert: technical overview](#) (note that BitZ can be used to prove much more general constraints, such as the ones in Zinc+ [\[AGK+26\]](#), cf. ?? [Albert: some remark in TO](#)).

Following the lead of ?? and other usages of AI ?? to optimize against verified specs, we used AI to optimize the prover while keeping the verifier implementation untouched. As such, we consider our prover fairly optimal, while our verifier's implementation is pending significant optimization [Albert: review](#).

Future work: lattice-type computations Since the BitZ PCS can prove MLE evaluations over arbitrary finitely generated rings, it can be combined with, for example, the arithmetization and PIOP from Zinc+ [\[AGK+26\]](#) so as to prove lattice operations (e.g. Falcon signature verification). Implementing and benchmarking this is left as future work.

7.1 Raw performance of the BitZ PCS for the core LinBitsRings relation

We measure the performance of [Construction 5.4](#) for proving the Core LinBitsRing relation $\mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, \mathbb{F}_q)$, following the instantiation in [Section 5.7](#). Concretely, $\mathbb{F}_{2^\nu} = \mathbb{F}_{2^{128}}$ and $\mathcal{C}$ is a Reed-Solomon code.

We set $|\mathbb{F}_q| < (2^{128} - 1)/\ell_1$ and we use the unique decoding radius as proximity parameter, so that Round 0 (OOD sampling) and Round 1 of [Construction 5.4](#) do not trigger. Our goal for now is to measure the performance of the grand product and the WHIR/Ligerito commit and opening. In later experiments, Round 0 and Round 1 do trigger.

$\log_2 \ell$	Prover time (ms)					Verifier (ms)	Proof size (KB)		
	Commit	Grand prod.	Ring switch	Ligerito	Total		Non-Lig.	Ligerito	Total
20	0.93	4.88	1.22	4.46	11.8	1.45	15.1	44.9	60.0
21	0.79	6.93	1.28	4.59	13.9	1.78	16.0	49.0	65.0
22	1.32	10.1	1.60	5.00	18.5	2.29	16.9	53.5	70.4
23	2.19	17.2	2.08	12.0	34.1	2.30	21.8	65.2	87.0
24	4.05	28.6	2.83	10.8	47.4	3.02	22.7	71.0	93.7
25	7.98	50.8	4.52	12.4	76.9	3.43	31.5	76.3	108
26	15.2	91.0	7.59	17.9	134	4.14	32.8	89.5	122
27	32.7	169	13.5	25.6	244	6.15	33.8	94.9	129
28	62.9	330	23.5	34.3	453	6.56	50.8	101	152
29	139	701	61.6	66.8	982	9.63	52.2	116	168
30	277	1736	156	170	2318	9.98	85.6	123	208

Table 11: Cost of BitZ ([Construction 5.4](#)) for committing to $\ell = 2^n$ bits, $n = 20, \dots, 30$, and proving one claim $\langle \mathbf{v}, \mathbf{f} \rangle = \mu$ over $\mathbb{F}_q$ for a prime q sampled uniformly (after the commitment) from $[2^{b-1}, 2^b)$, $b = \min(113, 127 - \log \ell_1 - W)$ at cell width $W = 1$ ($b = 108, \dots, 113$), $\mathbf{v} = \text{eq}(\cdot, \mathbf{r}_1) \otimes \text{eq}(\cdot, \mathbf{r}_2)$ with $(\mathbf{r}_1, \mathbf{r}_2)$ sampled after q , and the tensor split $\ell_1 = 2^{\lceil 0.6n \rceil}$, $\ell_1 \cdot \ell_2 = \ell$ ([Section 5.7](#)). The commitment is opened with ring switching and Ligerito [[NA25](#)] over a Reed–Solomon code of rate 1/8 over $\mathbb{F}_{2^{128}}$ (initial folding of 2^4 rows, Johnson regime, BLAKE3 Merkle trees), configured for 100 bits of round-by-round security (the minimum over its levels and error terms is 100.0 bits); Round 0 (the out-of-domain sample) is executed with $\lceil \log \ell - 23.7 \rceil$ bits of grinding (0 to 7 over the table); every other round (the GKR, the sumcheck reducing to MLE evaluation claims, the ring switch) has error at most $7 \cdot 2^{-128}$. Prover columns: *commit* is the commitment to the ℓ bits; *grand products* is computing the integers μ_j and the batched GKR for the ℓ_2 grand products in the exponent ([Section 6](#)); *ring switch* is the sumcheck reducing the GKR output claims to MLE evaluation claims together with the ring-switching step; *Ligerito* is the Ligerito opening; *total* is the commitment plus the end-to-end proving time (each entry is a median, so the parts need not add up exactly). *Non-Ligerito* proof bytes are the μ_j , the GKR and sumcheck messages, and the ring-switch message; KB = 1000 bytes. Apple M4 (10 cores: 4 performance + 6 efficiency), 16 GB, 8 threads; medians of 5 runs after one warm-up.

Albert: Let’s have the AI and human repo output the same transcripts

Albert: Measure Binius64 proving the same inner product directly in Bin fields?

7.2 Integer multiplication

Albert: Add Limber

We measure the cost of proving N integer multiplications of the form $x \cdot y = z$, for x, y random 32-bit integers, and z a 64-bit integer. As we mentioned, this is proved via our implemented end-to-end proof system, called BitZ-SNARK for R1CS over the integers, with BitZ as its PCS, as described in ??.

Witness		Prover time (ms)						Verifier	Proof size (KB)			
$\log_2 N$	(ms)	Commit	PIOP	Grand prod.	Ring switch	Ligerito	Total	(ms)	PIOP	Non-Lig.	Ligerito	Total
15	0.04	1.40	1.37	10.4	1.84	6.31	21.8	4.38	1.7	15.2	53.4	70.4
16	0.10	2.35	2.21	16.4	2.22	12.0	35.8	6.84	1.9	18.1	65.4	85.4
17	0.18	4.74	3.36	28.1	3.38	9.65	51.9	14.0	2.0	19.1	71.3	92.4
18	0.49	9.12	5.97	49.9	4.48	13.6	85.8	23.8	2.1	23.8	76.9	103
19	1.05	17.7	10.1	94.7	7.23	19.2	153	44.2	2.2	25.0	89.4	117
20	1.70	39.2	18.3	178	11.1	24.8	276	81.1	2.3	33.8	94.3	130
21	2.06	77.0	31.3	382	20.3	38.8	550	150	2.4	35.0	102	139
22	3.63	150	64.8	745	36.4	60.8	1058	277	2.5	52.2	116	171

Table 12: Cost of proving N integer multiplications $x \cdot y = z$, $n = 15, \dots, 22$, for random 32-bit integers x, y (with z a 64-bit integer). Configured at 100 bits of security (cf. [Section 5.7](#)). Ran on a MacBook Air M4, 16 GB, 8 threads.

We compare the performance of BitZ-SNARK with Binius64 [\[Bin26\]](#), Plonky3 [\[Plo\]](#), Zinc+ [\[AGK+26\]](#), and Limber [\[CXNB26\]](#) on the same task. Both Binius64 and Plonky3 implement gadgets for proving integer multiplication constraints. We use those in our benchmarks [Albert: Confirm](#).

N	Scheme	Witgen (ms)	Prover (ms)	Verifier (ms)	Proof (KB)	Peak mem. (GB)
2^{15}	BitZ (this work)	0.04	16.3	4.84	158	—
	Binius64 [Bin26] , rate 1/2	0.63	43.1	1.75	327	—
	Plonky3 [Plo] (WHIR)	3.20	1313	3.93	196	—
2^{16}	BitZ (this work)	0.06	24.6	7.90	184	—
	Binius64 [Bin26] , rate 1/2	1.22	64.0	3.11	358	—
	Plonky3 [Plo] (WHIR)	6.25	2889	4.68	216	—
2^{17}	BitZ (this work)	0.10	43.0	14.9	205	—
	Binius64 [Bin26] , rate 1/2	2.44	109	5.57	403	—
	Plonky3 [Plo] (WHIR)	11.9	5939	5.08	226	—
2^{18}	BitZ (this work)	0.25	71.7	25.4	232	—
	Binius64 [Bin26] , rate 1/2	4.87	194	10.5	435	—
	Plonky3 [Plo] (WHIR)	23.6	12085	5.20	238	—
2^{19}	BitZ (this work)	0.52	130	45.4	265	—
	Binius64 [Bin26] , rate 1/2	10.1	398	29.0	475	—
	Plonky3 [Plo] (WHIR)	82.3	26283	5.23	249	—
2^{20}	BitZ (this work)	1.15	245	80.3	297	—
	Binius64 [Bin26] , rate 1/2	—	—	—	—	—
	Plonky3 [Plo] (WHIR)	412	63640	6.77	269	—
2^{21}	BitZ (this work)	6.08	685	151	325	—
	Binius64 [Bin26] , rate 1/2	—	—	—	—	—
	Plonky3 [Plo] (WHIR)	—	—	—	—	—
2^{22}	BitZ (this work)	3.71	1046	302	380	—
	Binius64 [Bin26] , rate 1/2	—	—	—	—	—
	Plonky3 [Plo] (WHIR)	—	—	—	—	—

Table 13: [Albert: We should do FRI Plonky3](#) Costs of proving N multiplications $x \cdot y = z$ for random 32 bit nonnegative integers x and y . Ran on a MacBook Air M4, 16 GB, 8 threads.

N	Scheme	Witgen (ms)	Prover (ms)	Verifier (ms)	Proof (KB)	Peak mem. (GB)
2^{15}	BitZ (this work)	0.04	25.8	3.73	184	—
	Binius64 [Bin26], rate 1/2	0.43	42.3	1.72	327	—
	Binius64 [Bin26], rate 1/8	0.37	46.0	1.56	245	—
2^{16}	BitZ (this work)	0.08	40.0	3.96	205	—
	Binius64 [Bin26], rate 1/2	0.75	63.0	2.82	358	—
	Binius64 [Bin26], rate 1/8	0.75	69.0	2.70	262	—
2^{17}	BitZ (this work)	0.18	73.8	8.70	227	—
	Binius64 [Bin26], rate 1/2	1.49	107	5.13	403	—
	Binius64 [Bin26], rate 1/8	1.48	119	4.85	286	—
2^{18}	BitZ (this work)	0.37	127	9.06	266	—
	Binius64 [Bin26], rate 1/2	3.07	189	9.86	435	—
	Binius64 [Bin26], rate 1/8	2.96	218	8.93	308	—
2^{19}	BitZ (this work)	0.74	251	11.3	289	—
	Binius64 [Bin26], rate 1/2	6.14	383	26.2	475	—
	Binius64 [Bin26], rate 1/8	6.13	437	26.2	333	—
2^{20}	BitZ (this work)	1.25	470	11.6	324	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	14.6	1022	52.0	358	—
2^{21}	BitZ (this work)	3.08	1187	14.9	362	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	—	—	—	—	—

Table 14: Native end-to-end proofs of N multiplications $x \cdot y = z$ of random 64-bit integers (z a 128-bit integer): BitZ (Spartan over a transcript-sampled prime with the BitZ opening, $\lambda = 100$) and Binius64 (native multiplication, ring switching and BaseFold at rate 1/8 with 121 queries for 100 bits). *Witgen* is the native witness generation; *prover* is the complete prover call after witness generation, commitment included; *verifier* is the complete verification. At 2^{21} the Binius64 prover exceeds the machine’s memory and pages; that row is omitted. Apple M4, 16 GB, 8 threads; medians of 5 runs after one warm-up.

N	Scheme	Witgen (ms)	Prover (ms)	Verifier (ms)	Proof (KB)	Peak mem. (GB)
2^{15}	BitZ (this work)	0.08	42.2	4.47	202	—
	Binius64 [Bin26], rate 1/2	1.78	110	5.89	403	—
	Binius64 [Bin26], rate 1/8	1.78	124	5.56	286	—
2^{16}	BitZ (this work)	0.16	70.1	4.48	228	—
	Binius64 [Bin26], rate 1/2	3.60	199	11.4	435	—
	Binius64 [Bin26], rate 1/8	3.53	228	10.6	308	—
2^{17}	BitZ (this work)	0.32	134	6.35	261	—
	Binius64 [Bin26], rate 1/2	7.13	395	29.8	475	—
	Binius64 [Bin26], rate 1/8	7.35	457	28.3	333	—
2^{18}	BitZ (this work)	0.63	241	6.41	289	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	—	—	—	—	—
2^{19}	BitZ (this work)	1.23	573	8.61	316	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	—	—	—	—	—
2^{20}	BitZ (this work)	2.31	1113	8.99	364	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	—	—	—	—	—
2^{21}	BitZ (this work)	4.76	2617	14.1	391	—
	Binius64 [Bin26], rate 1/2	—	—	—	—	—
	Binius64 [Bin26], rate 1/8	—	—	—	—	—

Table 15: Native end-to-end proofs of N multiplications $x \cdot y = z$ of random 128-bit integers (z a 256-bit integer): BitZ (Spartan over a transcript-sampled prime with the BitZ opening, $\lambda = 100$) and Binius64 (native multiplication, ring switching and BaseFold at rate 1/8 with 121 queries for 100 bits). *Witgen* is the native witness generation; *prover* is the complete prover call after witness generation, commitment included; *verifier* is the complete verification. Binius64 was not run at 2^{19} , 2^{20} , 2^{21} . At 2^{18} the Binius64 prover exceeds the machine’s memory and pages; that row is omitted. Apple M4, 16 GB, 8 threads; medians of 5 runs after one warm-up.

The Binius64 rows missing from the tables above are due to memory rather than time. Binius64 proves a multiplication $x \cdot y = z$ through an exponentiation argument in the multiplicative group of a 128-bit binary field [Bin26]: writing $z = z_{\text{lo}} + 2^{64} z_{\text{hi}}$, the statement becomes $(G^x)^y = G^{z_{\text{lo}}} \cdot (G^{2^{64}})^{z_{\text{hi}}}$ for a generator G , and each exponentiation is certified by a GKR-style product check over a layered tree of field elements. The prover materializes all layers of these trees: the variable-base tree for y alone has one 128-bit leaf per bit of y and per multiplication, and the fixed-base trees for x , z_{lo} and z_{hi} come on top, so that a single 64-bit multiplication costs several kilobytes of prover memory before the witness is even committed. On our 16 GB machine the resident set of the Binius64 prover reaches 8.3 GB at 2^{20} 64-bit multiplications and pages beyond; for 32-bit operands, whose gadget adds range checks, paging starts already at 2^{20} , and for 128-bit operands, where one multiplication becomes four such gates plus carry chains, at 2^{18} . The BitZ prover commits one bit per witness bit and its memory is dominated by the lookup tables of Section 6, a few bytes per committed bit: 3.9 GB at 2^{21} 64-bit multiplications.

7.3 RSA accumulation and Poseidon hashing

We compare Limber [CXNB26], Zinc+[AGK⁺26] and our BitZ-SNARK on proving the same circuit as in the Limber paper. This consists in verifying four exponentiations modulo a 2048-bit RSA modulus, with 352-bit exponents, plus a Poseidon-based hash-to-prime evaluation (see Section 7.1 of [CXNB26]). We take the corresponding R1CS constraint over the integers from the Limber repository. See ?? for further details on Limber, Zinc+, etc.

We set BitZ at 114 bits of security, matching Limber⁸ For the BitZ opening we use Ligerito in the unique-decoding regime (as opposed to the other benchmarks where we use the Johnson bound) at rate 1/8 with fold arity 4 **Albert: mention this elsewhere**, with grinding to achieve 114 bits of security.

Scheme	Prove (ms)		Verify (ms)		Proof (KB)
	1 thr	8 thr	1 thr	8 thr	
BitZ-SNARK (this work)	273	105	9.9	12.0	269
Zinc+ [AGK ⁺ 26]	2052	563	18.2	12.7	1272
Limber [CXNB26] (Brakedown)	1175	559	44.2	43.3	5769
Limber [CXNB26] (Hyrax)	1206	390	37.1	20.5	175
Arkworks/Garuda [†]	231000	–	25	–	7.2
MultiSwap-x.Jsnark [†]	88500	–	2	–	0.2

Table 16: Proving one MultiSwap [OWWB20] computation (four exponentiations modulo a 2048-bit RSA modulus, with 352-bit exponents, plus a Poseidon-based hash-to-prime evaluation). The first four schemes prove the same R1CS constraint over the integers, taken from [CXNB26], and were run on a MacBook Air M4 on one and 8 threads (thr). The two lower rows are a verbatim quote from [CXNB26, Table 1]. The first four schemes achieve 114 bits of security Zinc+ proof compresses to 847KB with `zstd`. [†] Quoted from [CXNB26]; not measured here.

7.4 SHA-256 hashing followed by ECDSA signature verification

We experiment with proving the SHA-256 hash of a 4KB message (a chain of 2⁶ SHA compressions) followed by an ECDSA signature verification on the message digest. This is a common application of proof systems in anonymous credentials. We compare BitZ-SNARK with Binius64 [Bin26], Zinc+ [AGK⁺26], **Albert: What else? Longfellow? Plonky3? Vega? Check the ethproofs dashboard.**

For BitZ, we arithmetize the computation as an integer R1CS constraint with $\mathbb{F}_2$ -virtualization (??). The SHA-256 constraints in particular are an instance of the CM relation from Eq. (14), obtained using the methods from Remark 3.3. In Table 18 we display the circuit and witness sizes achieved by our SHA-256 arithmetization, against that of binary-field native systems like Binius [Bin26] and Flock [BRW26].

We emphasize that, as done in the next experiment Section 7.5, when seeking to prove a large amount of hashes, it is preferable to arithmetize the hash over a binary field and follow the approach described in Section 7.5

⁸Limber states in the paper that it achieves 128 bits of security, but its fingerprint step only achieves 114 bits of security in the implementation, because the random prime sampled to perform the fingerprint is of 128 bits, and no other measures (like, say, grinding) are taken. **Albert: Double check and let the authors know.**

Scheme	Prove (ms)	Verify (ms)	Proof (KB)
BitZ-SNARK (this work)			
Zinc+ [AGK ⁺ 26]			
Binius64			
?			

Table 17: Proving a SHA-256 hash of a 4KB message (64 compressions) followed by ECDSA signature verification. Ran on a MacBook Air M4 multithreaded, configured at 100 and 128 bits of security.

Arithmetization type	Witness bit-size/compression
CM	7144 bits
R1CS over $\mathbb{F}_{2^\nu}$	31744 bits

Table 18: Witness bit-size per SHA-256 compression as per two different arithmetizations. The CM row indicates the bit-size when arithmetizing SHA-256 via an instance of the CM relation (see (14)) using the methods from Eq. (14) and Remark 3.3, see <https://github.com/reilabs/freigen/blob/324935e/Freigen/Circuit/Examples/Sha256.lean> Albert: Change this to avoid deanonymization in the EC submission. The last row displays the witness bit-size in the arithmetization from Binius64 [Bin26], which is an R1CS constraint over a binary field.

Albert: Use $\text{Id}_{2^r} \otimes M$ as virtualization matrix, leading to 2^r compressions per grand product

7.5 Mod 2^{32} multiplication and SHA-256 hashing

We prove simulataneously a large chain of SHA-256 compressions, together with a large amount of multiplications modulo 2^{32} . In this experiment, contrary to the previous one, SHA-256 is arithmetized over a binary field, using the R1CS constraint from Binius64 [Bin26]. The modular multiplications are arithmetized as an R1CS constrain over the integers, where each modular multiplication $x \cdot y = z \pmod{2^{32}}$ is expressed as an integer constraint of the form $x \cdot y = z + 2^{32} \cdot u$, where u is a new integer witness.

Our end-to-end BitZ-SNARK is built as described in ???. We compare our results with Binius64. To arithmetize modular multiplication in Binius64 we use a gadget in the repository built for that purpose Albert: double check.

References

- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: reed-solomon proximity testing with fewer queries. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 380–413. Springer, 2024.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: reed-solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques*,

Madrid, Spain, May 4-8, 2025, Proceedings, Part IV, volume 15604 of *Lecture Notes in Computer Science*, pages 214–243. Springer, 2025.

- [AGK⁺26] Alexander Abdugafarov, Albert Garreta, Amit Kumar, Michał Osadnik, Psi Vesely, Ilia Vlasov, and Kai Zhe Zheng. Zinc+: SNARKs for polynomial rings. Cryptology ePrint Archive, Paper 2026/855, 2026.
- [BCF⁺24] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. Blaze: Fast SNARKs from interleaved RAA codes. Cryptology ePrint Archive, Paper 2024/1609, 2024.
- [BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. Linear-time accumulation schemes. In *Theory of Cryptography - 23rd International Conference, TCC 2025*, 2025.
- [BDDT25] Suyash Bagad, Quang Dao, Yuval Domb, and Justin Thaler. Speeding up sum-check proving. Cryptology ePrint Archive, Paper 2025/1117, 2025.
- [BFS19] Benedikt Bünz, Ben Fisch, and Alan Szeponiec. Transparent SNARKs from DARK compilers. Cryptology ePrint Archive, Paper 2019/1229, 2019. EUROCRYPT 2020.
- [BHR⁺21] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. Time- and space-efficient arguments from groups of unknown order. Cryptology ePrint Archive, Paper 2021/358, 2021. CRYPTO 2021.
- [Bin26] Binius64. Binius64 whitepaper. GitHub repository, 2026.
- [BRW26] Benedikt Bünz, Ron Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026.
- [BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:17, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [CFW26] Alessandro Chiesa, Giacomo Fenzi, and Guy Weissenberg. Zero-knowledge IOPPs for constrained interleaved codes. Cryptology ePrint Archive, Paper 2026/391, 2026.
- [CHA24] Matteo Campanelli and Mathias Hall-Andersen. Fully succinct arguments over the integers from first principles. Cryptology ePrint Archive, Paper 2024/1548, 2024.
- [CXNB26] Jessica Chen, Lucas Xia, Wilson Nguyen, and Benedikt Bünz. Limber: Low overhead SNARKs for integers from any PCS. Cryptology ePrint Archive, Paper 2026/1635, 2026.

- [DP23] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784, 2023. EUROCRYPT 2025.
- [DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.
- [DT24] Quang Dao and Justin Thaler. More optimizations to sum-check proving. Cryptology ePrint Archive, Paper 2024/1210, 2024.
- [Gru24] Angus Gruen. Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108, 2024.
- [GWV⁺25] Albert Garreta, Hendrik Waldner, Ilia Vlasov, Katerina Hristova, Luca Dall’Ava, Marko Čupić, and Matthew Klein. Zinc: Succinct arguments with small arithmetic overheads from iops of proximity to the integers. In *Advances in Cryptology – CRYPTO 2025: 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VII*, pages 259–291, Berlin, Heidelberg, 2025. Springer-Verlag.
- [HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.
- [NA25] Andrija Novakovic and Guillermo Angeris. Ligerito: A small and concretely fast polynomial commitment scheme. Cryptology ePrint Archive, Paper 2025/1187, 2025.
- [OWWB20] Alex Ozdemir, Riad S. Wahby, Barry Whitehat, and Dan Boneh. Scaling verifiable computation using efficient set accumulators. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2075–2092, 2020.
- [Plo] Plonky3. Plonky3.
- [Sou26] Lev Soukhanov. Char 2 fieldswitch. HackMD post, 2026.
- [SSPP26] Alireza Shirzad, Sriram Sridhar, Dimitrios Papadopoulos, and Charalampos Papanthou. Relaxed modular PCS from arbitrary PCS and applications to SNARKs for integers. Cryptology ePrint Archive, Paper 2026/347, 2026.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. 01 2022.
- [ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.

A Deferred proofs

A.1 Completeness and soundness of **Construction 5.14**

Albert: Review: AI written

We prove [Theorem 5.16](#). For readability, we temporarily denote $\pi : \mathbb{Z} \rightarrow R$ for the reduction modulo $\text{char}(R)$ and we let

$$\mathcal{R}_{\text{gen}} := \mathcal{R}_{\text{LinBitsRings}}(S_{\leq B}^{< D_S}, \psi, R, \mathcal{C}, L), \quad \mathcal{R}_{\text{core}} := \mathcal{R}_{\text{LinBitsRings}}(\{0, 1\}, \pi, R, \mathcal{C}), \quad \mathbf{x}' := (\mu, \mathbf{u}),$$

and $\mathcal{R}_{\text{PESAT}} = \mathcal{R}_{\text{PESAT}}(\mathbb{F}_{2^\nu}, Q_{\text{BitZ}}, \mathcal{C}, \text{Pack}_\nu)$, $\mathcal{R}_{\text{Lin}} = \mathcal{R}_{\text{Lin}}(\mathbb{F}_{2^\nu}, \mathcal{C}, \text{Pack}_\nu)$ (as in [Section 5.4](#)). Recall from [Definition 5.10](#) that a bitification of $\mathbf{f} \in (S_{\leq B}^{< D_S})^{\ell_0}$ is any $\mathbf{g} \in \{0, 1\}^\ell$ with $\mathbf{f} = L \cdot \mathbf{g}_S$. For such $\mathbf{g}$, we say that $((\mu, \mathbf{v}), f, \mathbf{f})$ belongs to $\mathcal{R}_{\text{gen}}^\delta$ via $\mathbf{g}$ if $\Delta(f, \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu \circ \pi_2(\mathbf{g})) < \delta$ and $\langle \psi(\mathbf{f}), \mathbf{v} \rangle = \mu$. Then $((\mu, \mathbf{v}), f, \mathbf{f}) \in \mathcal{R}_{\text{gen}}^\delta$ if and only if $((\mu, \mathbf{v}), f, \mathbf{f})$ belongs to $\mathcal{R}_{\text{gen}}^\delta$ via some bitification $\mathbf{g}$ of $\mathbf{f}$. The honest prover uses $\mathbf{g}$ as witness in the call to [Construction 5.4](#). The same applies to $\mathcal{R}_{\text{gen}}$, with equality of words in place of the distance condition. Recall from [Definition 5.10](#) that $L \cdot \mathbf{g}_S \in (S_{\leq B}^{< D_S})^{\ell_0}$ for all $\mathbf{g} \in \{0, 1\}^\ell$.

For $\mathbf{g} \in \{0, 1\}^\ell$ let $\mathbf{f}_\mathbf{g} := L \cdot \mathbf{g}_S \in (S_{\leq B}^{< D_S})^{\ell_0}$. By [Lemma 5.13](#), $\langle \psi(\mathbf{f}_\mathbf{g}), \mathbf{v} \rangle = \langle \psi(\mathbf{g}_S), \psi(L)^\top \cdot \mathbf{v} \rangle = \langle \pi(\mathbf{g}), \mathbf{u} \rangle$, and hence

$$(\mathbf{x}', f, \mathbf{g}) \in \mathcal{R}_{\text{core}}^\delta \iff ((\mu, \mathbf{v}), f, \mathbf{f}_\mathbf{g}) \text{ belongs to } \mathcal{R}_{\text{gen}}^\delta \text{ via } \mathbf{g}, \quad (44)$$

and likewise for $\mathcal{R}_{\text{core}}$ and $\mathcal{R}_{\text{gen}}$. Completeness then follows from (44) and [Theorem 5.8](#).

For round-by-round knowledge soundness, we decompose [Construction 5.14](#) as in the proof of [Theorem 5.8](#): the first IOR Π'_1 is the instance translation $(\mu, \mathbf{v}) \mapsto \mathbf{x}'$ followed by the IOR Π_1 of that proof (of [Section A.2](#), if Round 0 is executed), and the remaining IORs are Π_{red} and Π_{iopp} (their variants from [Section A.2](#), if Round 0 is executed). Π'_1 has the same messages and output as Π_1 , and it is perfectly complete by (44). By [Lemma 4.7](#) it suffices to show that Π'_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{gen}}^\delta, \mathcal{R}_{\text{PESAT}}^\delta)$ (resp. $(\mathcal{R}_{\text{gen}}^\delta, \mathcal{R}_{\text{PESAT}}^{\delta, \zeta, y})$) and with the same errors as Π_1 ; here $\kappa_0 = \theta(\mathcal{P}, \mathcal{M}, \ell)$ is computed with respect to the tensor decomposition $\mathbf{u} = \mathbf{u}^{(1)} \otimes \mathbf{u}^{(2)}$ used in [Construction 5.4](#) (cf. [Remark 5.15](#)).

Let KState_1 be the knowledge state function of Π'_1 ; recall that its extractor is the identity map. We set $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{gen}}^\delta$ when tr contains no verifier message, and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{w})$ otherwise; and we set $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \mathbf{f}_\mathbf{w}$ if tr ends with the first verifier message and $\mathbf{w} \in \{0, 1\}^\ell$, and $\mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) := \mathbf{w}$ otherwise. The conditions of [Definition 4.5](#) hold: at the empty transcript by construction, at prover moves because KState is constant before the first verifier message and coincides with KState_1 afterwards, and at full transcripts because Π'_1 and Π_1 have the same output. At every verifier message after the first one, KState coincides with KState_1 before and after the message and $\mathcal{E}_{\text{rbr}}$ is the identity map, so the error of Π'_1 is that of Π_1 . Finally, let ρ be the first verifier message and tr the transcript preceding it, and suppose that $\text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w}) = 1$ and $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathcal{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho, \mathbf{w})) = 0$. Then $(\mathbf{x}', \mathbf{y}, \mathbf{w}) \notin \mathcal{R}_{\text{core}}^\delta$: otherwise $\mathbf{w} = \mathbf{g} \in \{0, 1\}^\ell$, $\mathcal{E}_{\text{rbr}}$ outputs $\mathbf{f}_\mathbf{g}$, and (44) gives $\text{KState}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{f}_\mathbf{g}) = 1$. We distinguish three cases.

- $\rho = (q, \pi_{\text{rand}})$ (Round 1 executed, Round 0 not executed). By the definition of KState_1 , $\mathbf{w} = \mathbf{g} \in \{0, 1\}^\ell$, and $\text{KState}_1(\mathbf{x}', \mathbf{y}, \text{tr}, \mathbf{g}) = 0$ by the above. This is precisely the event bounded by κ_0 in the proof of [Theorem 5.8](#), so the error is at most κ_0 .
- ρ is the empty message (Round 1 skipped). Then $\text{tr} \parallel \rho$ is the full transcript, so $\mathbf{w} = \mathbf{g} \in \{0, 1\}^\ell$, and, since appending the empty message does not change KState_1 , the prover-move condition for KState_1 gives $\text{KState}_1(\mathbf{x}', \mathbf{y}, \emptyset, \mathbf{g}) = 1$, i.e. $(\mathbf{x}', \mathbf{y}, \mathbf{g}) \in \mathcal{R}_{\text{core}}^\delta$, a contradiction. The error is 0.
- $\rho = \zeta$ (Round 0 executed). By the definition of KState_1 in [Section A.2](#), $\text{Coll}(f, \zeta)$ holds, which happens with probability at most κ_{OOD} over ζ by [Lemma A.2](#).

A.2 Soundness of Out Of Domain Sampling (OOD) in Construction 5.4

Albert: Review: AI written When δ exceeds the unique decoding radius of $\mathcal{C}$, Round 0 of Construction 5.4 is executed and, as in [ACFY24, ACFY25], the OOD claim is carried along by the subsequent relations: $\mathcal{R}_{\text{PESAT}}^{\zeta,y}$ consists of the triples $((\mathbf{x}, \zeta, y), \mathbf{y}, \mathbf{w})$ with $\zeta, y \in \mathbb{F}_{2^\nu}$, $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}_{\text{PESAT}}$ and $\text{Pack}_\nu(\mathbf{w})(\zeta) = y$, where ζ is the evaluation point of Round 0; $\mathcal{R}_{\text{Lin}}^{\zeta,y}$ is defined analogously, and $\mathcal{R}_{\text{PESAT}}^{\delta,\zeta,y}$, $\mathcal{R}_{\text{Lin}}^{\delta,\zeta,y}$ are obtained by replacing $\mathcal{R}_{\text{PESAT}}$, $\mathcal{R}_{\text{Lin}}$ with $\mathcal{R}_{\text{PESAT}}^\delta$, $\mathcal{R}_{\text{Lin}}^\delta$.

Theorem A.1 (Theorem 5.8 when Round 0 is executed). *Let $\delta \in (0, 1)$ and $L_\delta \geq 1$ be such that $\mathcal{C}$ is (δ, L_δ) -list-decodable, let ℓ/ν be a power of two, and assume that Round 0 of Construction 5.4 is executed and that the remaining hypotheses of Theorem 5.8 hold, those on Π_{red} and Π_{iopp} also with $\mathcal{R}_{\text{PESAT}}, \mathcal{R}_{\text{Lin}}, \mathcal{R}_{\text{PESAT}}^\delta, \mathcal{R}_{\text{Lin}}^\delta$ replaced by $\mathcal{R}_{\text{PESAT}}^{\zeta,y}, \mathcal{R}_{\text{Lin}}^{\zeta,y}, \mathcal{R}_{\text{PESAT}}^{\delta,\zeta,y}, \mathcal{R}_{\text{Lin}}^{\delta,\zeta,y}$ (where Π_{red} copies (ζ, y) from its input instance to its output instance). Then the conclusion of Theorem 5.8 holds with one more round and the additional first error term $\kappa_{\text{OOD}} \leq \binom{L_\delta}{2} \cdot (\ell/\nu - 1)/|\mathbb{F}_{2^\nu}|$.*

Lemma A.2 (OOD collision probability). *Let $f \in \mathbb{F}_{2^\nu}^m$, let $\text{List}(f, \delta)$ be the set of $\mathbf{g} \in \mathbb{F}_2^\ell$ with $\Delta(f, \text{Enc}_\mathcal{C} \circ \text{Pack}_\nu(\mathbf{g})) < \delta$, and let $\text{Coll}(f, \zeta)$ be the event that two distinct $\mathbf{g}, \mathbf{g}' \in \text{List}(f, \delta)$ satisfy $\text{Pack}_\nu(\mathbf{g})(\zeta) = \text{Pack}_\nu(\mathbf{g}')(\zeta)$. Then*

$$\Pr_{\zeta \leftarrow \mathbb{F}_{2^\nu}} [\text{Coll}(f, \zeta)] \leq \binom{L_\delta}{2} \cdot \frac{\ell/\nu - 1}{|\mathbb{F}_{2^\nu}|}.$$

Proof. Since $\text{Enc}_\mathcal{C} \circ \text{Pack}_\nu$ is injective, $|\text{List}(f, \delta)| \leq L_\delta$. For distinct $\mathbf{g}, \mathbf{g}'$ in the list, $P := \widetilde{\text{Pack}_\nu(\mathbf{g})} - \widetilde{\text{Pack}_\nu(\mathbf{g}')}$ is a nonzero multilinear polynomial in $r := \log(\ell/\nu)$ variables, and the substitution $X_i \leftarrow \zeta^{2^{i-1}}$ maps its monomials $\prod_{i \in I} X_i$ injectively to the powers $\zeta^{\sum_{i \in I} 2^{i-1}}$, with exponents in $[0 .. \ell/\nu - 1]$. Hence $P(\zeta)$ is a nonzero univariate polynomial in ζ of degree at most $\ell/\nu - 1$, and it vanishes for at most that many ζ . A union bound over the $\binom{L_\delta}{2}$ pairs concludes. $\square$

Proof of Theorem A.1. **Albert: Review: AI written** We modify the proof of Theorem 5.8. The IOR Π_1 now consists of Rounds 0, 1 and 2, and outputs $(\mathbf{x}' = (g^\gamma, g^\mu, \zeta, y), \mathbf{y}' = f, \mathbf{w}' = \pi_2(\mathbf{f}))$: it is a 3-round IOR from $\mathcal{R}_{\text{LinBitsRings}}$ to $\mathcal{R}_{\text{PESAT}}^{\zeta,y}$ whose first prover message and last verifier message are empty, and it is perfectly complete because the honest y equals $\widetilde{\text{Pack}_\nu \circ \pi_2(\mathbf{f})}(\zeta)$. The IORs Π_2 and Π_3 are the variants of Π_{red} and Π_{iopp} for $\mathcal{R}_{\text{PESAT}}^{\zeta,y}$ and $\mathcal{R}_{\text{Lin}}^{\zeta,y}$ from the hypotheses; the round count and the composition are as before. It remains to show that Π_1 has round-by-round knowledge soundness with relaxation $(\mathcal{R}_{\text{LinBitsRings}}^\delta, \mathcal{R}_{\text{PESAT}}^{\delta,\zeta,y})$ and errors $(\kappa_{\text{OOD}}, \kappa_0, 0)$.

Let KState be the knowledge state of the proof of Theorem 5.8, and for a transcript tr of Π_1 let $\bar{\text{tr}}$ denote tr with the messages of Round 0 removed. Write Ev for the condition $\text{Pack}_\nu \circ \pi_2(\mathbf{w})(\zeta) = y$ on $\mathbf{w}$, and Coll for the event $\text{Coll}(f, \zeta)$ of Lemma A.2. We keep the identity extractor and define $\text{KState}'(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w})$ as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w})$ if tr is empty or $\text{tr} = (\perp)$; as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \vee \text{Coll}$ if $\text{tr} = (\perp, \zeta)$; as $(\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \wedge \text{Ev}) \vee \text{Coll}$ if tr ends with (y, μ') or with (q, π_{rand}) ; and as $\text{KState}(\mathbf{x}, \mathbf{y}, \bar{\text{tr}}, \mathbf{w}) \wedge \text{Ev}$ if tr contains μ . The conditions of Definition 4.5 hold: at the empty transcript because $\bar{\text{tr}}$ is empty, at full transcripts because $\mathcal{R}_{\text{PESAT}}^{\delta,\zeta,y}$ is $\mathcal{R}_{\text{PESAT}}^\delta$ together with Ev , and at prover moves because KState never increases at a prover move, adding the condition Ev only restricts, and Coll is never introduced by a prover message. Since KState' can only change from 0 to 1 at the message ζ through Coll , the error at ζ is at most $\Pr_\zeta[\text{Coll}] \leq \kappa_{\text{OOD}}$ by Lemma A.2; the error at the empty last verifier message is 0. For the error at (q, π_{rand}) : a witness $\mathbf{w}$ with

$\text{KState}' = 0$ before this message and $\text{KState}' = 1$ after it satisfies $\neg \text{Coll}$, Ev , $(\mathbb{x}, \mathbb{y}, \mathbf{w}) \notin \mathcal{R}_{\text{LinBitsRings}}^\delta$ and $\text{KState}(\mathbb{x}, \mathbb{y}, (\mu', (q, \pi_{\text{rand}})), \mathbf{w}) = 1$. In particular $\pi_2(\mathbf{w}) \in \text{List}(f, \delta)$ and, as Coll fails, it is the unique element of the list with evaluation y at ζ ; hence $\mathbf{w}$, and with it the polynomial h of the proof of [Theorem 5.8](#), are determined by (f, ζ, y, μ') before (q, π_{rand}) is drawn, and the first statement of that proof bounds the probability of this event by κ_0 . The case where Round 1 is skipped is handled as there. $\square$

B Ring switching via Galois orbits

Albert: Context: how does it relate to other references?

Let $b_1, \dots, b_m \in \mathbb{F}_2^{2^n}$ **Albert:** Use boldface. Using an $\mathbb{F}_2$ -basis $\beta_1, \dots, \beta_m$ of $\mathbb{F}_{2^m}$ we pack m inner products

$$\langle b_i, \mathbf{v} \rangle = \sum_{j \in \{0,1\}^n} b_{i,j} \cdot \mathbf{v}_j, \quad i = 1, \dots, m,$$

coefficient-wise into a single inner product

$$\begin{aligned} b &:= b_1 \cdot \beta_1 + \dots b_m \cdot \beta_m \in \mathbb{F}_{2^m}^{2^n} \\ \langle b, \mathbf{v} \rangle &= \langle b_1, \mathbf{v} \rangle \cdot \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \beta_m \\ &= \sum_{j \in \{0,1\}^n} (b_{1,j} \cdot \beta_1 + \dots + b_{m,j} \cdot \beta_m) \cdot \mathbf{v}_j. \end{aligned}$$

The Frobenius automorphism $\phi(z) = z^2$ keeps $\mathbb{F}_2 \subset \mathbb{F}_{2^m}$ fixed, and has no other fixed points. We extract the value of $\langle b_i, \mathbf{v} \rangle$ from the values of $\langle b, \mathbf{v} \rangle$ along the Galois orbit

$$\mathbf{v}, \mathbf{v}^2, \mathbf{v}^{2^2}, \dots, \mathbf{v}^{2^{m-1}},$$

as follows, where by $\mathbf{v}^t$ we mean that each component of $\mathbf{v}$ is raised to the power t . The Moore matrix

$$M = \begin{pmatrix} \beta_1 & \beta_2 & \dots & \beta_m \\ \beta_1^2 & \beta_2^2 & \dots & \beta_m^2 \\ \beta_1^{2^2} & \beta_2^{2^2} & \dots & \beta_m^{2^2} \\ \vdots & & & \\ \beta_1^{2^{m-1}} & \beta_2^{2^{m-1}} & \dots & \beta_m^{2^{m-1}} \end{pmatrix}$$

consists of the Galois orbits of the basis vectors, and is invertible. (See below.) Using that the component polynomials are over $\mathbb{F}_2$, we see that

$$\phi^k \left(\langle b, \phi^{-k}(\mathbf{v}) \rangle \right) = \langle b_1, \mathbf{v} \rangle \cdot \phi^k \beta_1 + \dots + \langle b_m, \mathbf{v} \rangle \cdot \phi^k \beta_m,$$

for any k . To this end, observe that

$$M \cdot \begin{pmatrix} \langle b_1, \mathbf{v} \rangle \\ \langle b_2, \mathbf{v} \rangle \\ \langle b_3, \mathbf{v} \rangle \\ \vdots \\ \langle b_m, \mathbf{v} \rangle \end{pmatrix} = \begin{pmatrix} \langle b, \mathbf{v} \rangle \\ \phi \left(\langle b, \phi^{-1}(\mathbf{v}) \rangle \right) \\ \phi^2 \left(\langle b, \phi^{-2}(\mathbf{v}) \rangle \right) \\ \vdots \\ \phi^{m-1} \left(\langle b, \phi^{-(m-1)}(\mathbf{v}) \rangle \right) \end{pmatrix}.$$

Multiplying both sides by M^{-1} (M is a Moore matrix and hence invertible because the $\beta_1, \dots, \beta_m$ are invertible) we obtain an expression of the values $\langle b_i, \mathbf{v} \rangle$ as a linear expression of the values $\phi^k(\langle b, \phi^{-k}(\mathbf{v}) \rangle)$.

Now one can prove the inner product claims $\langle b, \phi^{-i}(\mathbf{v}) \rangle$ for all i by first batching them all into one single claim on the value of $\langle b, \mathbf{u} \rangle$, and then proving this later claim normally, since we had committed to the vector b initially. The batching step introduces the error term

$$1/|\mathbb{F}_{2^\nu}|.$$

Ulrich: Invertibility of M follows from that of M^t : any linear functional $(c_1, \dots, c_m)$ in the kernel of M^t gives rise to the linearizable polynomial

$$p(X) = \sum_{i=0}^{m-1} c_i \cdot X^{2^i}$$

which has $\beta_1, \dots, \beta_m$, and hence the whole space F as roots. By dimension, $p(X)$ must be trivial, meaning that all its coefficients are zero.